

SnowPro Core Certification



SnowPro Core Certification

EXAM FORMAT FOR CORE CERTIFICATION

©Learn Anywhere

Exam Version:	COF-C02
Total Number of Questions:	100
Question Types:	Multiple Select, Multiple Choice
Time Limit:	115 minutes
Languages:	English and Japanese
Passing Score:	750 + Scaled Scoring from 0 - 1000



SnowPro Core Certification

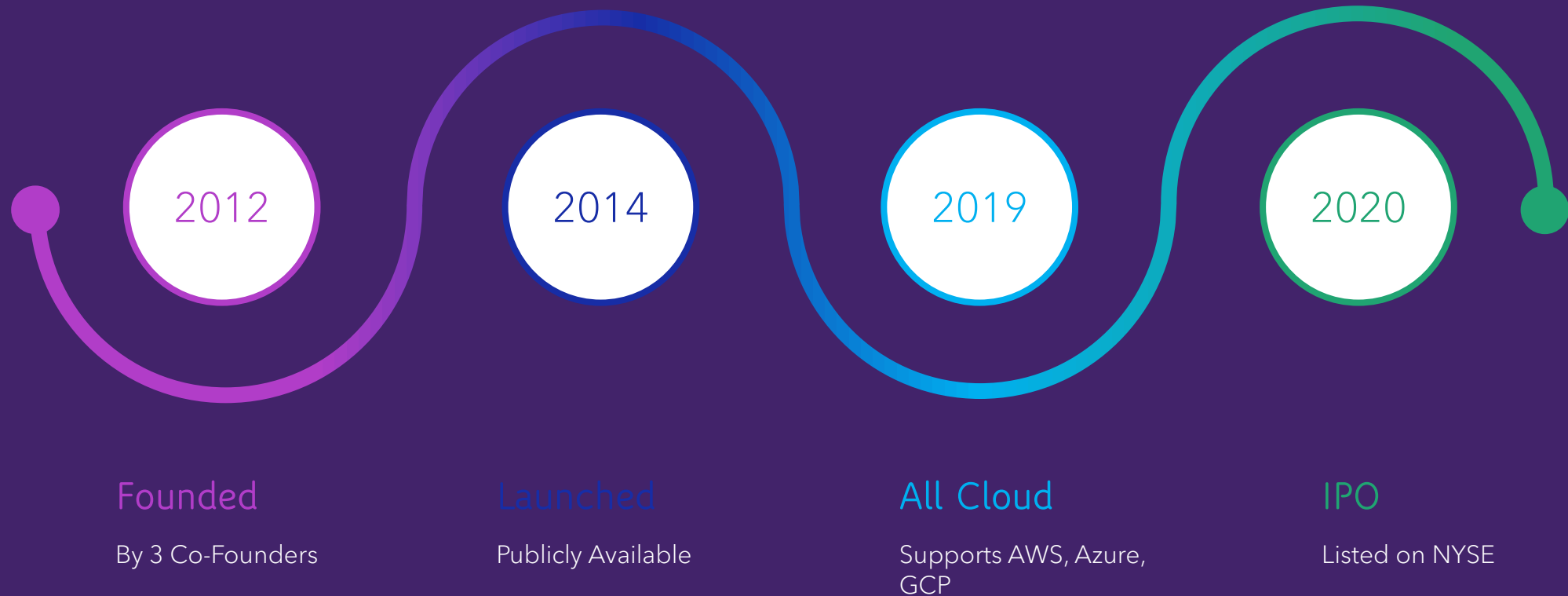
Domain	Domain Weightings
1.0 Snowflake AI Data Cloud Features and Architecture	24%
2.0 Account Access and Security	18%
3.0 Performance and Cost Optimization Concepts	16%
4.0 Data Loading and Unloading	12%
5.0 Data Transformations	18%
6.0 Data Protection and Data Sharing	12%



Snowflake History



Snowflake History



What is Snowflake?



What is Snowflake?

- Cloud based data platform
- Not built from old db tech
- Cloud native
- Separates Compute from Storage
- It is SaaS



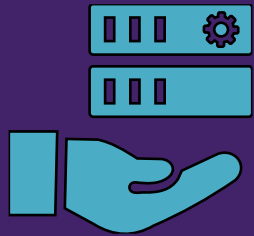
Software as a Service (SaaS)



Software as a Service (SaaS)



Quick setup and with minimal upfront cost.



It is fully managed and eliminates the need for customer-maintained infrastructure.



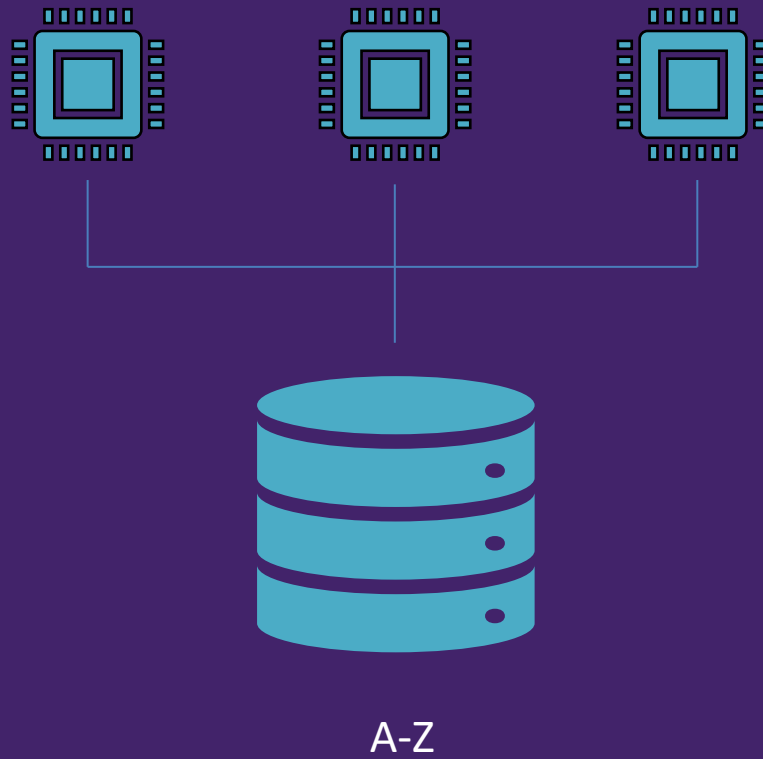
Snowflake uses a subscription-based pricing model.



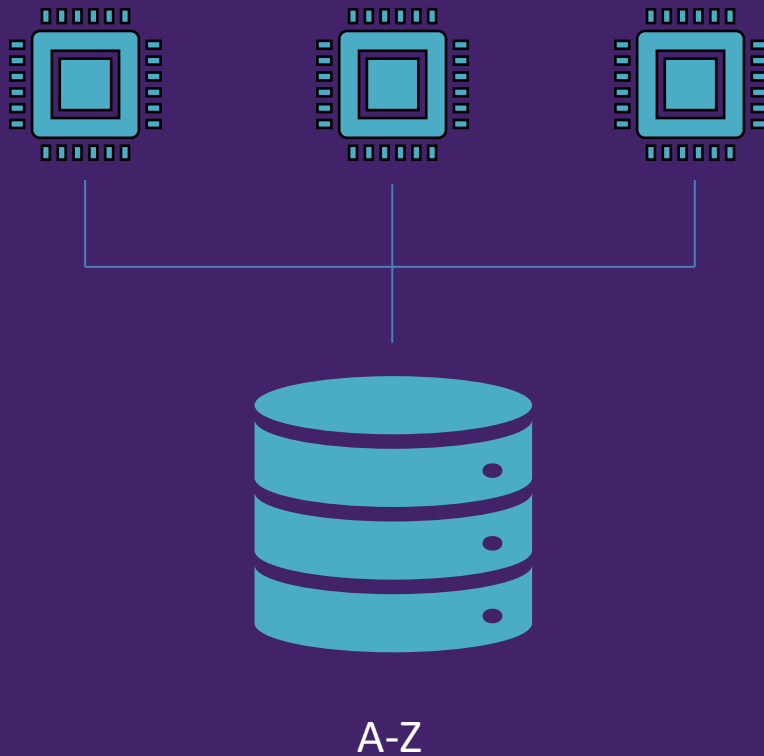
Traditional Architecture



Shared Disk Architecture



Shared Disk Architecture



Pros :

- High data availability and reliability
- Easy data sharing between nodes
- Dynamic Load balancing for mixed workloads

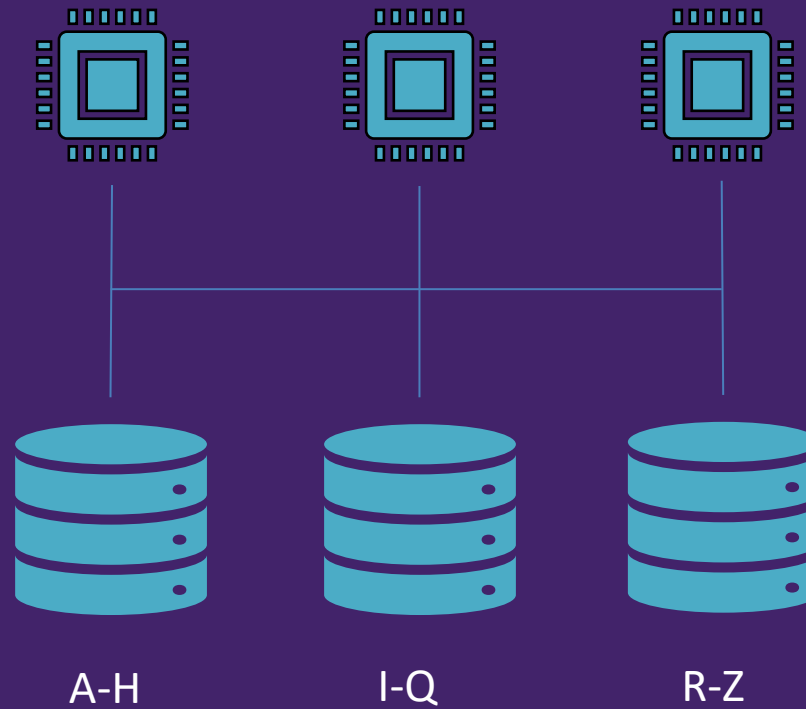
Cons:

- Single point of failure as shared resource.
- Performance issues as nodes/Volume increases.
- May have Network bottlenecks

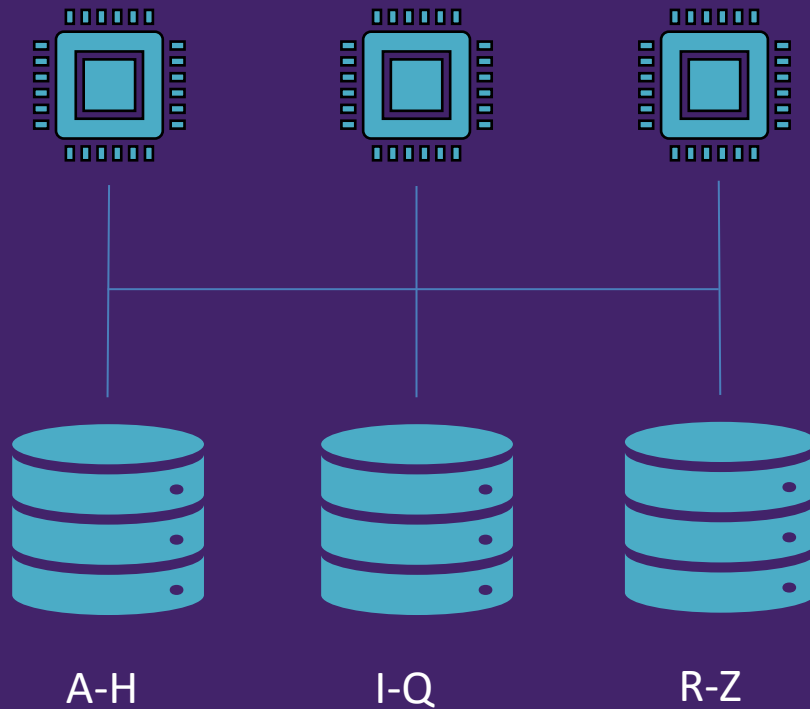
Oracle Real Application Clusters



Shared Nothing Architecture



Shared Nothing Architecture



Pros :

- Highly scalable architecture
- Better performance due to reduced network traffic
- Flexible HW and SW configurations

Cons:

- Data partitioning can be time-consuming
- Fixed Load balancing
- Can lead to data skew

Apache Cassandra, Teradata

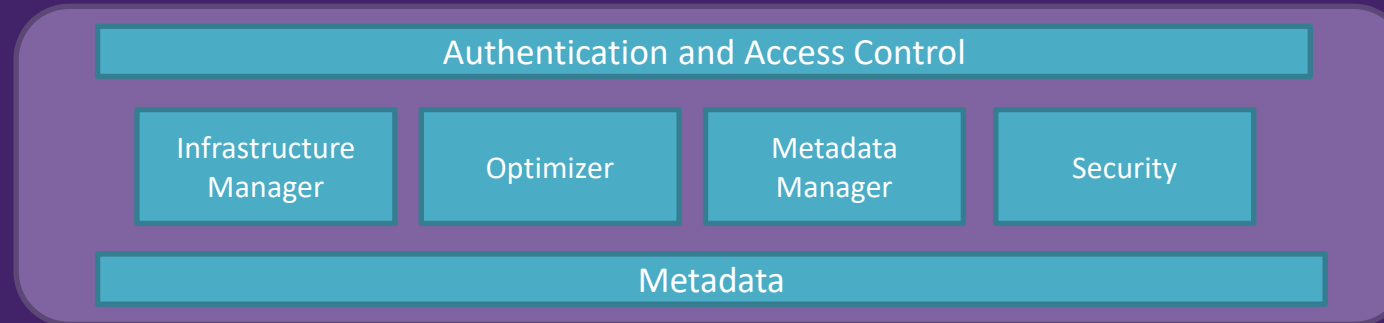


Snowflake Architecture

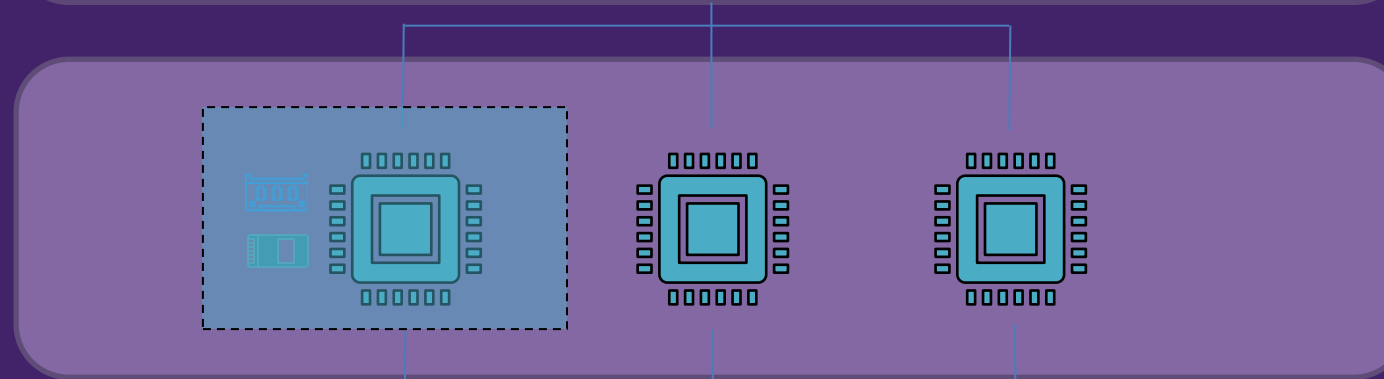


Snowflake Architecture

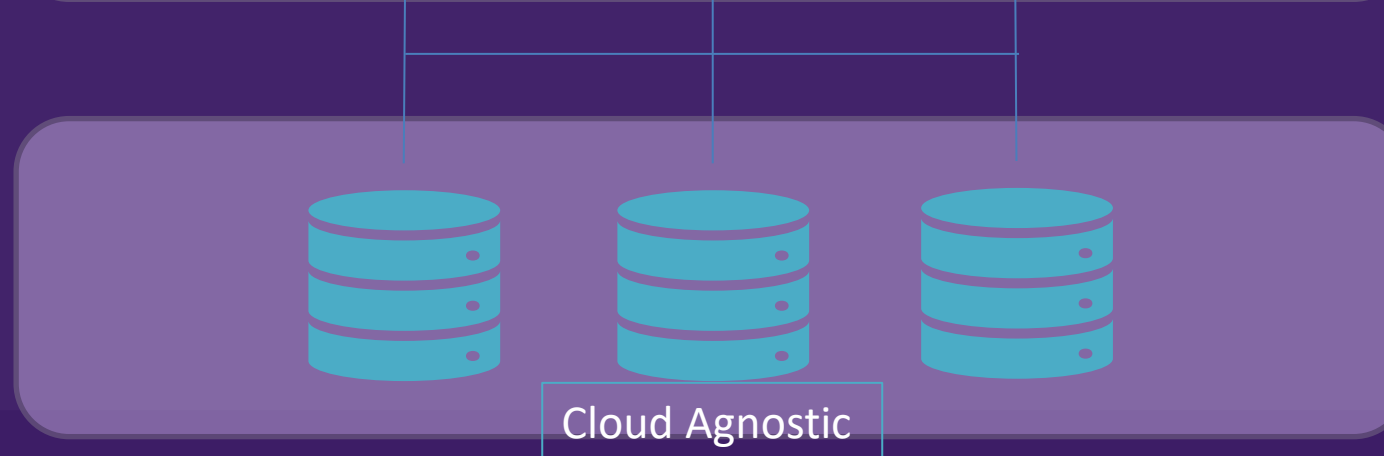
Cloud Services



Query Processing Layer



Database Storage



Snowflake Architecture

Database Storage

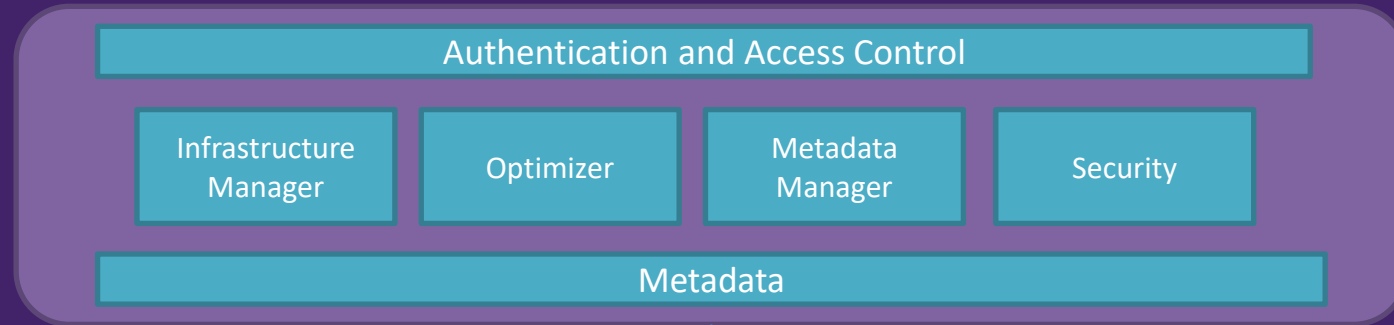


- Snowflake organizes data into optimized, Compressed and columnar format.
- Data is stored in cloud storage.
- Snowflake stores data in centralized DB, accessible to all nodes.
- Data objects are not directly accessible.
- Accessible only through Snowflake SQL queries.

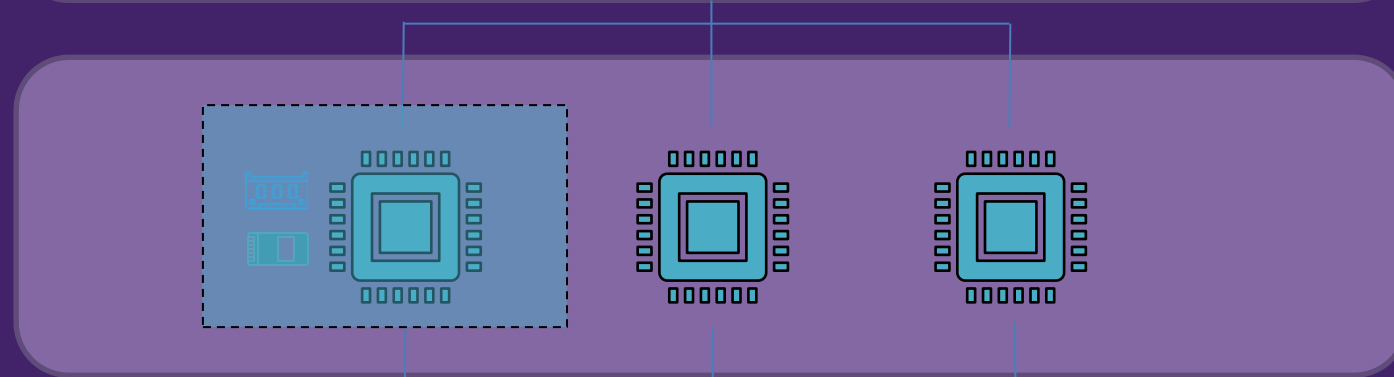


Snowflake Architecture

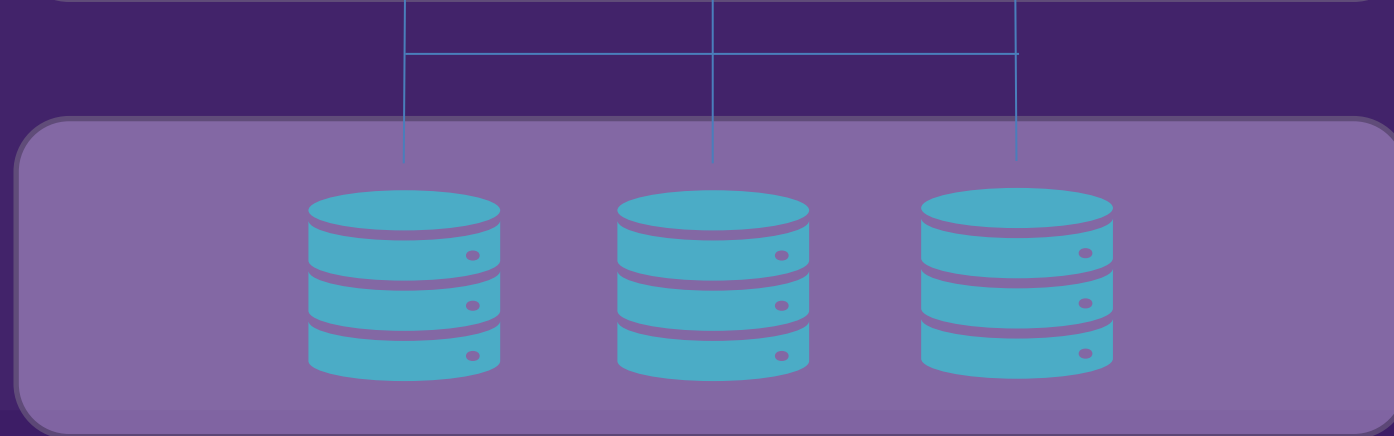
Cloud Services



Query Processing Layer

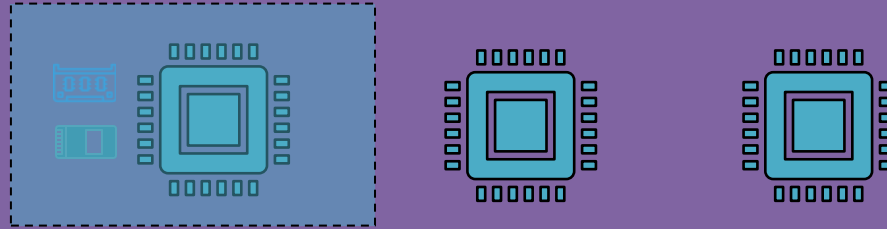


Database Storage



Snowflake Architecture

Query Processing Layer

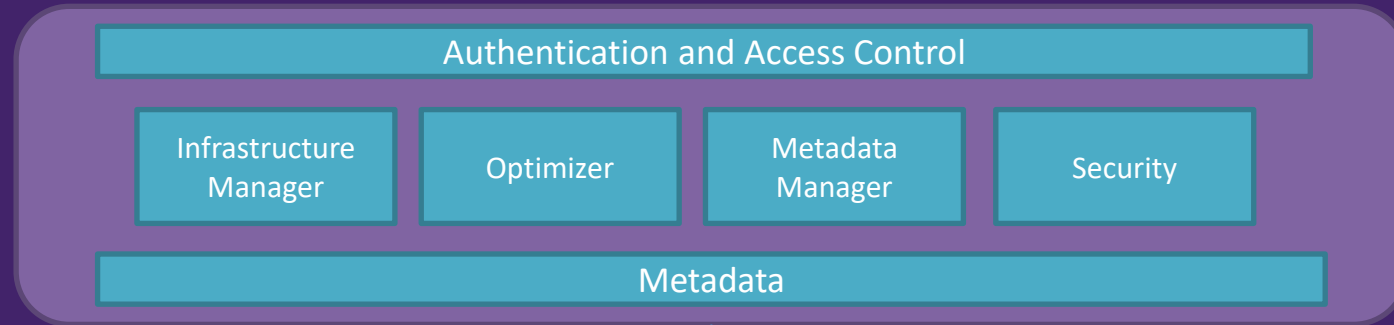


- Queries executed in the processing layer.
- Snowflake uses "virtual warehouses" for processing.
- Each VW is MPP cluster
- Each VW has its own temp memory and cache.
- Each warehouse is independent, with no impact on other warehouses performance.
- Each query result remains in cache, and purged after a point.

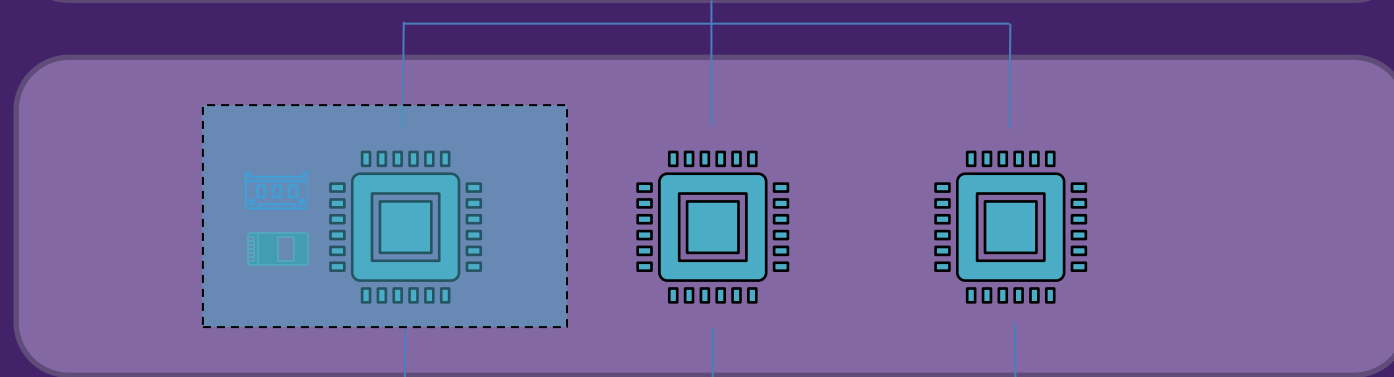


Snowflake Architecture

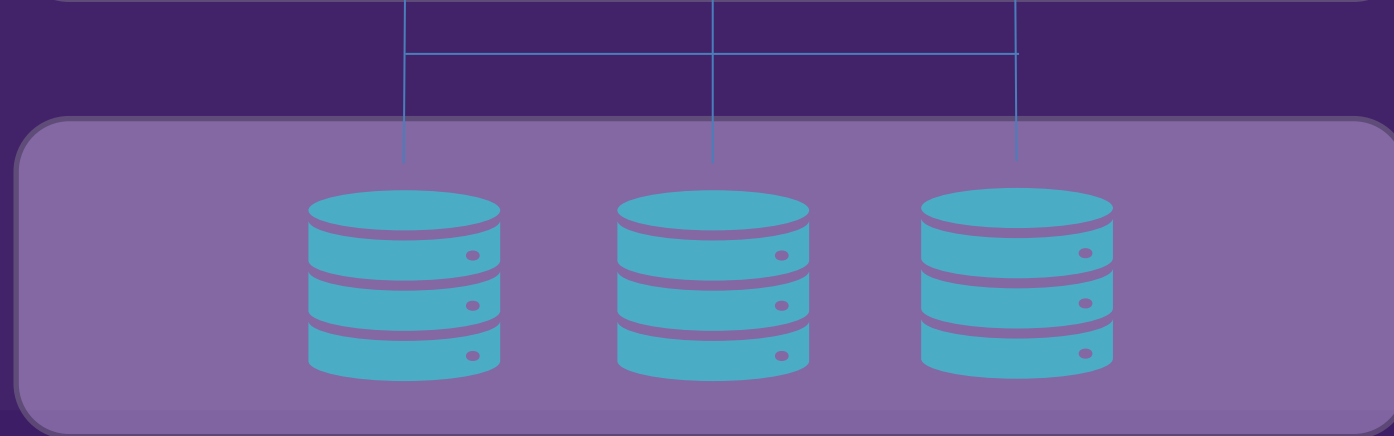
Cloud Services



Query Processing Layer

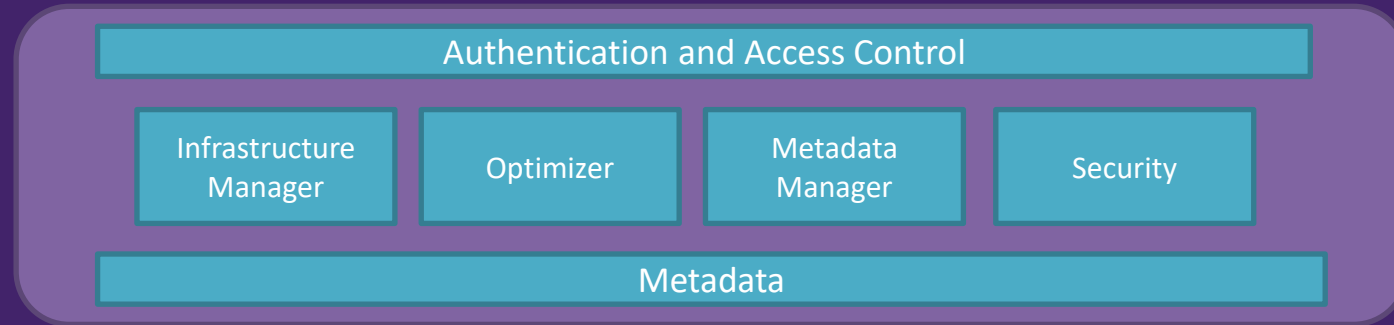


Database Storage



Snowflake Architecture

Cloud Services

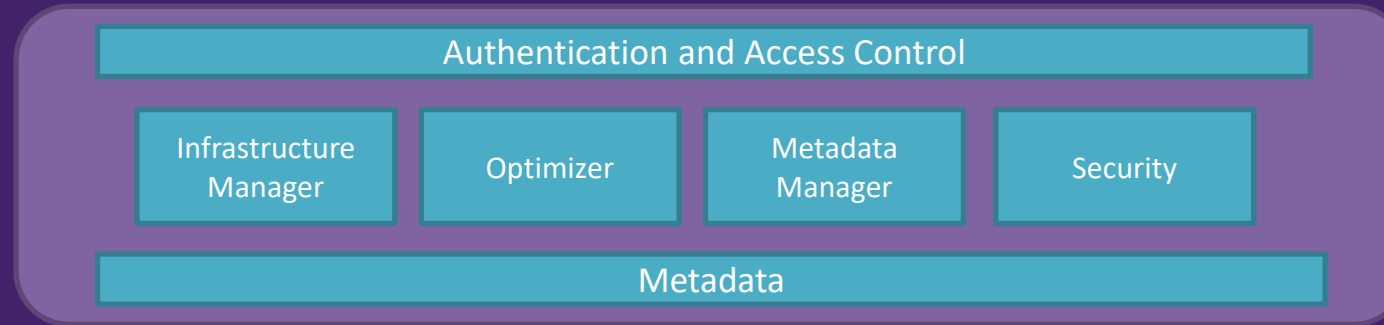


- Snowflake cloud services coordinate platform activities.
- Manage user requests from login to query dispatch.
- Responsible for security, access management, monitoring and other operational tasks



Snowflake Architecture

Cloud Services

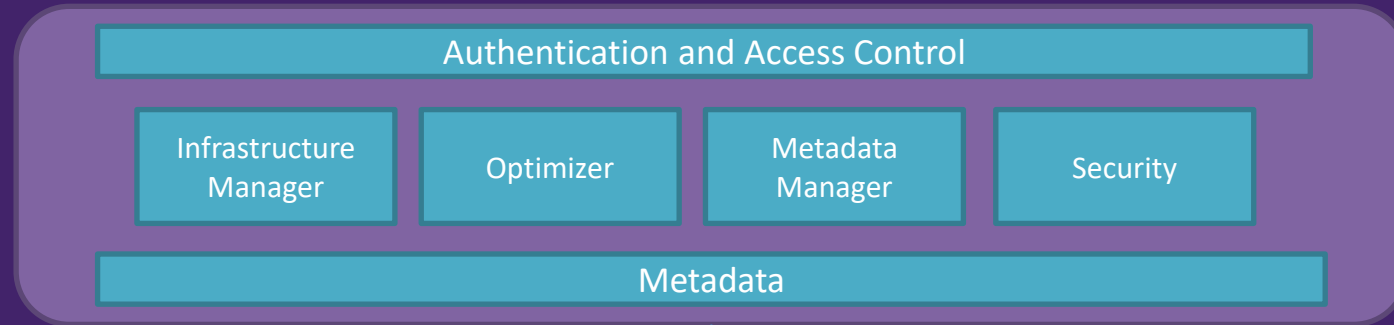


- **Authentication and access control** verifies user identity and manages access to data.
- **Infrastructure manager** provisions and manages compute resources for Snowflake.
- **Snowflake optimizer** determines the most efficient way to execute a query.
- **Metadata Manager** manages the metadata associated with data stored in Snowflake.
- **Snowflake Security** includes encryption, network isolation, and other security features to protect data.

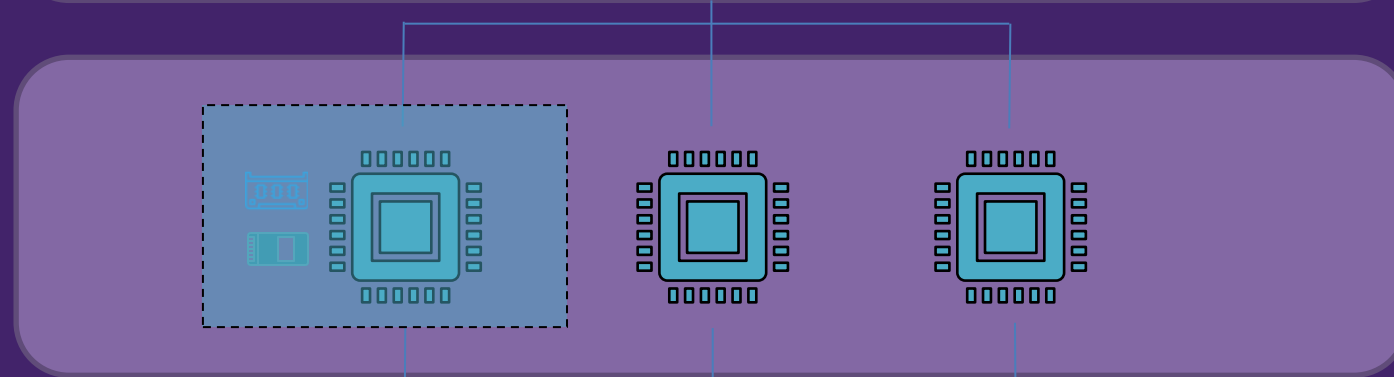


Snowflake Architecture

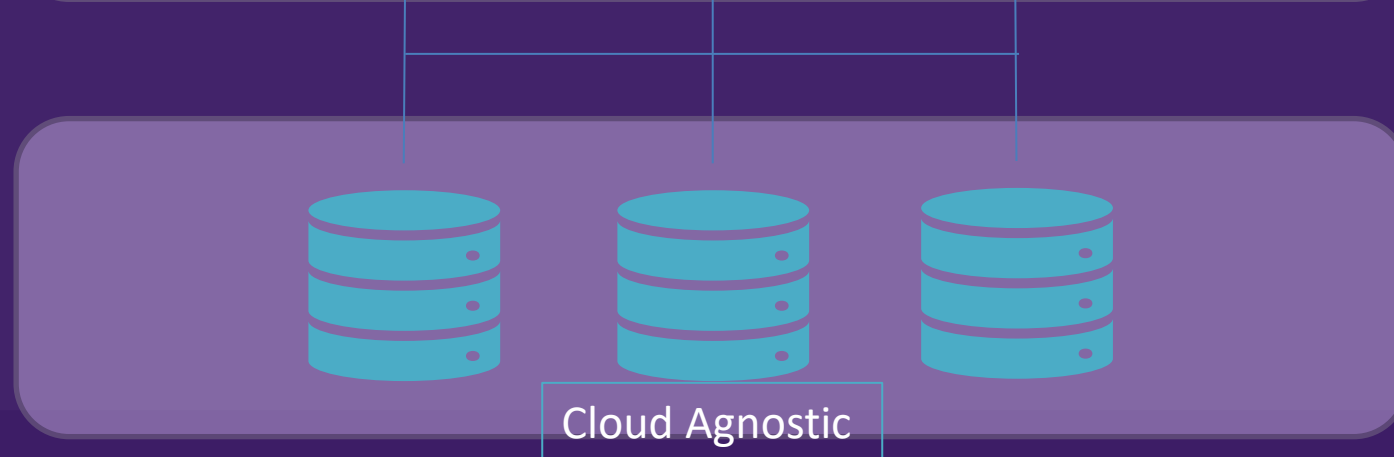
Cloud Services



Query Processing Layer



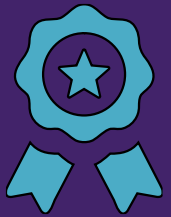
Database Storage



Snowflake Editions



Snowflake Editions



Standard



Enterprise



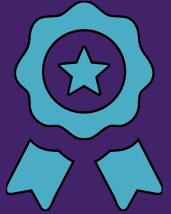
Business Critical



VPS



Snowflake Editions



Standard



Enterprise



Business Critical

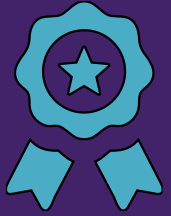


VPS

Standard	Enterprise	Business Critical	VPS
All Standard Features	All Standard Features	All features of Enterprise	Most features of Business Critical
	Multi Clustered VW	Failover and DR	Highest level of Security
	Materialized View, Search Optimization	Redirecting Client Connection	Separate Environment
	Extended Time Travel	HIPAA, HITRUST CSF Environment	



Snowflake Editions



Standard



Enterprise



Business Critical

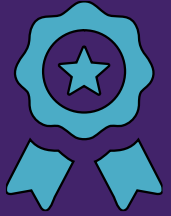


VPS

Standard	Enterprise	Business Critical	VPS
All Standard Features	All Standard Features	All features of Enterprise	Most features of Business Critical
	Multi Clustered VW	Failover and DR	Highest level of Security
	Materialized View, Search Optimization	Redirecting Client Connection	Separate Environment
	Extended Time Travel	HIPAA, HITRUST CSF Environment	



Snowflake Editions



Standard



Enterprise



Business Critical



VPS



Google Cloud



Azure

North & South America



Europe & Middle East



Asia Pacific



Virtual Warehouse



Virtual Warehouse

New Warehouse

Creating as ACCOUNTADMIN

Name

Size ?

X-Small 1 credit/hour

Comment (optional)

Query Acceleration

Accelerate outlier queries with additional flexible compute resources

Multi-cluster Warehouse

Scale compute resources as query needs change

Advanced Warehouse Options

Cancel

Create Warehouse

Size ?

X-Small 1 credit/hour

X-Small 1 credit/hour

Small 2 credits/hour

Medium 4 credits/hour

Large 8 credits/hour

X-Large 16 credits/hour

2X-Large 32 credits/hour

3X-Large 64 credits/hour

4X-Large 128 credits/hour

5X-Large 256 credits/hour

6X-Large 512 credits/hour

Query Acceleration

Accelerate outlier queries with additional flexible compute resources

Scale Factor ?

No Limit



Virtual Warehouse

Warehouse Size	Credits / Hour	Credits / Second	Notes
X-Small	1	0.0003	Default size for warehouses created using <code>CREATE WAREHOUSE</code> .
Small	2	0.0006	
Medium	4	0.0011	
Large	8	0.0022	
X-Large	16	0.0044	Default for warehouses created in the web interface.
2X-Large	32	0.0089	
3X-Large	64	0.0178	
4X-Large	128	0.0356	
5X-Large	256	0.0711	Generally available in Amazon Web Services regions, and in preview in US Government and Azure regions.
6X-Large	512	0.1422	



Virtual Warehouse

1 ▾

✓ 1
2
3
4
5
6
7
8
9
10

Multi-cluster Warehouse

Scale compute resources as query needs change

Mode

Auto-scale ▾

Min Clusters

1 ▾

Max Clusters

1 ▾

Scaling Policy

Standard ▾

Auto-scale ▾

✓ Auto-scale
Maximized

Standard ▾

✓ Standard
Economy

Advanced Warehouse Options ^

Auto Resume

Auto Suspend

Suspend After (min)

10

Warehouse Type ?

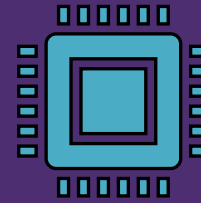
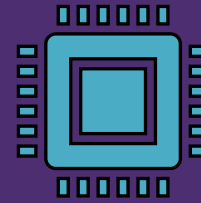
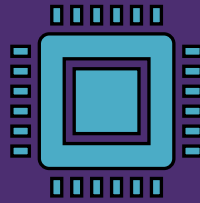
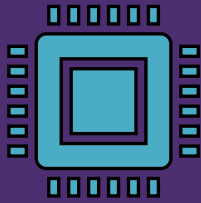
Standard ▾



Scaling Out VS Scaling Up



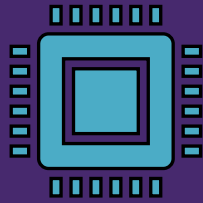
Scale Out



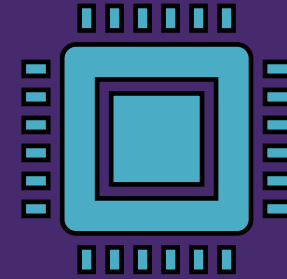
- Can be automated
- Used when you have concurrent Queries
- Soft limit of Max 10 cluster



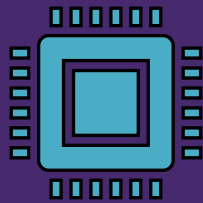
Scale Up/Down



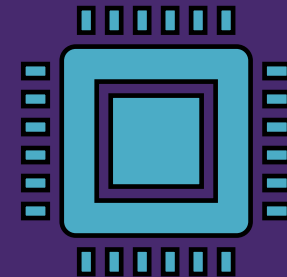
Small VW



BIG VW



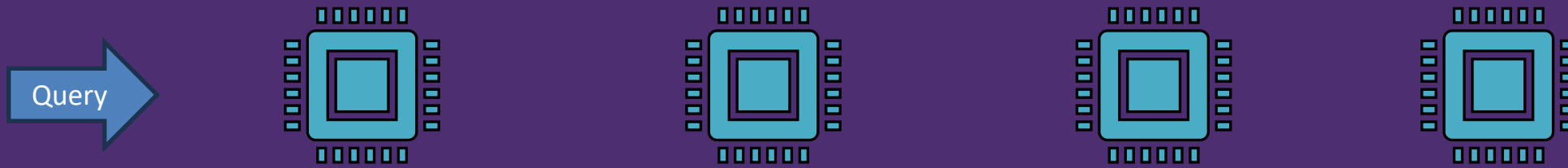
Small VW



BIG VW



Standard Scaling Policy



Warehouse Starts

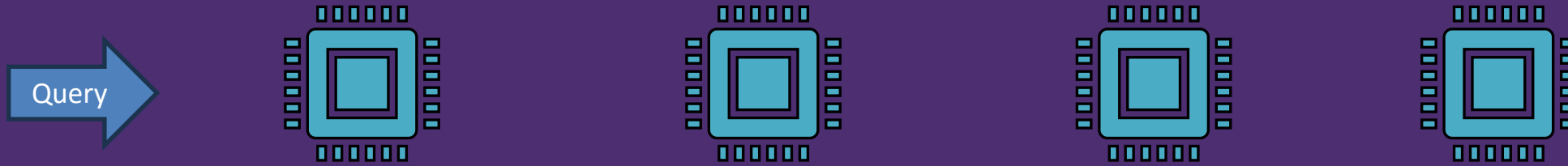
- Starts cluster when query is queued
- Each new cluster waits for 20 second after the prior one started

Warehouse Shuts

- Cluster shuts, when load can be redistributed from least-loaded cluster to others
- After 2-3 consecutive checks and 1 minute interval



Economy Scaling Policy



Warehouse Starts

- Starts cluster when enough load to run for 6 minute

Warehouse Shuts

- Cluster shuts, when load can be redistributed from least-loaded cluster to others
- After 5-6 consecutive checks and 1 minute interval

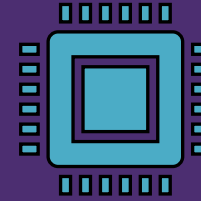
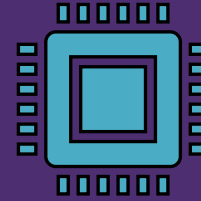
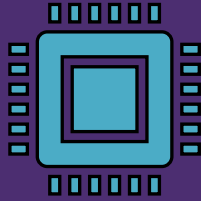
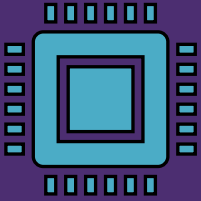
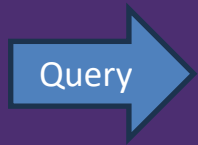


Scaling Policy

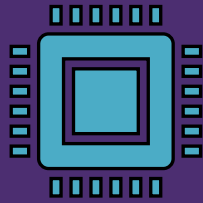
Policy	Description	Warehouse Starts...	Warehouse Shuts Down...
Standard (default)	Prevents/minimizes queuing by favoring starting additional clusters over conserving credits.	The first cluster starts immediately when either a query is queued or the system detects that there's one more query than the currently-running clusters can execute. Each successive cluster waits to start 20 seconds after the prior one has started. For example, if your warehouse is configured with 10 max clusters, it can take a full 200+ seconds to start all 10 clusters.	After 2 to 3 consecutive successful checks (performed at 1 minute intervals), which determine whether the load on the least-loaded cluster could be redistributed to the other clusters without spinning up the cluster again.
Economy	Conserves credits by favoring keeping running clusters fully-loaded rather than starting additional clusters, which may result in queries being queued and taking longer to complete.	Only if the system estimates there's enough query load to keep the cluster busy for at least 6 minutes.	After 5 to 6 consecutive successful checks (performed at 1 minute intervals), which determine whether the load on the least-loaded cluster could be redistributed to the other clusters without spinning up the cluster again.



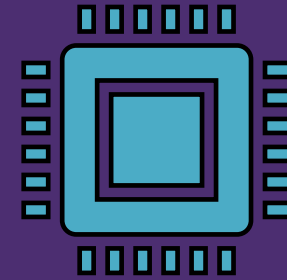
Economy Scaling Policy



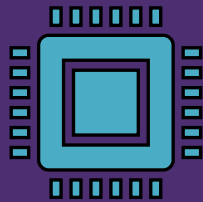
Scale Up/Down



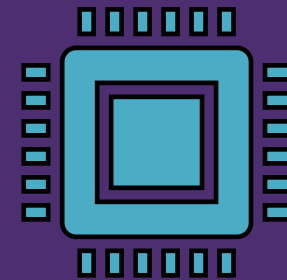
Small VW



BIG VW



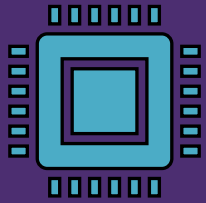
Small VW



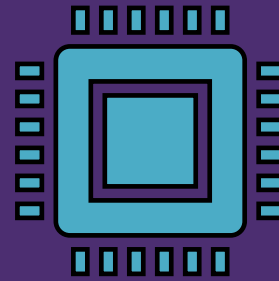
BIG VW



Scale Up/Down

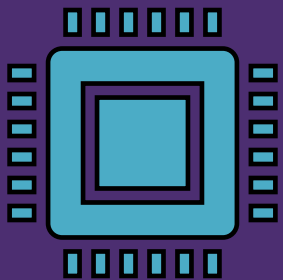


Small VW

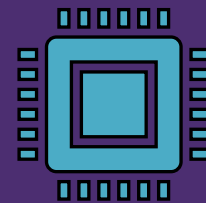


BIG VW

- Not automatic
- Used when we have complex queries



BIG VW



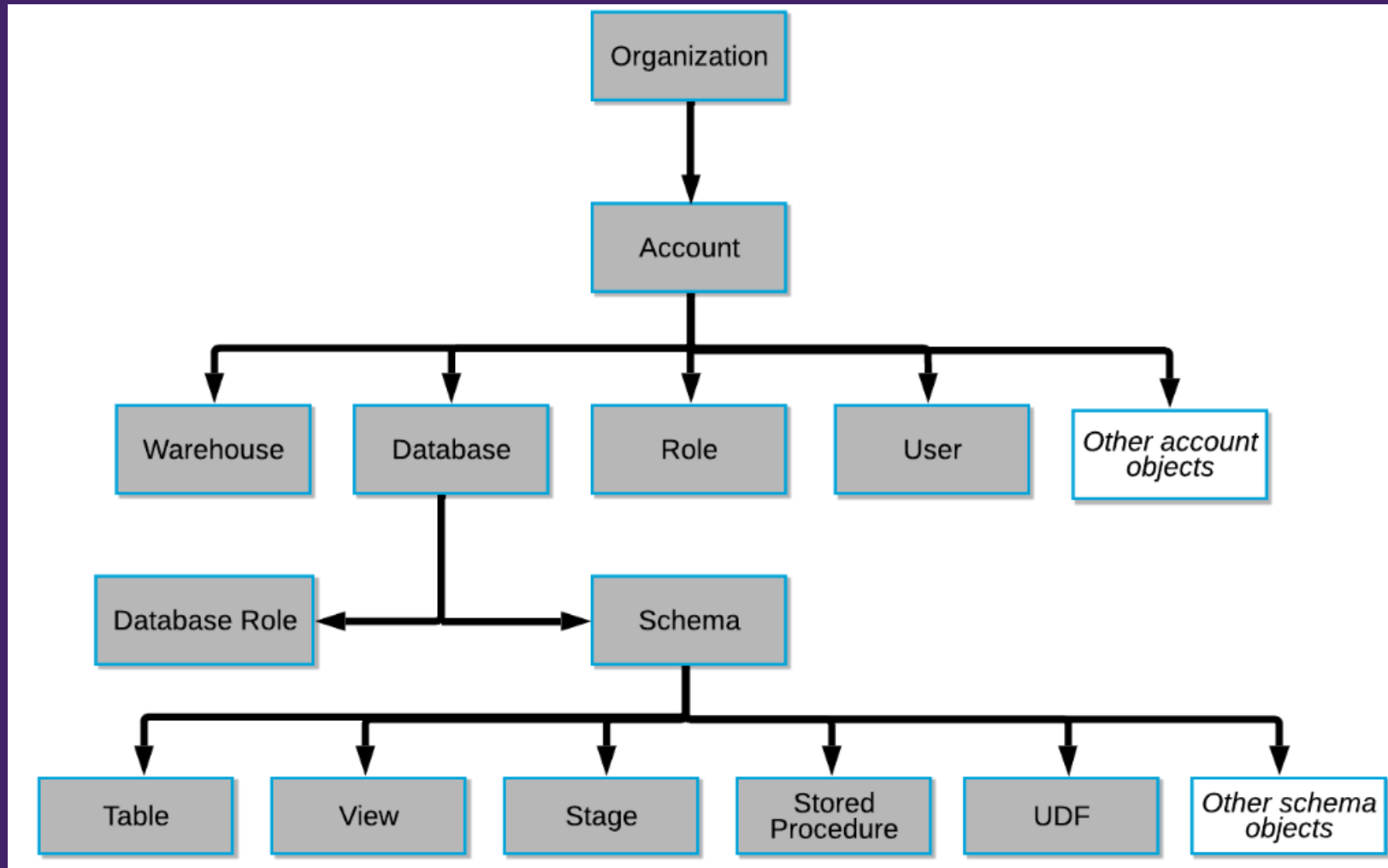
Small VW



Hierarchy Of Containers



Hierarchy Of Containers



Data Types



Numeric

- Number
- Decimal
- Numeric
- INT, FLOAT,
- DOUBLE



String/Binary

- VARCHAR
- CHAR
- STRING,TEXT
- BINARY



Logical

- BOOLEAN



Date & Time

- DATE
- DATETIME
- TIMESTAMP



Geospatial

- GEOGRAPHY
- GEOMETRY



Semi-Structured

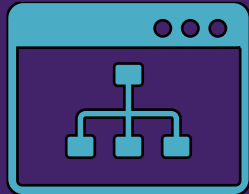
- VARIANT
- OBJECT
- ARRAY



Database & Schemas



Database : A database is a logical grouping of schemas. Each database belongs to a single Snowflake account.

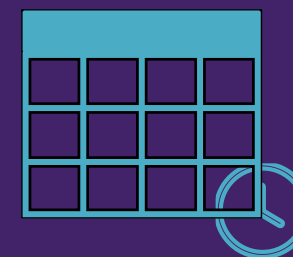
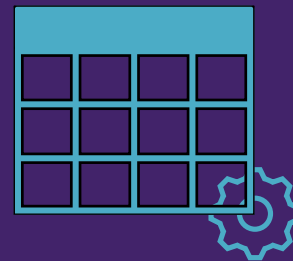


Schema : A schema is a logical grouping of database objects (tables, views, etc.). Each schema belongs to a single database



Tables

Types of Tables



Features	Permanent	Transient	Temporary
Persistence	Until Dropped	Until Dropped	Session
Cloning	To Any type	To Transient or Temporary	To Transient or Temporary
Time Travel (Days, Default is 1)	 0-1 (0-90) 	0-1	0-1
Fail Safe (Days, Fixed)	7	0	0

* External table, none of the above criteria applies.

Continuous Data Protection Lifecycle



Continuous Data Protection Lifecycle



Perform All Standard Operations

AT|BEFORE

TIMESTAMP => <timestamp>

OFFSET => <time_difference>

STATEMENT => <id>

UNDROP

No user operations allowed.
(Recoverable by Snowflake Staff)



Permanent Table : 7 Days

Period Retention is configurable at
account level with

`DATA_RETENTION_TIME_IN_DAYS`

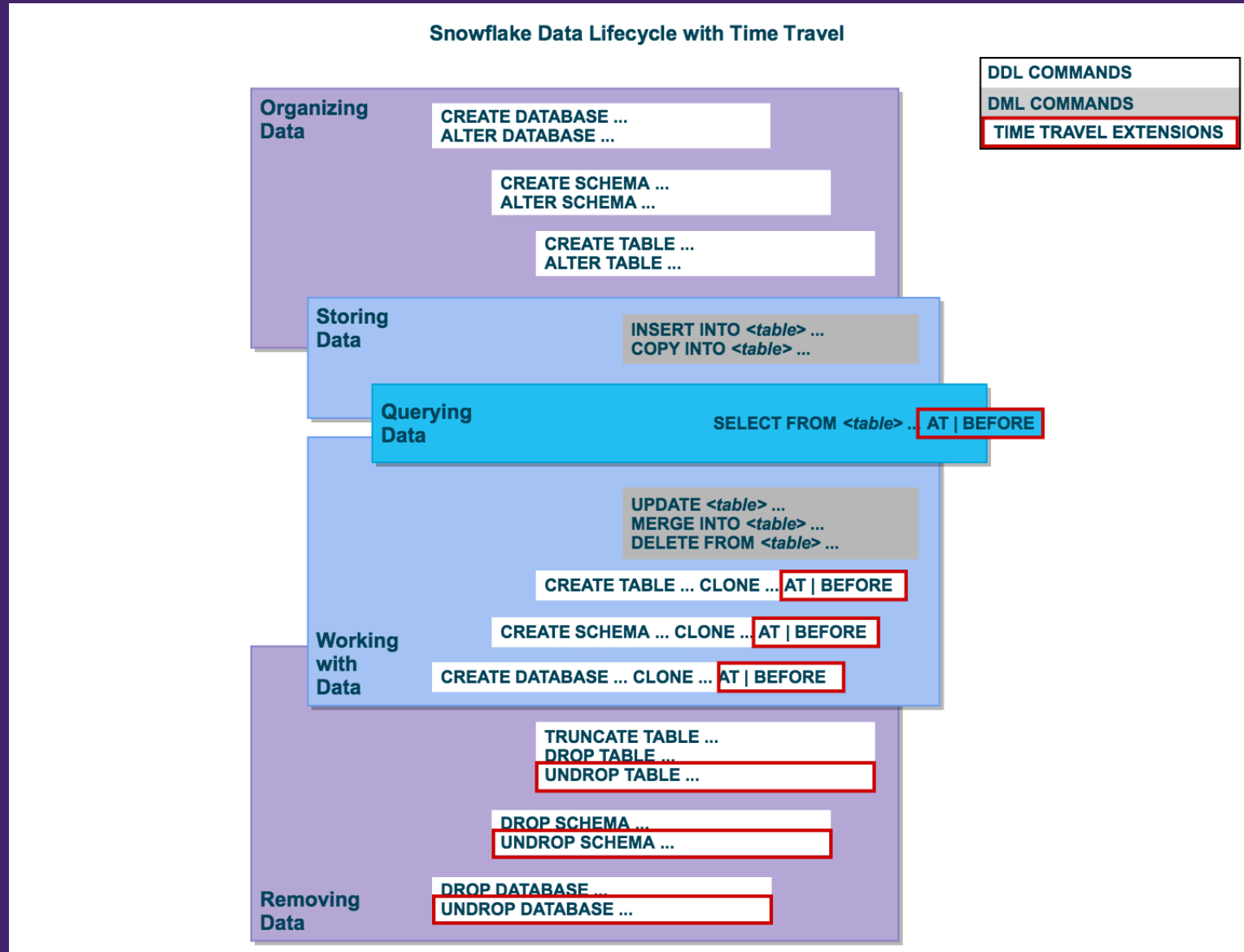


0-1 Standard Edition

0-90 Enterprise+ Edition



Snowflake Data Lifecycle with Time Travel



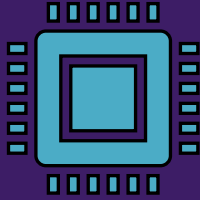
Source : Snowflake



Snowflake Billing



Snowflake Billing



Compute

- User Managed VW
- Snowflake Managed VW
- Cloud Services



Storage

- Data Staging
- Database Tables
- Materialized View
- Time Travel and Fail Safe

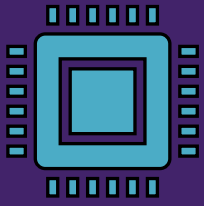


Data Transfer Cost

- Data Transfer

Billing depends on Snowflake Edition, Cloud provider and Region
Check <https://www.snowflake.com/pricing/>





Compute

Snowflake Billing

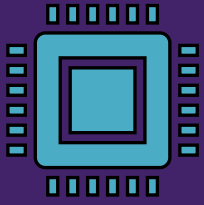
User Managed VW

- A virtual warehouse is one or more clusters of compute resources that enable **executing queries, loading data, and performing other DML operations**.
- Snowflake **credits** are used to pay for the processing time used by each virtual warehouse.

Based on Size of VW and How long they Run, not on complexity of the Query

Type of Virtual Warehouse	X-Small	Small	Medium	Large	X-Large	2X-Large	3X-Large	4X-Large	5X-Large	6X-Large
Standard	1	2	4	8	16	32	64	128	256	512
Snowpark-optimized	n/a	n/a	6	12	24	48	96	192	384	768





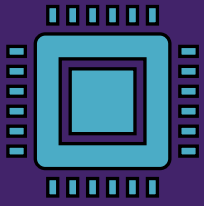
Compute

Snowflake Billing

User Managed VW

- Billed By Seconds
- 60 Sec minimum
- When Scale Up, Charged only for extra compute
- When Scale Down, Charged for both, for brief period





Compute

Snowflake Billing

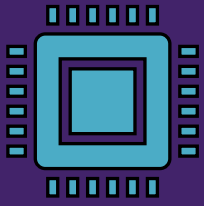
Snowflake VW / Serverless

- Compute resource provided and Managed resources
- Snowflake **credits** are used to pay for the processing time used by each virtual warehouse.

Serverless Features%

Feature	Required Compute	Additional Information
Automatic Clustering	Automated background maintenance of each clustered table, including initial clustering and reclustered as needed.	Automatic Clustering Cost
External Tables	Automated refreshing of the external table metadata with the latest set of associated files in the external stage and path.	External Tables Cost
Materialized Views	Automated background synchronization of each materialized view with changes in the base table for the view.	Materialized Views Cost
Query Acceleration Service [1]	Execution of portions of eligible queries.	Query Acceleration Service Cost
Replication	Automated copying of data between accounts, including initial replication and maintenance as needed.	Database Replication Cost Replication Group Cost
Search Optimization Service	Automated background maintenance of the search access paths used by the search optimization service.	Search Optimization Service Cost
Snowpipe	Automated processing of file loading requests for each pipe object.	Snowpipe Cost
Tasks	Execution of SQL code.	Tasks Cost





Compute

Snowflake Billing

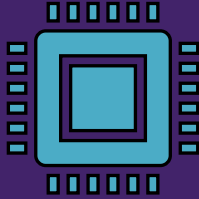
Cloud Services

- Authentication, enforces security, performs query compilation and optimization, handles request query caching etc.
- Snowflake **credits** are used to pay for the processing time used by each virtual warehouse.
- If usage is less than 10% of daily credit consumed, it is free

Date	Compute Credits Used (Warehouses only)	Cloud Services Credits Used	Credit Adjustment for Cloud Services (Lesser of 10% of Compute or Cloud Services)	Credits Billed (Sum of Compute, Cloud Services, and Adjustment)
Nov 1	100	20	-10	110
Nov 2	120	10	-10	120
Nov 3	80	5	-5	80
Nov 4	100	13	-10	103
Total	400	48	-35	413



Snowflake Billing



Compute

- User Managed VW
- Snowflake Managed VW
- Cloud Services



Storage

- Data Staging
- Database Tables
- Materialized View
- Time Travel and Fail Safe



Data Transfer Cost

- Data Transfer

Billing depends on Snowflake Edition, Cloud provider and Region
Check <https://www.snowflake.com/pricing/>





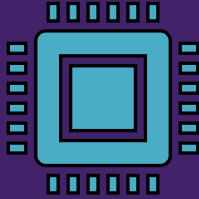
Storage

Snowflake Billing

- Staged File Costs
- Database Costs
- Time Travel and Fail-safe Costs
- Temporary and Transient Tables Costs
- Cloning Tables, Schemas, and Databases Costs



Snowflake Billing



Compute

- User Managed VW
- Snowflake Managed VW
- Cloud Services



Storage

- Data Staging
- Database Tables
- Materialized View
- Time Travel and Fail Safe



Data Transfer Cost

- Ingress is free
- Egress is charged

Billing depends on Snowflake Edition, Cloud provider and Region
Check <https://www.snowflake.com/pricing/>





Snowflake Billing

Data Transfer Cost

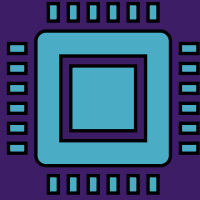
Data transfer is the process of moving data into (ingress) and out of (egress) Snowflake.

- **Unloading** - No charges for data ingress/loading but cost if you unload
- **Replication** - Data transfer in same region and same cloud provider is free
Cost when region or cloud provider is different
- **Writing External Function** - When data gets transferred out for function
- Per-Byte fee for Egress

Snowflake does **not apply data egress charges when a Snowflake client or driver retrieves query results across regions within the same cloud platform or across different cloud platforms.*



Snowflake Billing



Compute

- User Managed VW
- Snowflake Managed VW
- Cloud Services



Storage

- Data Staging
- Database Tables
- Materialized View
- Time Travel and Fail Safe



Data Transfer Cost

- Data Transfer

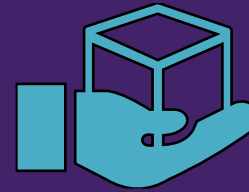
Billing depends on Snowflake Edition, Cloud provider and Region
Check <https://www.snowflake.com/pricing/>



Snowflake Billing



On Demand



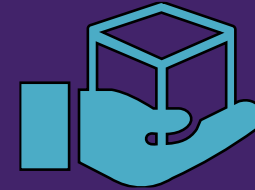
Capacity



Snowflake Billing



On Demand



Capacity

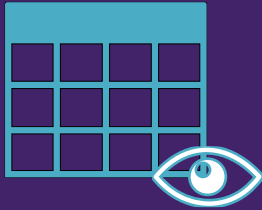
Features	On Demand	Capacity
Billing	Pay-as-you-go	Reserve in advance
Rates	Prevailing prices	Discounted prices



Snowflake Views



Snowflake Views

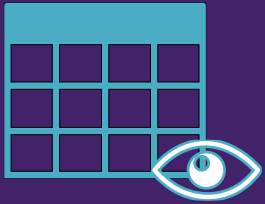


A view allows the result of a query to be accessed as if it were a table. The query is specified in the CREATE VIEW statement.

- Views/Non-Materialized View
- Materialized View
- Secure View

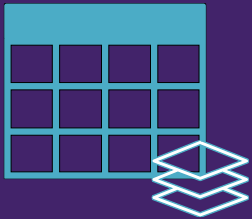


Snowflake Views



Non-Materialized View

Typical View, Virtual, No storage of Data



Materialized View

Behaves like a table, Serverless, Faster, Lots of Limitation*

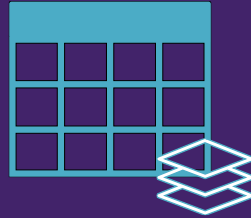


Secure View

For extra layer of security, can be done for both the views



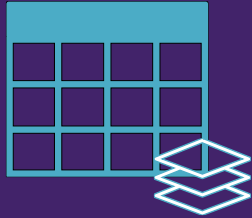
Materialized Views



Table



Materialized Views



When to Create Materialized View?

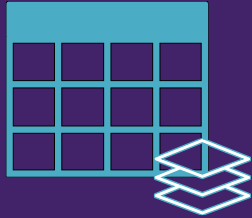
- Query results contain a small number of rows and/or columns
- Query results contain results that require significant processing, including:
 - Analysis of semi-structured data
 - Aggregates that take a long time to calculate
- The query is on an external table
- The base table does not change frequently.



Table



Materialized Views



Table

Advantage:

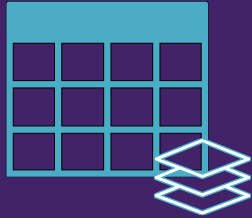
- Materialized views can improve the performance of repeat queries.
- Materialized views are automatically by Snowflake.
- Data accessed through materialized views is always Up-to-date.

Cost:

- Storage
- Compute (Managed by SF)



Materialized Views



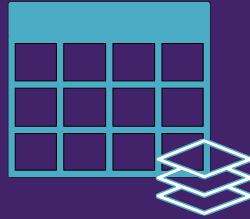
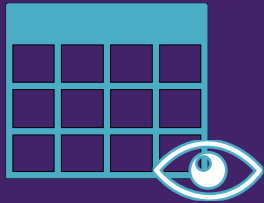
Table

Limitation:

- Only on single table
- Joins are not supported
- Many aggregate functions not allowed
- Full list : <https://docs.snowflake.com/en/user-guide/views-materialized#limitations-on-creating-materialized-views>



Snowflake Views



Definition

Storage

Use Case

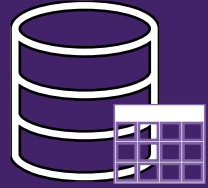
Non-Materialized View	Materialized View	Secure View
A virtual table based on the result of a SELECT statement.	A pre-computed and stored table based on the result of a SELECT statement.	Non-Materialized and Materialized view can be set as Secure View
Data is not stored, it is computed on the fly each time the view is queried.	Data is stored in the cache, allowing for faster query execution.	Same as underlying View.
Data transformation, providing a simplified view of the underlying data.	Reporting and analytics, frequently queried data.	When you want to limit access to the data



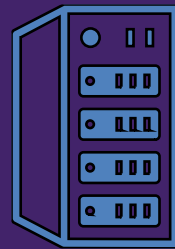
Snowflake Data Loading



Snowflake Data Loading



Various Data Types



Data Source



Stage

“Snowflake refers to the location of data files in cloud storage as a stage”



Internal



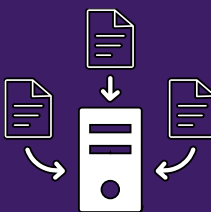
External



Stage



Internal



External

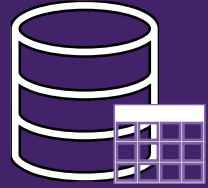


Table Stage	User Stage	Named Stage
Auto Created for each table	Auto Created for each User	Created by user
Can be used by Multi-user for 1 table	Can be used by single user for multiple tables	Used as any other object
Cant be altered/dropped	Cant be altered/dropped	Can be altered/dropped
@%	@~	@
Load using PUT	Load using PUT	Load using PUT

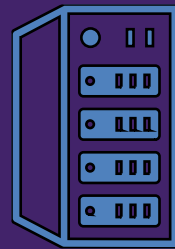
Named Stage (Optional)
Created by user
Refers object on cloud
Specify a URL
Cant point to Archived data



Snowflake Data Loading



Various Data Types



Data Source



Snowflake Data Loading



SNOWSQL



Limit of 50 MB



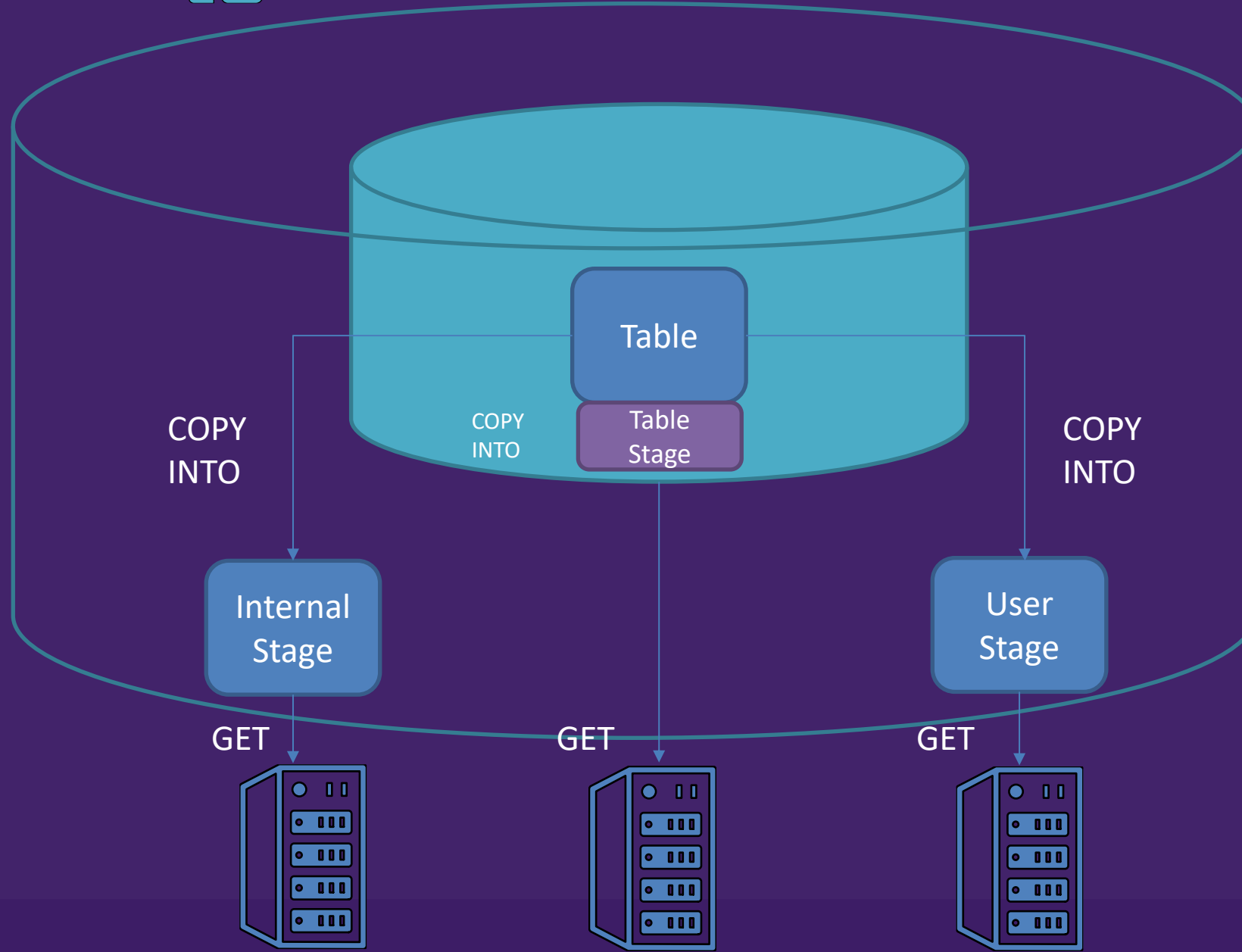


Unloading Data



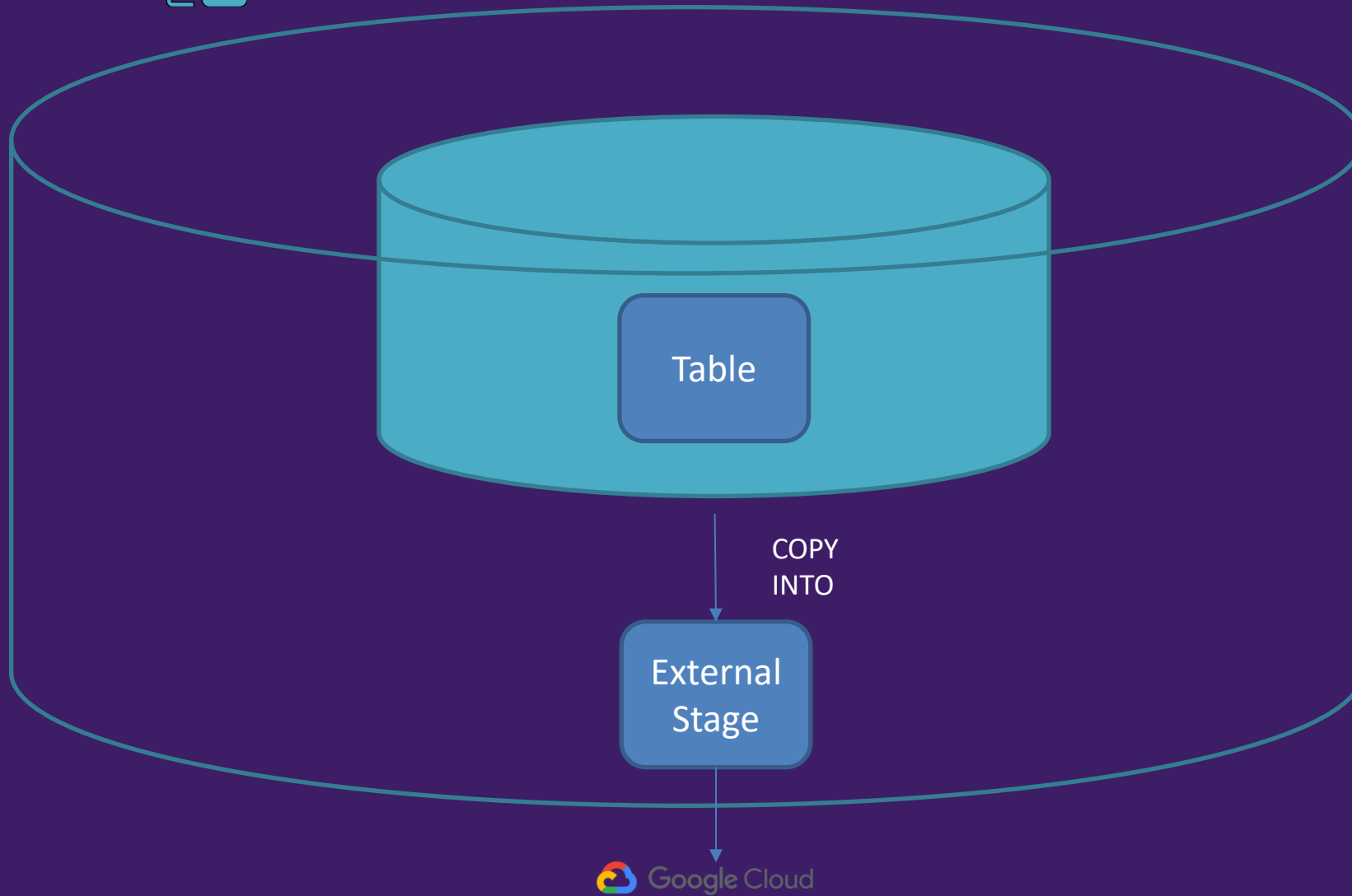


Unloading Data





Unloading Data



Google Cloud





Unloading Data

- Bulk unloading with **COPY INTO** <Location>
- For getting single file from multiple files use **SINGLE = TRUE**, default is **FALSE**
- Can partition the data by **PARTITION BY**

File Format

- CSV,TSV
- JSON
- PARQUET

File Encoding

- UTF-8

File Compression

- Gzip
- Bzip2
- Brotli
- Zstandard

Encryption of o/p File

- Internal (128/256-bit key)
- External (Cloud-User supplied Key)



Snowflake Data Loading



- CLI based tool
- Performs DML/DDDL
- Loading and unloading data



Snowflake Loading



Snowflake Loading

Bulk Loading



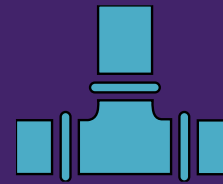
Copy Command

Large Data

User Managed WH

64 days Metadata

Continuous Loading



Snowpipe + Copy

Small Chunks

Serverless

14 days Metadata

Supported Type : CSV, TSV, JSON, PARQUET, AVRO, ORC, XML



Bulk Loading



Bulk Loading

- Load using Copy Command
- Can also transform the data
 - Column re-ordering*
 - Column omission*
 - Casts*
 - Truncating text strings that exceed the target column length*
- Auto Duplicate detection
- Stored in the metadata of the target table for 64 days. Available upon completion of the COPY statement as the statement output.



Snowflake Loading

Bulk Loading



Copy Command

Large Data

User Managed WH

64 days Metadata

Auth done by client

Continuous Loading



Snowpipe + Copy

Small Chunks

Serverless

14 days Metadata

Requires key pair authentication
with JSON Web Token



Snowpipe



Continuous Loading

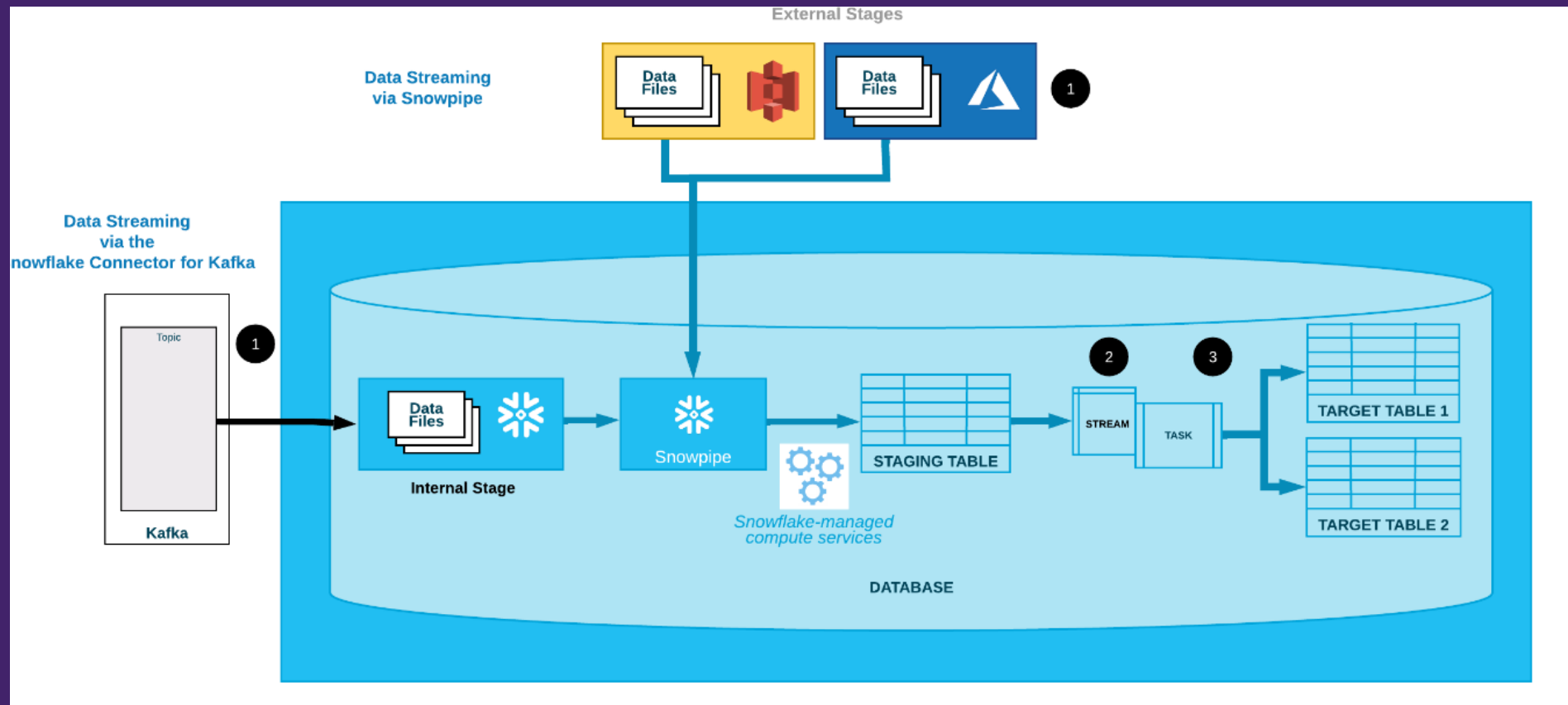
- We use Snowpipe
- Loads data in micro batches, within a minute
- Serverless – Managed by Snowflake
- Use options like Stream, Tasks



Snowpipe



Continuous Loading



Source : Snowflake





Continuous Loading

File Considerations while loading

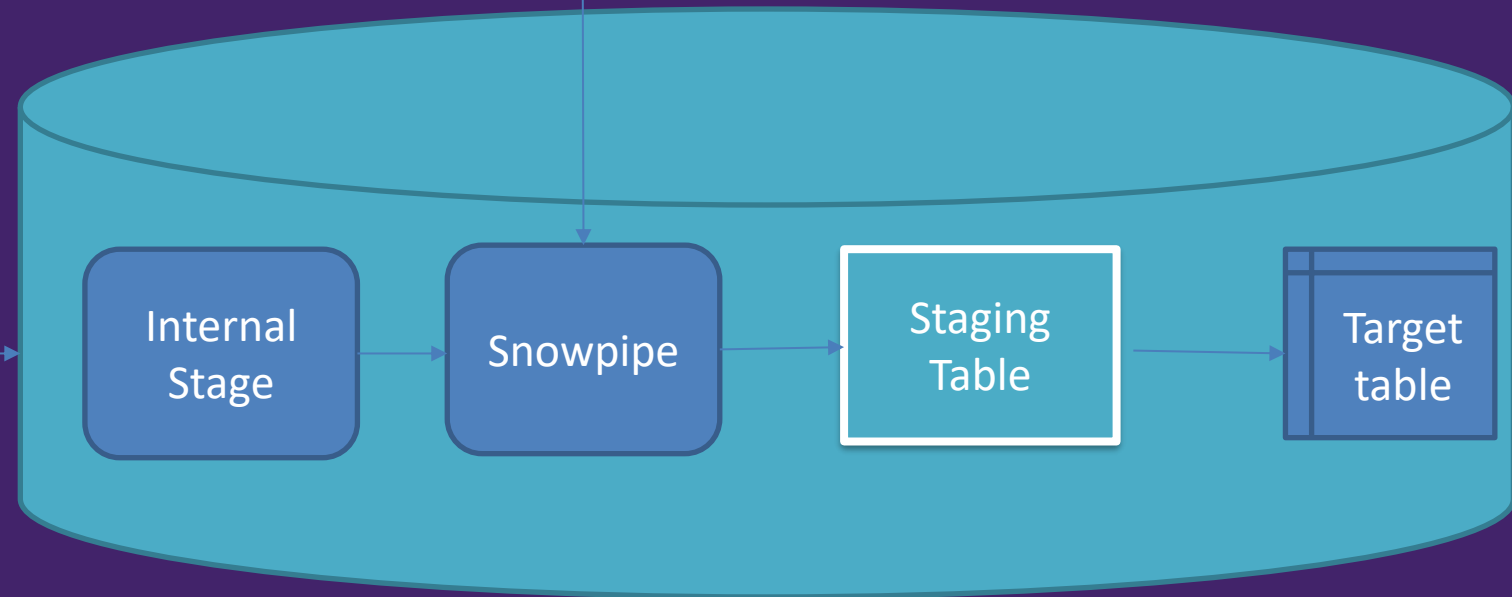
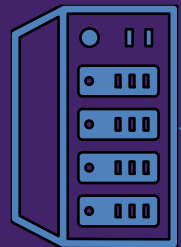
- The number of load operations that run in parallel cannot exceed the number of data files to be loaded
- Split Large files in smaller ones and combine small ones in large one
- Ideal size 100-250 mb (or larger) in size compressed
- Loading very large file eg. 100 GB or larger, not recommended
- For large Files , carefully use ON_ERROR option
- If load goes beyond 24 hours then job can be cancelled with no data load
- Consider accumulating file in source at 1 minute frequency



Continuous Loading

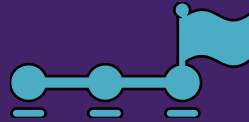


Continuous Loading



Continuous Loading

Stream



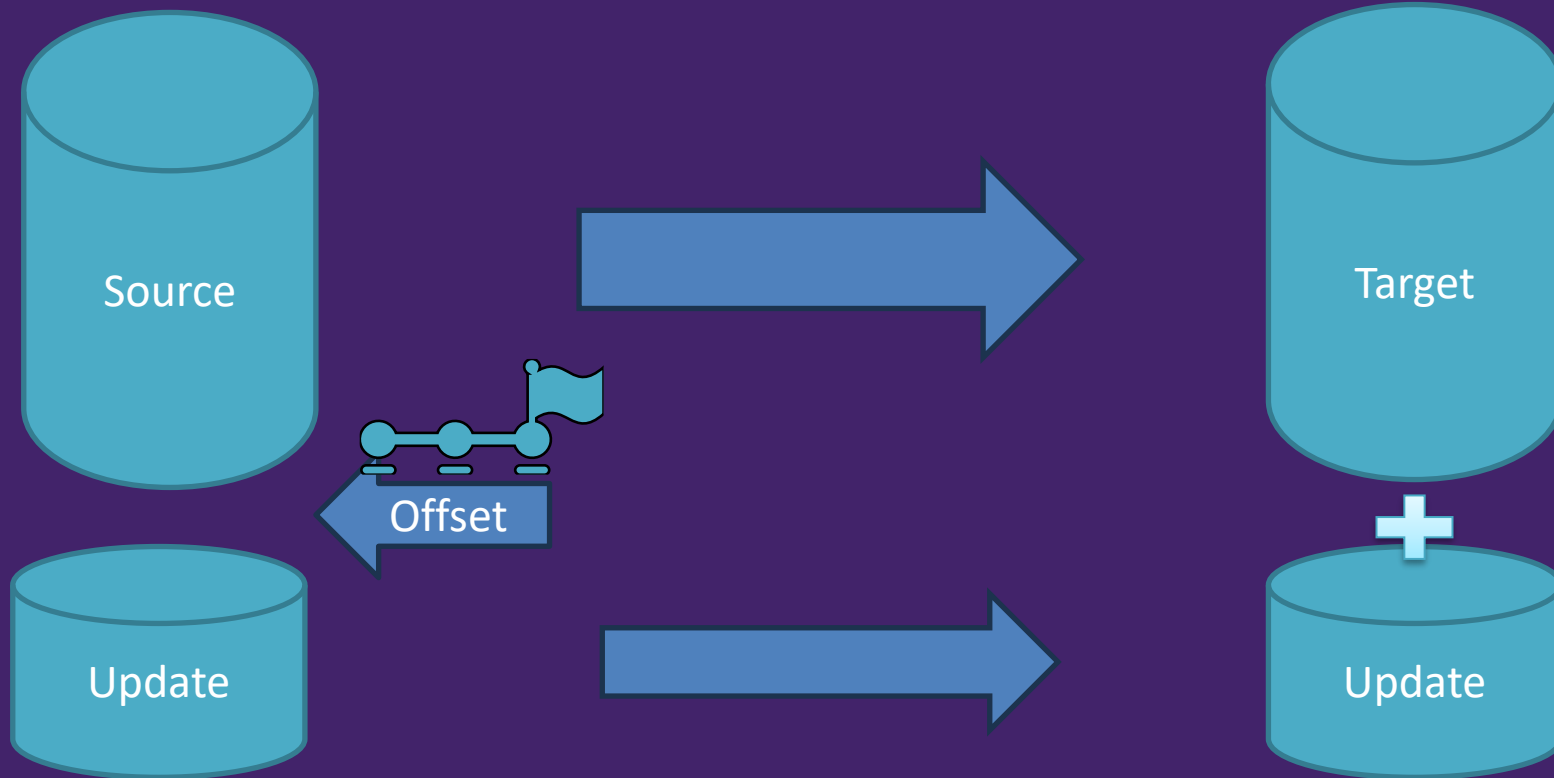
A *stream* is a Snowflake object type that provides change data capture (CDC) capabilities to track the delta of changes in a table, including inserts and data manipulation language (DML) changes, so action can be taken using the changed data.



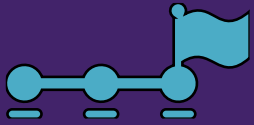
Stream



Continuous Loading



Stream

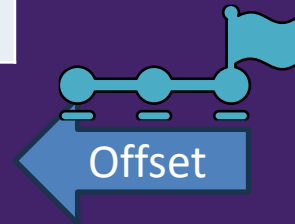


Continuous Loading

ID	NAME	AGE
1	John	25
2	Mary	30
3	Peter	40
4	Jane	35



ID	NAME	AGE
1	John	25
2	Mary	30
3	Peter	40
4	Jane	35



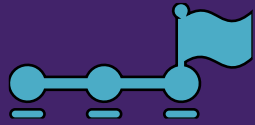
ID	NAME	AGE	Change
5	Mike	45	New
1	John	27	update
4	Jane	35	delete



ID	NAME	AGE
1	John	25-27
2	Mary	30
3	Peter	40
4	Jane	35
5	Mike	45



Stream



Continuous Loading

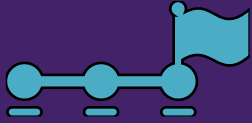
ID	NAME	AGE	Change
5	Mike	45	New
1	John	27	update
4	Jane	35	delete

- METADATA\$ACTION
- METADATA\$ISUPDATE
- METADATA\$ROW_ID

ID	NAME	AGE	METADATA\$ACTION	METADATA\$ISUPDATE	METADATA\$ROW_ID
5	Mike	45	INSERT	FALSE	6bbc69d84969b31a64c8f2ba2a8a61d8ae47c234
1	John	27	INSERT	TRUE	f98efdd8f203e090bb96a1bbcac1ffb5f3b1fd39
1	John	25	DELETE	TRUE	f98efdd8f203e090bb96a1bbcac1ffb5f3b1fd39
4	Jane	35	DELETE	FALSE	651a6fc5dc4bb247197e98aeaf876f7388d21b5d



Stream



Continuous Loading

Supported table types

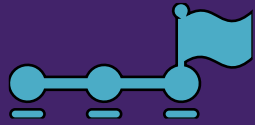
- Standard tables
- Directory tables
- External tables
- Underlying tables for a view

Types

- Standard
- Append-only
- Insert-only (External Table)



Stream



Continuous Loading

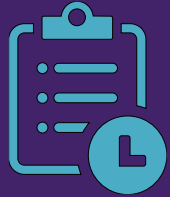
- Streams goes stale when source tables goes out of retention period
- Stale stream doesn't track changes
- Shall consume the stream within retention period

DATA_RETENTION_TIME_IN_DAYS	MAX_DATA_EXTENSION_TIME_IN_DAYS	Consume Streams in X Days
14	0	14
1	14	14
0	90	90

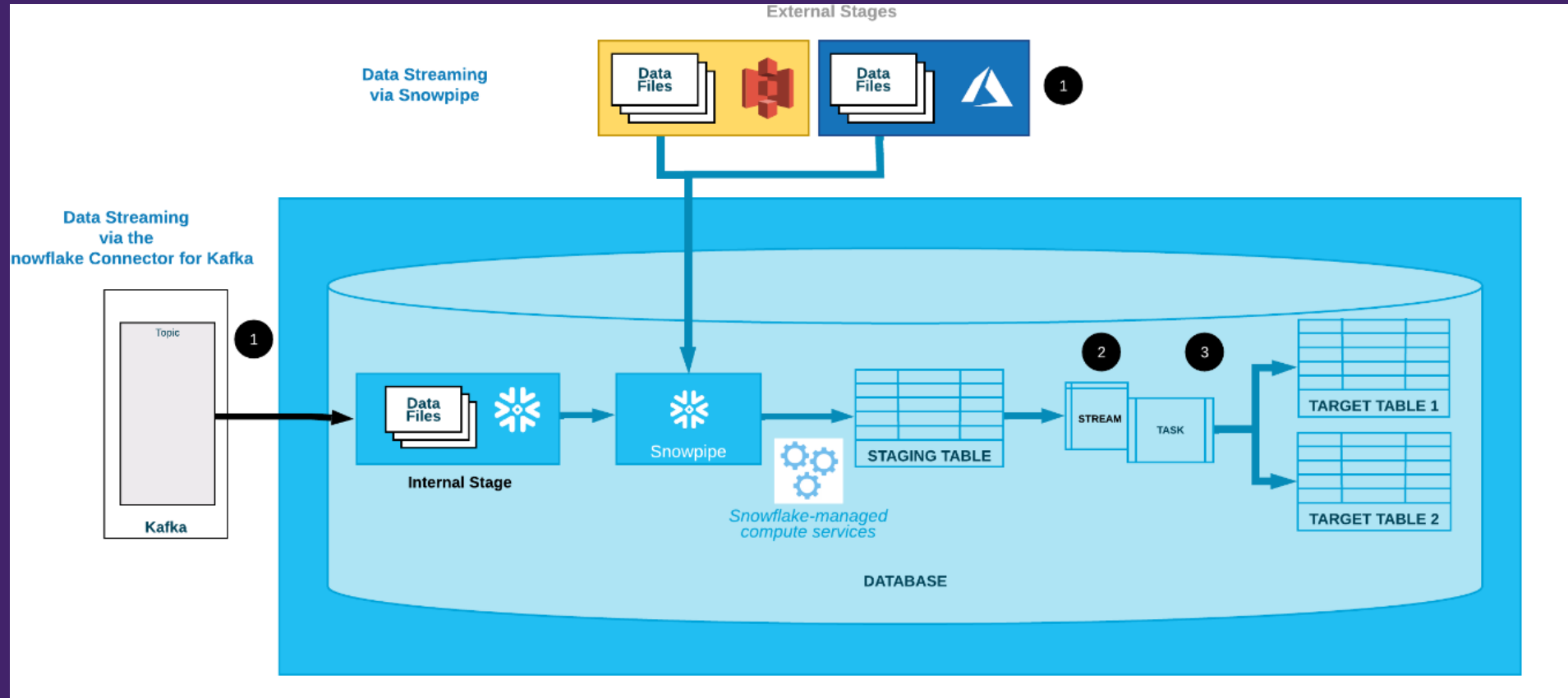
Check Stream's stale date in column STALE_AFTER



Tasks

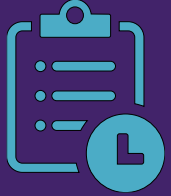


Continuous Loading



Source : Snowflake

Tasks



Continuous Loading

A task can execute any one of the following types of SQL code:

- Single SQL statement
- Call to a stored procedure
- Procedural logic using Snowflake Scripting





Continuous Loading

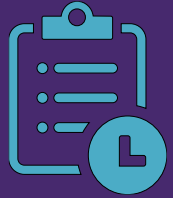
User Managed Tasks

```
CREATE TASK TASK_DEMO                                     ©Learn Anywhere
  WAREHOUSE = COMPUTE_WH
  SCHEDULE = '5 MINUTE'
AS
INSERT INTO mytable(ts) VALUES(CURRENT_TIMESTAMP);
```

Serverless Tasks

```
CREATE TASK TASK_DEMO                                     ©Learn Anywhere
  USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE = 'XSMALL'
  SCHEDULE = '5 MINUTE'
AS
INSERT INTO mytable(ts) VALUES(CURRENT_TIMESTAMP);
```



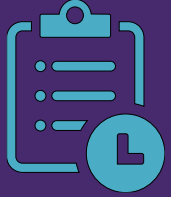


Continuous Loading

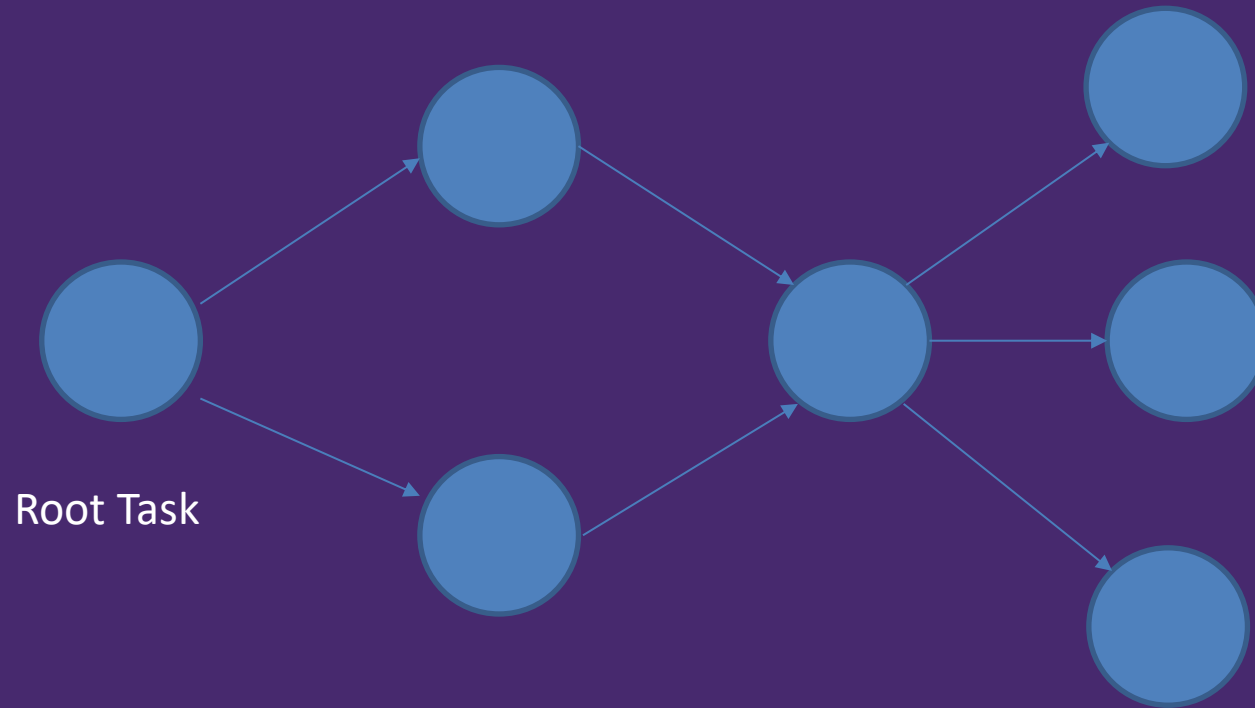
Category	Serverless Task	User-Managed Task
Utilization	When you cant fully utilize a WH	When you can fully utilize the WH
Workload	Predictable load	May have Spike
Scheduling	When Schedule is important	When Schedule is less important
Limitation	Max WH size is XXLARGE Java/Python UDF Scala Stored Procedure	



Tasks



Continuous Loading

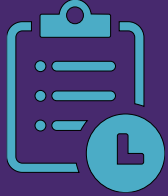


Root Task

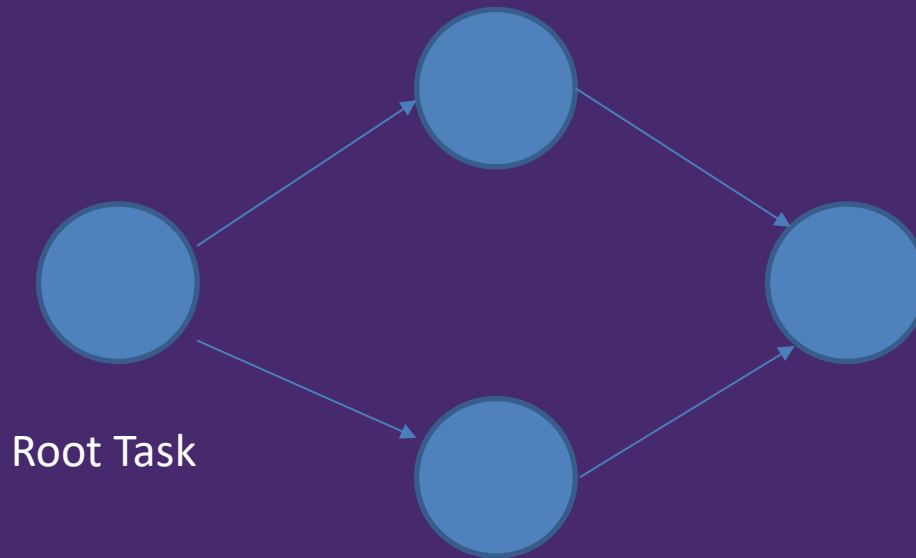
Directed Acyclic Graph (DAG)



Tasks



Continuous Loading



Root Task

Directed Acyclic Graph (DAG)

A Directed Acyclic Graph (DAG) is a series of tasks composed of a single root task and additional tasks, organized by their dependencies

- One directional
- Can have multiple predecessor/Successor
- No looping allowed
- Max 1000 task
- Task can have Max 100 Predecessor tasks
- Task can have Max 100 Child tasks





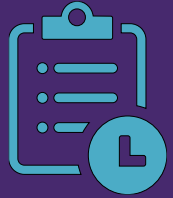
Continuous Loading

CRON OPTION

```
CREATE TASK t1                                     ©Learn Anywhere
  SCHEDULE = 'USING CRON 0 9 * * * SUN America/Los_Angeles'
  USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE = 'XSMALL'
  AS <INSERT Command>;
```

Option	Description	Example
Minute	The minute of the hour.	0-59
Hour	The hour of the day.	0-23
Day of Month	The day of the month.	1-31
Month	The month of the year.	1-12
Day of Week	The day of the week.	0-7 (0=Sunday, 1=Monday, etc.)





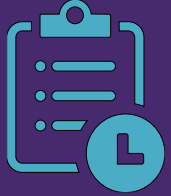
Continuous Loading

Snowflake Cron:

1. Scheduling feature in Snowflake Data Warehouse.
2. Automates repetitive tasks using cron-style syntax.
3. Specifies start and end time, frequency, and interval of a task.
4. Automates routine tasks such as data loading and transformations.
5. Flexible and powerful tool for scheduling and automating tasks



Tasks



Continuous Loading

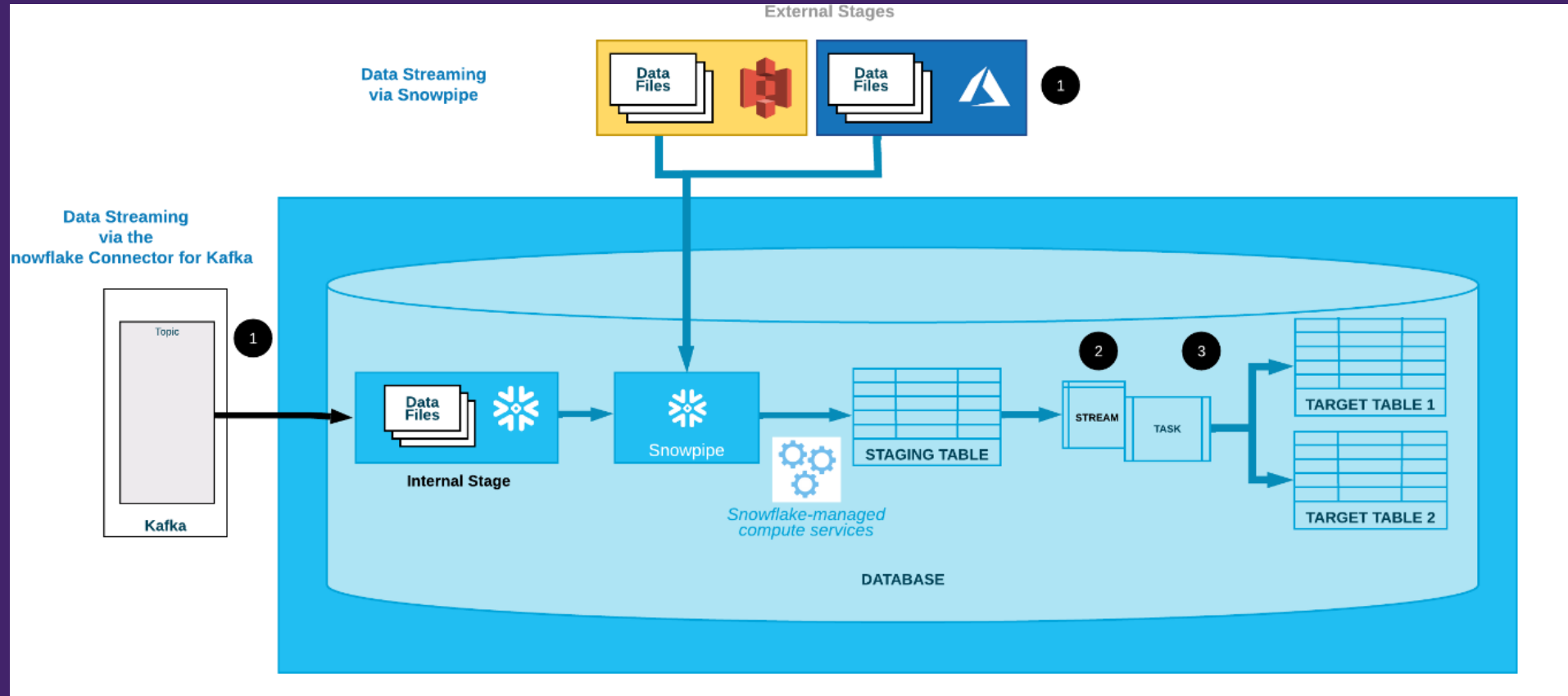
_____ minute (0 - 59)
| _____ hour (0 - 23)
| | _____ day of the month (1 - 31)
| | | _____ month (1 - 12)
| | | | _____ day of the week (0 - 6) (Sunday to Saturday;
| | | | 7 is also Sunday on some systems)
| | | |
| | | |
* * * * *
0 9 * * * America/Los_Angeles



Snowpipe

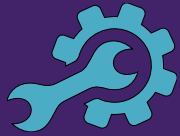


Continuous Loading



Source : Snowflake

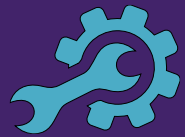




User Defined Functions (UDFs)

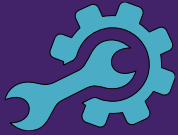
Language	Tabular	Looping and Branching	Pre-Compiled or Staged Source	In-line	Sharable
SQL	Yes	No	No	Yes	Yes
JavaScript	Yes	Yes	No	Yes	Yes
Java	Yes	Yes	Yes (precompiled)	Yes	No [1]
Python	Yes	Yes	Yes (staged)	Yes	No [2]





User Defined Functions (UDFs)





User Defined Functions (UDFs)

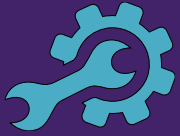
“With user-defined functions (UDFs), you can extend the system to perform operations that are not available through the built-in, system-defined functions provided by Snowflake.”

Scalar UDF



Table UDF (UDTF)





User Defined Functions (UDFs)

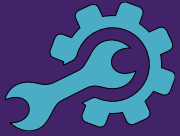
UDFs are DB object, created in specific DB and Schema.

db.schema.function_name

demodb.demoschema.udf_sample

SELECT demodb.demoschema.udf_sample()





User Defined Functions (UDFs)

UDFs are DB object, created in specific DB and Schema.

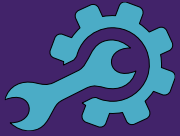
db.schema.function_name

demodb.demoschema.udf_sample

SELECT demodb.demoschema.udf_sample()

*Not allowed to create UDF with same name as system defined function

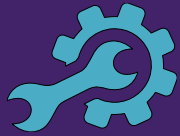




User Defined Functions (UDFs)

```
CREATE OR REPLACE FUNCTION square(x FLOAT)
RETURNS FLOAT
LANGUAGE JAVASCRIPT
AS
$$
return x * x;
$$;
```





User Defined Functions (UDFs)

```
CREATE OR REPLACE FUNCTION square(x FLOAT)
RETURNS FLOAT
LANGUAGE JAVASCRIPT
AS
$$
return x * x;
$$;
```



```
CREATE OR REPLACE FUNCTION square(x FLOAT)
RETURNS FLOAT
LANGUAGE PYTHON
AS
$$
    return x * x
$$;
```



```
CREATE OR REPLACE FUNCTION square(x FLOAT)
RETURNS FLOAT
AS
$$
SELECT x * x;
$$;
```



```
CREATE OR REPLACE FUNCTION square(x FLOAT)
RETURNS FLOAT
LANGUAGE JAVA
AS
$$
    class Square {
        public static float square(float x) {
            return x * x;
        }
    }
    return Square.square(x);
$$;
```



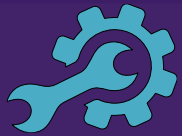
Overloading Functions

```
CREATE OR REPLACE FUNCTION add_numbers(x NUMBER)
RETURNS NUMBER
AS
$$
SELECT x + 8;
$$;
```

```
CREATE OR REPLACE FUNCTION add_numbers(x STRING)
RETURNS STRING
AS
$$
SELECT 'x || "8"';
$$;
```

```
SELECT add_numbers(1); -- Returns 9
SELECT add_numbers('1'); -- Returns 18
```





User Defined Table Functions (UDTFs)

```
CREATE OR REPLACE FUNCTION get_orders_by_customer(customer_id INT)
RETURNS TABLE (order_id INT, order_date DATE, total_amount FLOAT)
AS
$$
SELECT order_id, order_date, total_amount
FROM orders
WHERE customer_id = customer_id;
$$;
```

```
SELECT * FROM TABLE(get_orders_by_customer(1234));
```





Stored Procedure

A stored procedure is a named, precompiled collection of SQL statements and control-of-flow language constructs that are stored in a database and can be called from SQL CALL command, including other stored procedures.

```
CREATE OR REPLACE PROCEDURE procedure_name (parameter_list)
RETURNS return_type NOT NULL
LANGUAGE {SQL | JAVASCRIPT | PYTHON | JAVA}
EXECUTE AS CALLER/OWNER
AS
$$
BEGIN
RETURN message;
END;
$$;
```

```
CALL procedure_name(parameter_values);
```





Stored Procedure



Snowpark



Snowflake Scripting



Java



Python



Scala



JavaScript



SQL





Stored Procedure

```
CREATE OR REPLACE PROCEDURE get_customer_orders(customer_id INT)
RETURNS TABLE (order_id INT, order_date DATE, total_amount FLOAT)
LANGUAGE SQL
EXECUTE AS OWNER
AS
$$
BEGIN
    SELECT order_id, order_date, total_amount
    FROM orders
    WHERE customer_id = customer_id;
END;
$$;
```

```
CALL get_customer_orders(1234);
```



UDF vs Stored Procedure

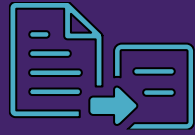
<i>Feature</i>	<i>User-Defined Function (UDF)</i>	<i>Stored Procedure</i>
Return Value	Must return a value	Not Mandatory to return a value
Use case	When part of SQL stmt	For DML, DDL Operations
Languages	Java, JS, Python, Scala, SQL	Java, JS, Python, Scala, Snowflake Scripting
Granularity	Can be called for every row of a query result set	Can only be executed as a standalone command
Inside SELECT	Multiple UDFs can be called	Single SP with a statement, Can have nested SP
DB Access	May not have access	Has access





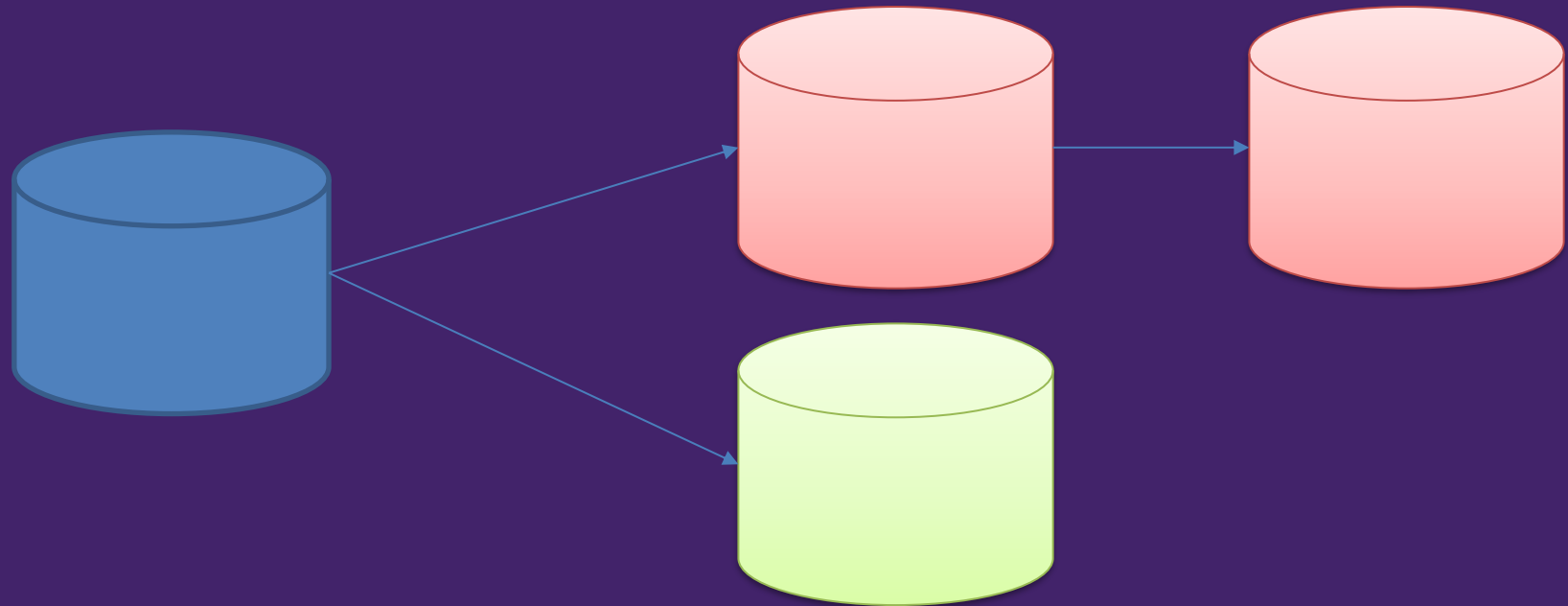
Cloning

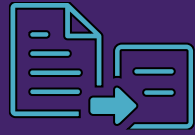




Cloning

Cloning in Snowflake refers to the process of creating an exact copy of a database, schema, or table. This feature can be used to create a backup of existing data, test new data structures, or create a new environment for development or testing purposes.



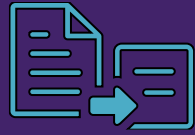


Cloning

- Database
- Schema
 - External table
 - Internal named Stage
- Tables
- File Format
- Sequence
- Tags
- Clustering Key (Suspended)
- Streams
- External Stage
- Table Stage (empty)
- Tasks (By default suspended)
- Pipe (referring External stage)
- View (Only with DB/Schema)

Users
Roles
Grants
Virtual Warehouses
Resource monitors
Storage integrations
Policy
View
Internal named staged
Temporary Table

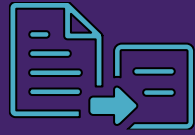




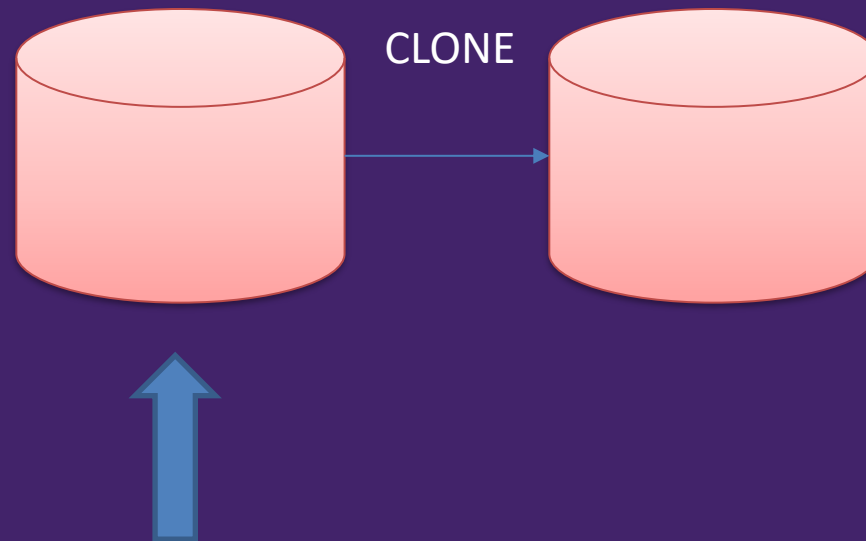
Cloning

Object	Privilege
Tables	SELECT
Pipes, Streams, Tasks	OWNERSHIP
Other objects	USAGE





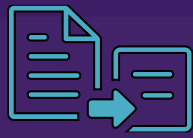
Cloning



Update Records

DATA_RETENTION_TIME_IN_DAYS = 1





Cloning (with Time Travel)

Only available for Database, Schema, Table & Streams

Cloning fails if

- Source in past doesn't exist
- Pipe object with `AUTO_INGEST = TRUE` recreated

* By default current timestamp is taken





Semi- Structured Data





Semi- Structured Data

“Semi-structured data is data that does not conform to the standards of traditional structured data, but contains tags (labels) or other types of mark-up that identify individual, distinct entities within the data.”

- Doesn't have fixed schema
- Contains N-level hierarchies of nested information

```
{
  "name": "John Doe",
  "age": 35,
  "isMarried": true,
  "hobbies": [
    "reading",
    "hiking",
    "cooking"
  ],
  "address": {
    "street": "123 Main Street",
    "city": "Anytown",
    "state": "CA",
    "zip": "12345"
  }
}
```



Semi- Structured Data

- Snowflake provides native data types (ARRAY, OBJECT, and VARIANT)
- Snowflake supports querying the data
- Supported Data formats (For Load)
 - JSON
 - Avro
 - ORC
 - Parquet
 - XML
- Supported Data formats (For Unload)
 - JSON
 - Parquet





Semi- Structured Data

Data Type	Description	Example
ARRAY (Max 16MB)	A data type used to store an ordered list of values of the same data type.	['apple', 'banana', 'orange']





Semi- Structured Data

Data Type	Description	Example
ARRAY (Max 16MB)	A data type used to store an ordered list of values of the same data type.	['apple', 'banana', 'orange']
OBJECT(Max 16MB)	A data type used to store an unordered collection of key-value pairs, where each key is a string and each value can be a different data type.	{'name': 'John', 'age': 35, 'isMarried': true}





Semi- Structured Data

Data Type	Description	Example
ARRAY (Max 16MB)	A data type used to store an ordered list of values of the same data type.	['apple', 'banana', 'orange']
OBJECT(Max 16MB)	A data type used to store an unordered collection of key-value pairs, where each key is a string and each value can be a different data type.	{'name': 'John', 'age': 35, 'isMarried': true}
VARIANT(Max 16MB)	A data type used to store semi-structured data, including JSON documents, XML documents, and other complex data types. VARIANT can store any type of data and allows you to query and manipulate it as needed.	{ 'name': 'John', 'age': 35, 'isMarried': true, 'hobbies': ['reading', 'hiking', 'cooking'], 'address': { 'street': '123 Main Street', 'city': 'Anytown', 'state': 'CA', 'zip': '12345'} }





Unstructured Data

External stages

Amazon S3, GCP, Azure

Blob storage

Data Lake Storage Gen2

General-purpose v1

General-purpose v2





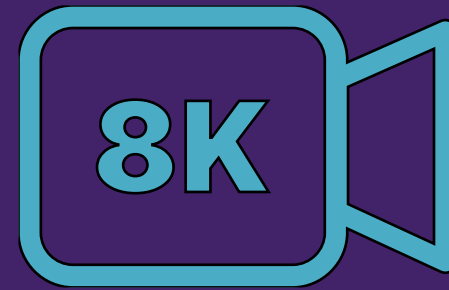
Unstructured Data





Unstructured Data

“Unstructured data is information that does not fit into a predefined data model or schema”.



VCF (genomics), KDF (semiconductors), or HDF5 (aeronautics)





Unstructured Data

- Securely access data files located in cloud storage.
- Share file access URLs with collaborators and partners.
- Load file access URLs and other file metadata into Snowflake tables.
- Process Unstructured data





Types of URLs to Access File

	Scoped URL	File URL	Pre-signed URL
Definition	Encoded URL that permits temporary access to a staged file without granting privileges to the stage	URL that identifies the database, schema, stage, and file path to a set of files. A role that has sufficient privileges on the stage can access the files.	Simple HTTPS URL used to access a file via a web browser. A file is temporarily accessible to users via this URL using a pre-signed access token.





Types of URLs to Access File

	Scoped URL	File URL	Pre-signed URL
Definition	Encoded URL that permits temporary access to a staged file without granting privileges to the stage	URL that identifies the database, schema, stage, and file path to a set of files. A role that has sufficient privileges on the stage can access the files.	Simple HTTPS URL used to access a file via a web browser. A file is temporarily accessible to users via this URL using a pre-signed access token.
Access Control	Only the user who generates a scoped URL .	User with access.	Anyone with URL





Types of URLs to Access File

	Scoped URL	File URL	Pre-signed URL
Definition	Encoded URL that permits temporary access to a staged file without granting privileges to the stage	URL that identifies the database, schema, stage, and file path to a set of files. A role that has sufficient privileges on the stage can access the files.	Simple HTTPS URL used to access a file via a web browser. A file is temporarily accessible to users via this URL using a pre-signed access token.
Access Control	Only the user who generates a scoped URL .	User with access.	Anyone with URL
Expiration	query results cache = 24 hours	Permanent	Can be configured with <i>expiration_time</i>





Types of URLs to Access File

	Scoped URL	File URL	Pre-signed URL
Definition	Encoded URL that permits temporary access to a staged file without granting privileges to the stage	URL that identifies the database, schema, stage, and file path to a set of files. A role that has sufficient privileges on the stage can access the files.	Simple HTTPS URL used to access a file via a web browser. A file is temporarily accessible to users via this URL using a pre-signed access token.
Access Control	Only the user who generates a scoped URL .	User with access.	Anyone with URL
Expiration	query results cache = 24 hours	Permanent	Can be configured with <i>expiration_time</i>
How-to-Fun	BUILD_SCOPED_FILE_URL	BUILD_STAGE_FILE_URL	GET_PREIGNED_URL





Types of URLs to Access File

	Scoped URL	File URL	Pre-signed URL
Definition	Encoded URL that permits temporary access to a staged file without granting privileges to the stage	URL that identifies the database, schema, stage, and file path to a set of files. A role that has sufficient privileges on the stage can access the files.	Simple HTTPS URL used to access a file via a web browser. A file is temporarily accessible to users via this URL using a pre-signed access token.
Access Control	Only the user who generates a scoped URL .	User with access.	Anyone with URL
Expiration	query results cache = 24 hours	Permanent	Can be configured with <i>expiration_time</i>
How-to-Fun	BUILD_SCOPED_FILE_URL	BUILD_STAGE_FILE_URL	GET_PREIGNED_URL
Syntax	BUILD_SCOPED_FILE_URL(@<stage_name> , '<relative_file_path>')	BUILD_STAGE_FILE_URL(@<stage_name> , '<relative_file_path>')	GET_PREIGNED_URL(@<stage_name> , '<relative_file_path>' , [<expiration_time>])





SQL Functions

SQL Function	Description
GET_STAGE_LOCATION	Returns the URL for an external or internal named stage using the stage name as the input.
GET_RELATIVE_PATH	Extracts the path of a staged file relative to its location in the stage using the stage name and absolute file path in cloud storage as inputs.
GET_ABSOLUTE_PATH	Returns the absolute path of a staged file using the stage name and path of the file relative to its location in the stage as inputs.
GET_PREIGNED_URL	Generates the pre-signed URL to a staged file using the stage name and relative file path as inputs. Access files in an external stage using the function.
BUILD_SCOPED_FILE_URL	Generates a scoped Snowflake-hosted URL to a staged file using the stage name and relative file path as inputs.
BUILD_STAGE_FILE_URL	Generates a Snowflake-hosted file URL to a staged file using the stage name and relative file path as inputs.

`GET_PREIGNED_URL` and `BUILD_SCOPED_FILE_URL` are non-deterministic functions; the others are deterministic.

Source : Snowflake





Directory





Directory

- Stores file level meta data about files in stage
- Implicit object layered on stage
- Can be enabled for external/internal stage
- Stores URL by BUILD_STAGE_FILE_URL
- Can be enabled during stage creation or later (by alter)
- Data can be refreshed manually or automatically (only for cloud i.e external stages)
- No explicit privilege required to access





Connect to Cloud



- Create bucket and upload file in S3
- Create a Policy
- Create a Role
- Create Storage Integration
- Update role with AWS IAM user and External ID
- Create external stage
- Use COPY INTO







SAMPLING





SAMPLING





SAMPLING

FIXED-SIZE

Fixed Numbers of rows are returned

FRACTION BASED

BERNOULLI | ROW : $(p/100)*n$, Number of rows vary, row wise

SYSTEM | BLOCK : Faster, Might be biased, block wise

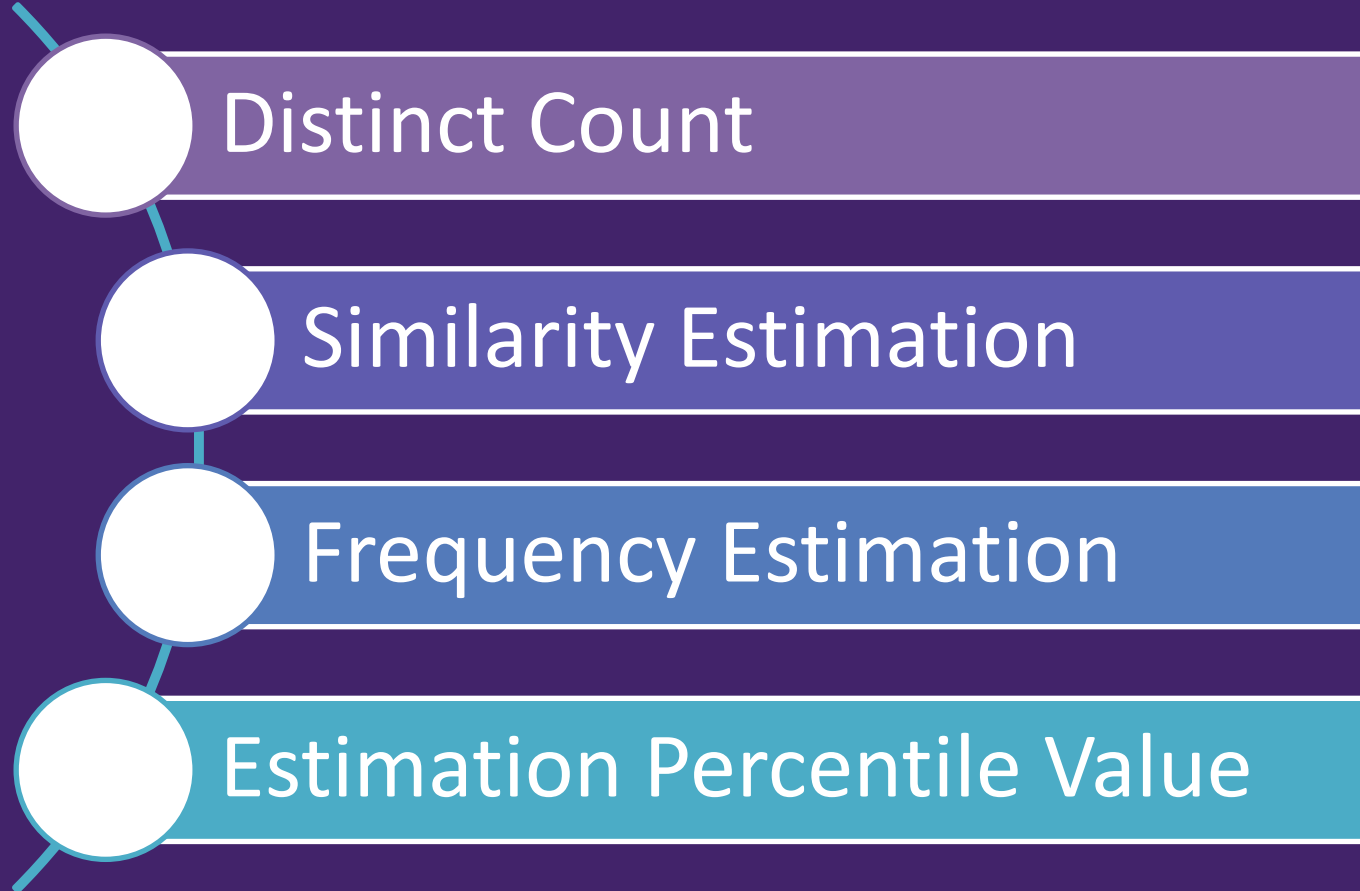
Sampling on Join is possible if, BERNOULLI/ROW BASED, No Seed Used.



fx Estimating Functions



fx Estimating Functions



fx Distinct Count

- Idea is to count distinct rows in a table
- Snowflake uses HLL (HyperLogLog) Algo

12
13
14
15

```
SELECT COUNT(DISTINCT WR_ORDER_NUMBER) FROM SNOWFLAKE_SAMPLE_DATA.TPCDS_SF100TCL.WEB_RETURNS;
```

Results Chart

	COUNT(DISTINCT WR_ORDER_NUMBER)
1	4,240,802,950

Query Details ...

Query duration 6m 25s

Rows 1

16
17

```
SELECT APPROX_COUNT_DISTINCT (WR_ORDER_NUMBER) FROM SNOWFLAKE_SAMPLE_DATA.TPCDS_SF100TCL.WEB_RETURNS;
```

Results Chart

	APPROX_COUNT_DISTINCT (WR_ORDER_NUMBER)
1	4,201,561,184

Query Details ...

Query duration 37s



*fx*Similarity Estimation

- Find Similarity between 2 or more datasets
- Uses Minhash function
- Output ranges from 0 to 1,
- 1 : Same table, 0 - Completely distinct



fx Frequency Estimation

- To approximate frequency of value in the dataset
- Snowflake usage Space-saving Algo
- We use function like : APPROX_TOP_K,
APPROX_TOP_K_ACCUMULATE/COMBINE/ESTIMATE



*fx*Percentile Estimation

- To approximate percentile values in the dataset
- Snowflake usage t-Digest Algo
- We use function like : APPROX_PERCENTILE, APPROX_PERCENTILE_ACCUMULATE/COMBINE/ESTIMATE
- Approximate values
- Works only on Numeric Value
- May return data which is not part of the dataset



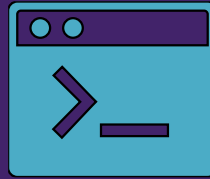
fx Supported-Function Type



fx Supported-Function Type



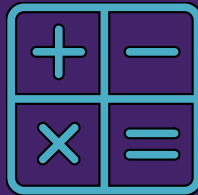
Scalar



Window



Table



Aggregate



System

Snowflake also supports , UDF and External Function



fx Supported-Function Type



Scalar

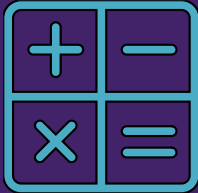
Returns 1 value per invocation

Like : File Function, Numeric Function etc

*Example : SELECT CEIL(135.135);
Returns >> 136*



fx Supported-Function Type



Aggregate

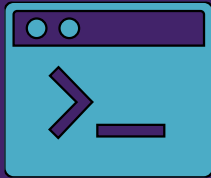
Returns 1 value per group of rows

Like : SUM, AVG etc

*Example : SELECT **AVG**(salary) from <table>;*



fx Supported-Function Type



Window

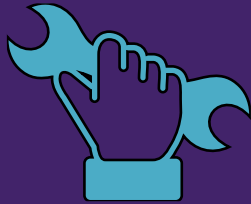
Aggregate functions that can operate on a subset of rows within the set of input rows

Like : ANY_VALUE, RATIO_TO_REPORT etc

*Example : SELECT province, store_ID, profit,
100 * RATIO_TO_REPORT(profit) OVER (PARTITION BY province) AS percent_profit
FROM store_profit
ORDER BY province, store_ID;*



fx Supported-Function Type



System

Control Functions that allows you to execute action in the system.

*Like : **SYSTEM\$**.... etc*

```
SELECT SYSTEM$ABORT_SESSION(1065153868222);
```



fx Supported-Function Type



Table

A table function returns a set of rows for each input row. The returned set can contain zero, one, or more rows. Each row can contain one or more columns.

*Like : **VALIDATE....** etc*





Micro-Partitioning





Micro-Partitioning



Traditional DBs have static partitioning to manage performance and scaling , with limitations like data skew and maintenance overheads



Snowflake overcomes limitations with Micro-Partition





Micro-Partitioning

- Snowflake automatically divides table in micro-partitions
- Each micro-partition contain 50-500 MB of uncompressed data (SF compress it further and stores)
- Organized in columnar format
- Very large tables can have multi-million micro-partitions
- Micro-partitions are immutable (Time-Travel, Cloning)





Micro-Partitioning

Table: t1

Logical Structure

type	name	country	date
2	A	UK	11/2
4	C	SP	11/2
3	C	DE	11/2
2	B	DE	11/2
3	A	FR	11/2
2	C	SP	11/2
3	Z	DE	11/2
2	B	UK	11/2
4	C	NL	11/2
5	X	FR	11/3
1	A	NL	11/3
5	A	FR	11/3
2	X	FR	11/2
4	Z	NL	11/2
2	Y	SP	11/2
1	B	SP	11/3
5	X	DE	11/3
3	A	UK	11/4
1	C	FR	11/3
4	Z	NL	11/4
5	Y	SP	11/4
5	B	SP	11/5
3	X	DE	11/5
2	Z	UK	11/5

Physical Structure

	Micro-partition 1 (rows 1-6)	Micro-partition 2 (rows 7-12)	Micro-partition 3 (rows 13-18)	Micro-partition 4 (rows 19-24)																								
type	<table><tr><td>2</td><td>4</td><td>3</td></tr><tr><td>2</td><td>3</td><td>2</td></tr></table>	2	4	3	2	3	2	<table><tr><td>3</td><td>2</td><td>4</td></tr><tr><td>5</td><td>1</td><td>5</td></tr></table>	3	2	4	5	1	5	<table><tr><td>2</td><td>4</td><td>2</td></tr><tr><td>1</td><td>5</td><td>3</td></tr></table>	2	4	2	1	5	3	<table><tr><td>1</td><td>4</td><td>5</td></tr><tr><td>5</td><td>3</td><td>2</td></tr></table>	1	4	5	5	3	2
2	4	3																										
2	3	2																										
3	2	4																										
5	1	5																										
2	4	2																										
1	5	3																										
1	4	5																										
5	3	2																										
name	<table><tr><td>A</td><td>C</td><td>C</td></tr><tr><td>B</td><td>A</td><td>C</td></tr></table>	A	C	C	B	A	C	<table><tr><td>Z</td><td>B</td><td>C</td></tr><tr><td>X</td><td>A</td><td>A</td></tr></table>	Z	B	C	X	A	A	<table><tr><td>X</td><td>Z</td><td>Y</td></tr><tr><td>B</td><td>X</td><td>A</td></tr></table>	X	Z	Y	B	X	A	<table><tr><td>C</td><td>Z</td><td>Y</td></tr><tr><td>B</td><td>X</td><td>Z</td></tr></table>	C	Z	Y	B	X	Z
A	C	C																										
B	A	C																										
Z	B	C																										
X	A	A																										
X	Z	Y																										
B	X	A																										
C	Z	Y																										
B	X	Z																										
country	<table><tr><td>UK</td><td>SP</td><td>DE</td></tr><tr><td>DE</td><td>FR</td><td>SP</td></tr></table>	UK	SP	DE	DE	FR	SP	<table><tr><td>DE</td><td>UK</td><td>NL</td></tr><tr><td>FR</td><td>NL</td><td>FR</td></tr></table>	DE	UK	NL	FR	NL	FR	<table><tr><td>FR</td><td>NL</td><td>SP</td></tr><tr><td>SP</td><td>DE</td><td>UK</td></tr></table>	FR	NL	SP	SP	DE	UK	<table><tr><td>FR</td><td>NL</td><td>SP</td></tr><tr><td>SP</td><td>DE</td><td>UK</td></tr></table>	FR	NL	SP	SP	DE	UK
UK	SP	DE																										
DE	FR	SP																										
DE	UK	NL																										
FR	NL	FR																										
FR	NL	SP																										
SP	DE	UK																										
FR	NL	SP																										
SP	DE	UK																										
date	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr></table>	11/2	11/2	11/2	11/2	11/2	11/2	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/3</td><td>11/3</td><td>11/3</td></tr></table>	11/2	11/2	11/2	11/3	11/3	11/3	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/3</td><td>11/3</td><td>11/4</td></tr></table>	11/2	11/2	11/2	11/3	11/3	11/4	<table><tr><td>11/3</td><td>11/4</td><td>11/4</td></tr><tr><td>11/5</td><td>11/5</td><td>11/5</td></tr></table>	11/3	11/4	11/4	11/5	11/5	11/5
11/2	11/2	11/2																										
11/2	11/2	11/2																										
11/2	11/2	11/2																										
11/3	11/3	11/3																										
11/2	11/2	11/2																										
11/3	11/3	11/4																										
11/3	11/4	11/4																										
11/5	11/5	11/5																										





Micro-Partitioning

Table: t1

Logical Structure

type	name	country	date
2	A	UK	11/2
4	C	SP	11/2
3	C	DE	11/2
2	B	DE	11/2
3	A	FR	11/2
2	C	SP	11/2
3	Z	DE	11/2
2	B	UK	11/2
4	C	NL	11/2
5	X	FR	11/3
1	A	NL	11/3
5	A	FR	11/3
2	X	FR	11/2
4	Z	NL	11/2
2	Y	SP	11/2
1	B	SP	11/3
5	X	DE	11/3
3	A	UK	11/4
1	C	FR	11/3
4	Z	NL	11/4
5	Y	SP	11/4
5	B	SP	11/5
3	X	DE	11/5
2	Z	UK	11/5

Physical Structure

	Micro-partition 1 (rows 1-6)	Micro-partition 2 (rows 7-12)	Micro-partition 3 (rows 13-18)	Micro-partition 4 (rows 19-24)																								
type	<table><tr><td>2</td><td>4</td><td>3</td></tr><tr><td>2</td><td>3</td><td>2</td></tr></table>	2	4	3	2	3	2	<table><tr><td>3</td><td>2</td><td>4</td></tr><tr><td>5</td><td>1</td><td>5</td></tr></table>	3	2	4	5	1	5	<table><tr><td>2</td><td>4</td><td>2</td></tr><tr><td>1</td><td>5</td><td>3</td></tr></table>	2	4	2	1	5	3	<table><tr><td>1</td><td>4</td><td>5</td></tr><tr><td>5</td><td>3</td><td>2</td></tr></table>	1	4	5	5	3	2
2	4	3																										
2	3	2																										
3	2	4																										
5	1	5																										
2	4	2																										
1	5	3																										
1	4	5																										
5	3	2																										
name	<table><tr><td>A</td><td>C</td><td>C</td></tr><tr><td>B</td><td>A</td><td>C</td></tr></table>	A	C	C	B	A	C	<table><tr><td>Z</td><td>B</td><td>C</td></tr><tr><td>X</td><td>A</td><td>A</td></tr></table>	Z	B	C	X	A	A	<table><tr><td>X</td><td>Z</td><td>Y</td></tr><tr><td>B</td><td>X</td><td>A</td></tr></table>	X	Z	Y	B	X	A	<table><tr><td>C</td><td>Z</td><td>Y</td></tr><tr><td>B</td><td>X</td><td>Z</td></tr></table>	C	Z	Y	B	X	Z
A	C	C																										
B	A	C																										
Z	B	C																										
X	A	A																										
X	Z	Y																										
B	X	A																										
C	Z	Y																										
B	X	Z																										
country	<table><tr><td>UK</td><td>SP</td><td>DE</td></tr><tr><td>DE</td><td>FR</td><td>SP</td></tr></table>	UK	SP	DE	DE	FR	SP	<table><tr><td>DE</td><td>UK</td><td>NL</td></tr><tr><td>FR</td><td>NL</td><td>FR</td></tr></table>	DE	UK	NL	FR	NL	FR	<table><tr><td>FR</td><td>NL</td><td>SP</td></tr><tr><td>SP</td><td>DE</td><td>UK</td></tr></table>	FR	NL	SP	SP	DE	UK	<table><tr><td>FR</td><td>NL</td><td>SP</td></tr><tr><td>SP</td><td>DE</td><td>UK</td></tr></table>	FR	NL	SP	SP	DE	UK
UK	SP	DE																										
DE	FR	SP																										
DE	UK	NL																										
FR	NL	FR																										
FR	NL	SP																										
SP	DE	UK																										
FR	NL	SP																										
SP	DE	UK																										
date	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr></table>	11/2	11/2	11/2	11/2	11/2	11/2	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/3</td><td>11/3</td><td>11/3</td></tr></table>	11/2	11/2	11/2	11/3	11/3	11/3	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/3</td><td>11/3</td><td>11/4</td></tr></table>	11/2	11/2	11/2	11/3	11/3	11/4	<table><tr><td>11/3</td><td>11/4</td><td>11/4</td></tr><tr><td>11/5</td><td>11/5</td><td>11/5</td></tr></table>	11/3	11/4	11/4	11/5	11/5	11/5
11/2	11/2	11/2																										
11/2	11/2	11/2																										
11/2	11/2	11/2																										
11/3	11/3	11/3																										
11/2	11/2	11/2																										
11/3	11/3	11/4																										
11/3	11/4	11/4																										
11/5	11/5	11/5																										

Snowflake stores MP metadata

- Range of values for each of the column in MP
- Number of distinct values
- Additional info which can be used for optimized and efficient query processing





Micro-Partitioning

Table: t1

Logical Structure

type	name	country	date
2	A	UK	11/2
4	C	SP	11/2
3	C	DE	11/2
2	B	DE	11/2
3	A	FR	11/2
2	C	SP	11/2
3	Z	DE	11/2
2	B	UK	11/2
4	C	NL	11/2
5	X	FR	11/3
1	A	NL	11/3
5	A	FR	11/3
2	X	FR	11/2
4	Z	NL	11/2
2	Y	SP	11/2
1	B	SP	11/3
5	X	DE	11/3
3	A	UK	11/4
1	C	FR	11/3
4	Z	NL	11/4
5	Y	SP	11/4
5	B	SP	11/5
3	X	DE	11/5
2	Z	UK	11/5

Physical Structure

	Micro-partition 1 (rows 1-6)	Micro-partition 2 (rows 7-12)	Micro-partition 3 (rows 13-18)	Micro-partition 4 (rows 19-24)																								
type	<table><tr><td>2</td><td>4</td><td>3</td></tr><tr><td>2</td><td>3</td><td>2</td></tr></table>	2	4	3	2	3	2	<table><tr><td>3</td><td>2</td><td>4</td></tr><tr><td>5</td><td>1</td><td>5</td></tr></table>	3	2	4	5	1	5	<table><tr><td>2</td><td>4</td><td>2</td></tr><tr><td>1</td><td>5</td><td>3</td></tr></table>	2	4	2	1	5	3	<table><tr><td>1</td><td>4</td><td>5</td></tr><tr><td>5</td><td>3</td><td>2</td></tr></table>	1	4	5	5	3	2
2	4	3																										
2	3	2																										
3	2	4																										
5	1	5																										
2	4	2																										
1	5	3																										
1	4	5																										
5	3	2																										
name	<table><tr><td>A</td><td>C</td><td>C</td></tr><tr><td>B</td><td>A</td><td>C</td></tr></table>	A	C	C	B	A	C	<table><tr><td>Z</td><td>B</td><td>C</td></tr><tr><td>X</td><td>A</td><td>A</td></tr></table>	Z	B	C	X	A	A	<table><tr><td>X</td><td>Z</td><td>Y</td></tr><tr><td>B</td><td>X</td><td>A</td></tr></table>	X	Z	Y	B	X	A	<table><tr><td>C</td><td>Z</td><td>Y</td></tr><tr><td>B</td><td>X</td><td>Z</td></tr></table>	C	Z	Y	B	X	Z
A	C	C																										
B	A	C																										
Z	B	C																										
X	A	A																										
X	Z	Y																										
B	X	A																										
C	Z	Y																										
B	X	Z																										
country	<table><tr><td>UK</td><td>SP</td><td>DE</td></tr><tr><td>DE</td><td>FR</td><td>SP</td></tr></table>	UK	SP	DE	DE	FR	SP	<table><tr><td>DE</td><td>UK</td><td>NL</td></tr><tr><td>FR</td><td>NL</td><td>FR</td></tr></table>	DE	UK	NL	FR	NL	FR	<table><tr><td>FR</td><td>NL</td><td>SP</td></tr><tr><td>SP</td><td>DE</td><td>UK</td></tr></table>	FR	NL	SP	SP	DE	UK	<table><tr><td>FR</td><td>NL</td><td>SP</td></tr><tr><td>SP</td><td>DE</td><td>UK</td></tr></table>	FR	NL	SP	SP	DE	UK
UK	SP	DE																										
DE	FR	SP																										
DE	UK	NL																										
FR	NL	FR																										
FR	NL	SP																										
SP	DE	UK																										
FR	NL	SP																										
SP	DE	UK																										
date	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr></table>	11/2	11/2	11/2	11/2	11/2	11/2	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/3</td><td>11/3</td><td>11/3</td></tr></table>	11/2	11/2	11/2	11/3	11/3	11/3	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/3</td><td>11/3</td><td>11/4</td></tr></table>	11/2	11/2	11/2	11/3	11/3	11/4	<table><tr><td>11/3</td><td>11/4</td><td>11/4</td></tr><tr><td>11/5</td><td>11/5</td><td>11/5</td></tr></table>	11/3	11/4	11/4	11/5	11/5	11/5
11/2	11/2	11/2																										
11/2	11/2	11/2																										
11/2	11/2	11/2																										
11/3	11/3	11/3																										
11/2	11/2	11/2																										
11/3	11/3	11/4																										
11/3	11/4	11/4																										
11/5	11/5	11/5																										

Query Pruning





Micro-Partitioning

Table: t1

Logical Structure

type	name	country	date
2	A	UK	11/2
4	C	SP	11/2
3	C	DE	11/2
2	B	DE	11/2
3	A	FR	11/2
2	C	SP	11/2
3	Z	DE	11/2
2	B	UK	11/2
4	C	NL	11/2
5	X	FR	11/3
1	A	NL	11/3
5	A	FR	11/3
2	X	FR	11/2
4	Z	NL	11/2
2	Y	SP	11/2
1	B	SP	11/3
5	X	DE	11/3
3	A	UK	11/4
1	C	FR	11/3
4	Z	NL	11/4
5	Y	SP	11/4
5	B	SP	11/5
3	X	DE	11/5
2	Z	UK	11/5

Physical Structure

	Micro-partition 1 (rows 1-6)	Micro-partition 2 (rows 7-12)	Micro-partition 3 (rows 13-18)	Micro-partition 4 (rows 19-24)
type	2 4 3 2 3 2	3 2 4 5 1 5	2 4 2 1 5 3	1 4 5 5 3 2
name	A C C B A C	Z B C X A A	X Z Y B X A	C Z Y B X Z
country	UK SP DE DE FR SP	DE UK NL FR NL FR	FR NL SP SP DE UK	FR NL SP SP DE UK
date	11/2 11/2 11/2 11/2 11/2 11/2	11/2 11/2 11/2 11/3 11/3 11/3	11/2 11/2 11/2 11/3 11/3 11/4	11/3 11/4 11/4 11/5 11/5 11/5

Query Pruning

- Skipping unnecessary scan of micro-partition is Query Pruning
- Improves the performance





Data Clustering

“Clustering is a technique by which Snowflake co-locate same value data in same micro-partitions, To improve the performance and optimize the cost”





Data Clustering

- It is automatic (Once enabled manually)
- Reduces the scanning of micro-partitions
- Improves the performance
- Storage Cost
- Compute Cost (Serverless)





Data Clustering

How to choose candidate for Clustering:

- Table with large number of Micro-partitions
- Queries need to read small amount of data (WHERE clause)
- Queries Sort the data (Order by)
- Many/most queries do select on, or sort on, the same few column(s)





Data Clustering

Snowflake maintains clustering metadata

- Total Number of Micro-Partitions
- Number of micro-partitions containing values that overlap with each other
- Depth of the overlapping partitions





Data Clustering

			Overlapping MP	Overlap Depth
		A-Z	5	5
		A-Z		
		A-Z		
		A-Z		
		A-Z		
		L-Z	3	3
		L-Z		
		L-Z		
A-E	F-K			





Data Clustering

Overlapping MP

Overlap Depth

Q-Z

L-S

A-D

E-J

K-N

3

2

A-D

E-J

K-N

O-S

T-Z

0

1





Data Clustering

SYSTEM\$CLUSTERING_DEPTH

SYSTEM\$CLUSTERING_INFORMATION (including clustering depth)



Clustering Key

Just like Index/Key in RDBMS System, It is set of columns which are design to co-locate in micro-partitions.





Clustering Key



Clustering Key

How to choose Clustering Keys :

- Use column most used in where clause
- Choose Join predicates
- Balance out the cardinality (Lower >> minimal pruning, Higher >> Inefficient grouping)
- For Multi Clustering Key move from low to high cardinality





Clustering Key

Logical Structure

type	name	country	date
2	A	UK	11/2
4	C	SP	11/2
3	C	DE	11/2
2	B	DE	11/2
3	A	FR	11/2
2	C	SP	11/2
3	Z	DE	11/2
2	B	UK	11/2
4	C	NL	11/2
5	X	FR	11/3
1	A	NL	11/3
5	A	FR	11/3
2	X	FR	11/2
4	Z	NL	11/2
2	Y	SP	11/2
1	B	SP	11/3
5	X	DE	11/3
3	A	UK	11/4
1	C	FR	11/3
4	Z	NL	11/4
5	Y	SP	11/4
5	B	SP	11/5
3	X	DE	11/5
2	Z	UK	11/5

Physical Structure

Original Micro-partitions															
Micro-partition 1 (rows 1-6)				Micro-partition 2 (rows 7-12)				Micro-partition 3 (rows 13-18)				Micro-partition 4 (rows 19-24)			
2	type	2 4 3			3 2 4			2 4 2			1 4 5				
		2 3 2			5 1 5			1 5 3			5 3 2				
	name														
	country														
1	date	11/2 11/2 11/2			11/2 11/2 11/2			11/2 11/2 11/2			11/3 11/4 11/4				
		11/2 11/2 11/2			11/3 11/3 11/3			11/3 11/3 11/4			11/5 11/5 11/5				

```
SELECT name, country FROM t1
WHERE type = 2
AND date = '11/2';
```

Clustering Key

```
ALTER TABLE t1
CLUSTER BY (date, type);
```

Physical Structure

Original Micro-partitions

Micro-partition 1 (rows 1-6)				Micro-partition 2 (rows 7-12)				Micro-partition 3 (rows 13-18)				Micro-partition 4 (rows 19-24)			
2	type	2	4	3	3	2	4	2	4	2	1	4	5		
		2	3	2	5	1	5	1	5	3	5	3	2		
	name														
	country														
1	date	11/2	11/2	11/2	11/2	11/2	11/2	11/2	11/2	11/2	11/3	11/4	11/4		
		11/2	11/2	11/2	11/3	11/3	11/3	11/3	11/3	11/4	11/5	11/5	11/5		

New Micro-partitions (After Reclustering)

	Micro-partition 5 (rows 1, 4, 6, 8, 13, 15)	Micro-partition 6 (rows 3, 5, 7, 2, 9, 14)	Micro-partition 7 (rows 10, 12, 17, 11, 16, 19)	Micro-partition 8 (rows 18, 20-24)
type	222	333	555	345
	222	444	111	532
name				
country				
date	11/211/211/2	11/211/211/2	11/311/311/3	11/411/411/4
	11/211/211/2	11/211/211/2	11/311/311/3	11/511/511/5

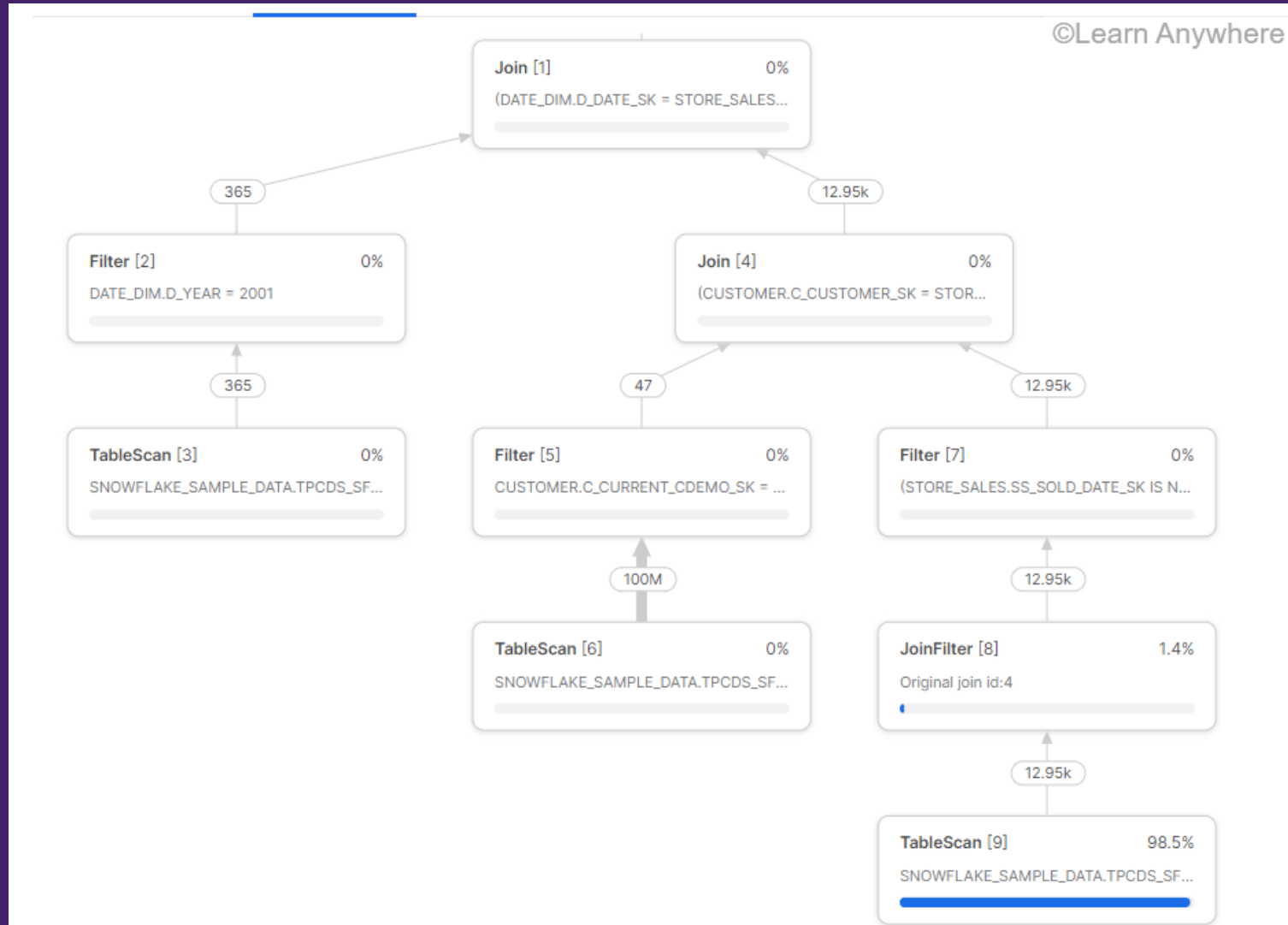
```
SELECT name, country FROM t1
WHERE type = 2
AND date = '11/2';
```

Query Profile





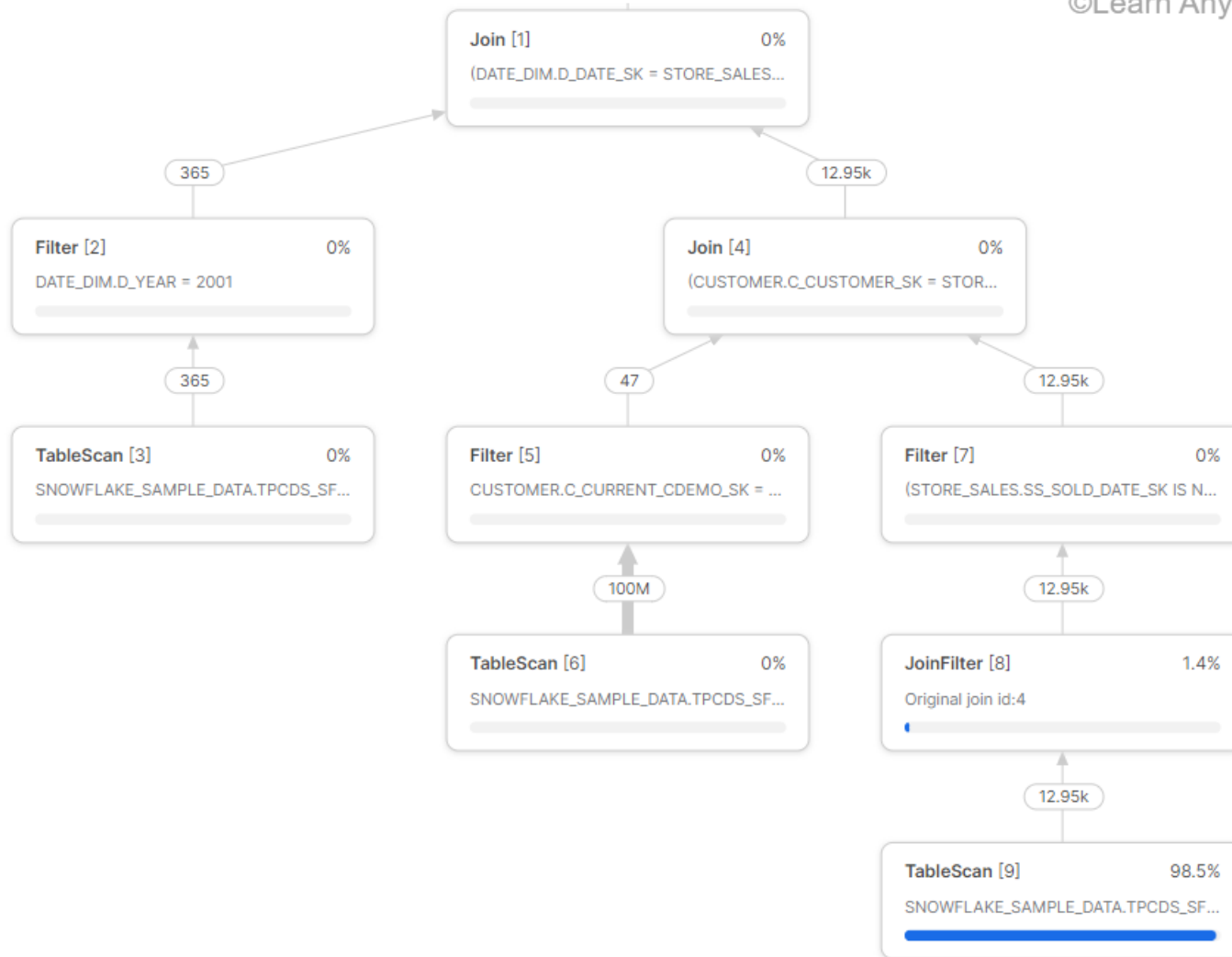
Query Profile





Query Profile

©Learn Anywhere



©Learn Anywhere

Most Expensive Nodes (2 of ...)

TableScan [9]	98.5%
JoinFilter [8]	1.4%

Profile Overview (Finished)

Total Execution Time (15m 26s) 100.0%

Processing	3.1%
Local Disk I/O	2.0%
Remote Disk I/O	94.8%
Initialization	0.1%

Statistics

Scan progress	100.00%
Bytes scanned	310.79GB
Percentage scanned from cache	0.00%
Bytes written to result	0.11MB
Partitions scanned	139738
Partitions total	721729



Query Profile : Data Spilling

©Learn Anywhere

Statistics

Scan progress	25.75%
Bytes scanned	541.63GB
Percentage scanned from cache	0.00%
Partitions scanned	196697
Partitions total	784050
Bytes spilled to local storage	400.87GB
Bytes spilled to remote storage	132.40GB

©Learn Anywhere

Statistics

Scan progress	27.87%
Bytes scanned	599.96GB
Percentage scanned from cache	0.00%
Bytes sent over the network	226.96GB
Partitions scanned	212942
Partitions total	784050
Bytes spilled to local storage	447.91GB





Query Profile

EXPLAIN PLANS

EXPLAIN [USING { TABULAR | JSON | TEXT }] <statement>

Query History (Option under Monitoring)

Provides 14 days history

QUERY_HISTORY (Under Information Schema)

Last 7 days activity

QUERY_HISTORY View (Under SF Account Usage)

Up to 365 days activity



Information Schema



Information Schema

The Snowflake Information Schema (aka “Data Dictionary”) consists of a set of system-defined views and table functions that provide extensive metadata information about the objects created in your account.

The Snowflake Information Schema is based on the SQL-92 ANSI Information Schema, but with the addition of views and functions that are specific to Snowflake.





Information Schema

- Read Only Schema
- Available for every DB
- Maintains data about all DB object and some account level information, In form of Views
- Table function for historical and usage data across your account
- Need Privilege



Account Usage



Account Usage

- Views that display object metadata and usage metrics for your account.
- Account_Usage and Readers_Account_Usage
- Very similar to Information Schema with some differences



Account Usage Vs Information Schema

Parameters	Account Usage	Information Schema
Includes dropped object	Yes	No
Latency of data	45 Mins – 3 Hours	Instant
Retention Period	1 Year	7 Days to 6 Months





Access History





Access History

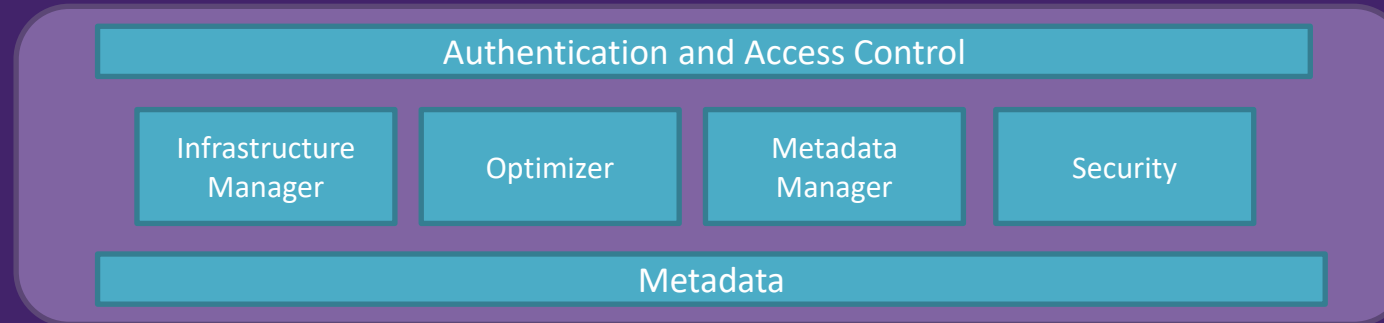
- Needs Enterprise Edition or Higher
- Maintains history of read and write operations
- Used for compliance auditing
- Stores data on source column, object accessed directly and indirectly

*QUERY_ID | QUERY_START_TIME | USER_NAME | DIRECT_OBJECTS_ACCESSED |
BASE_OBJECTS_ACCESSED | OBJECTS_MODIFIED | OBJECT_MODIFIED_BY_DDL |
POLICIES_REFERENCED*

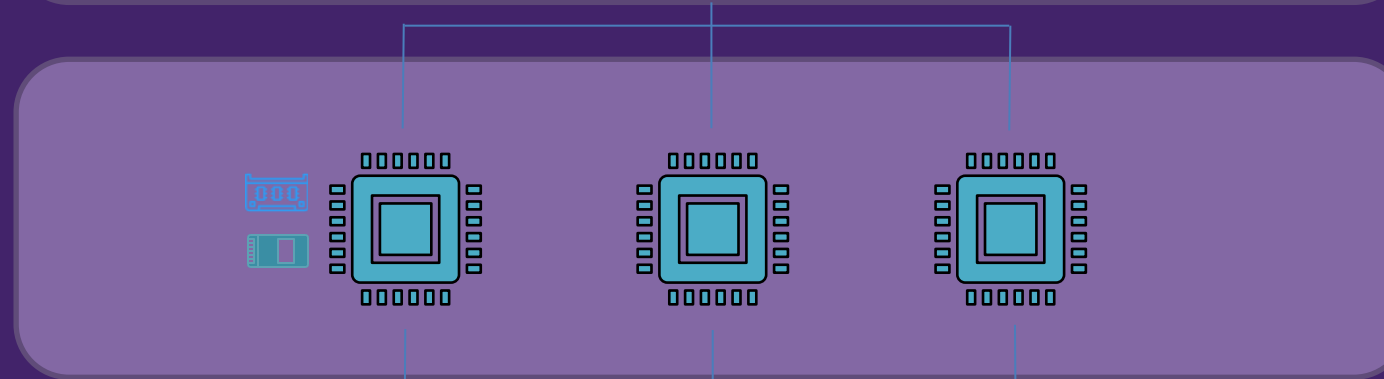


Snowflake Architecture

Cloud Services



Query Processing Layer



Database Storage



Query Result Cache

Metadata Cache

Warehouse Cache





Caching

Query Result Cache (USE_CACHED_RESULT)

- In Cloud services layer
- Expires in 24 hours
- Work for exact same query
- Doesn't work with functions, which evaluates at runtime eg `CURRENT_TIMESTAMP()` and `UUID_STRING()`, `CURRENT_DATE()` is exception
- Doesn't work with external functions
- Doesn't work when , table change, config change, Micro-partitions change





Caching

Metadata Cache

- In Cloud services layer
- Stores Metadata Information
- Warehouse not needed
- Row Count, MIN/MAX values, DISTINCT count etc.





Caching

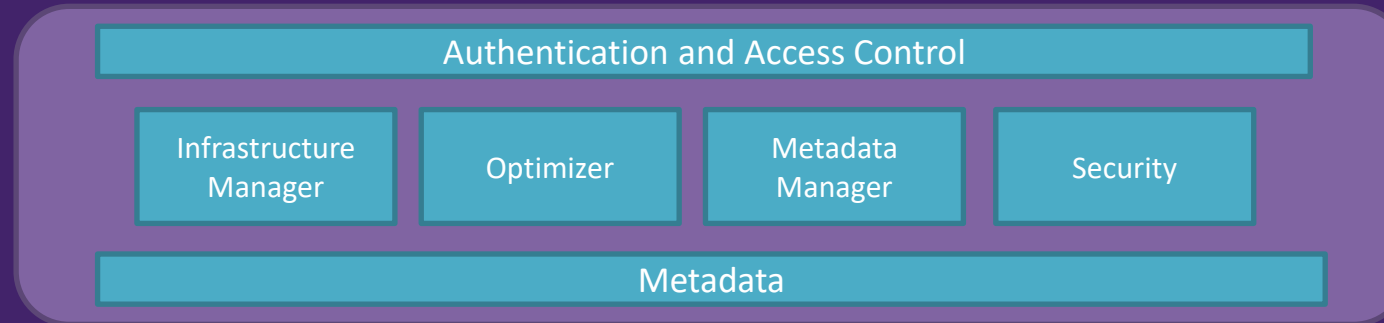
Warehouse Cache (Local Disk Cache)

- At Query Processing Layer
- Local to the WH
- Purged when WH is suspended or shut
- Need to find whether it is worth to auto-suspend a WH

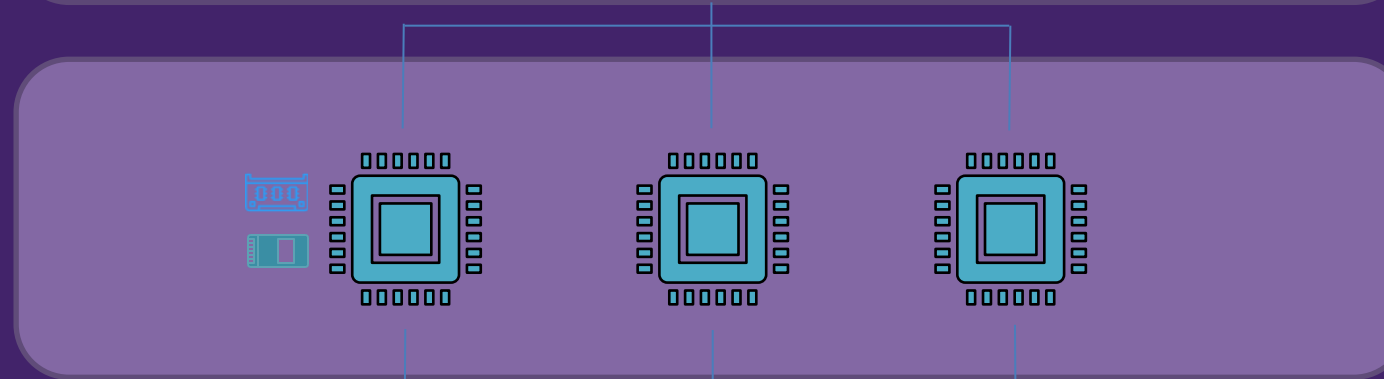


Snowflake Architecture

Cloud Services



Query Processing Layer



Database Storage



Query Result Cache

Metadata Cache

Warehouse Cache



*(Needs Enterprise
Edition or Higher)*

Search Optimization Service



Search Optimization Service

The search optimization service can significantly improve the performance of certain types of lookup and analytical queries that use an extensive set of predicates for filtering.



Search Optimization Service

What Kind of Queries can be improved?

- Selective point look up queries, returns small distinct rows
- Substring or regular expression searches (LIKE, ILIKE etc.)
- Queries on fields in VARIANT, OBJECT, ARRAY columns
(Equality, IN Predicates etc.)
- Queries using selected geo func

```
SELECT * FROM SAMPLE_TABLE WHERE CUSTOMER = '1234'
```



Search Optimization Service

*SELECT * FROM SAMPLE_TABLE WHERE CUSTOMER = '1234'*



Search Optimization Service

*SELECT * FROM SAMPLE_TABLE WHERE CUSTOMER = '1234'*



SOS creates and maintains a persistent data structure called [search access path](#).



Search Optimization Service

- *Search Access Path* gets updated with data needed for look up
- New updates in table gets automatically updated
- Check progress `search_optimization_progress` in SHOW TABLES
- Cost for storage and compute



Search Optimization Service

- *Search Access Path* gets updated with data needed for look up
- New updates in table gets automatically updated
- Check progress `search_optimization_progress` in SHOW TABLES
- Cost for storage and compute



Search Optimization Service

Before SOS

- Query should have a column with 100-200k distinct values
- Selective filters brings very few records
- Query runs at least for few seconds



Search Optimization Service

SOS Vs Clustering

- SOS applies for all columns
- SOS uses additional data structure



Query Acceleration Service



Query Acceleration Service

- QAS can accelerate part of WH workload
- Reduces the impact of outlier query
- Works for
 - Ad hoc Analysis
 - Unpredictable data volume per query
 - Queries with large scan and selective filter



Resource Monitoring

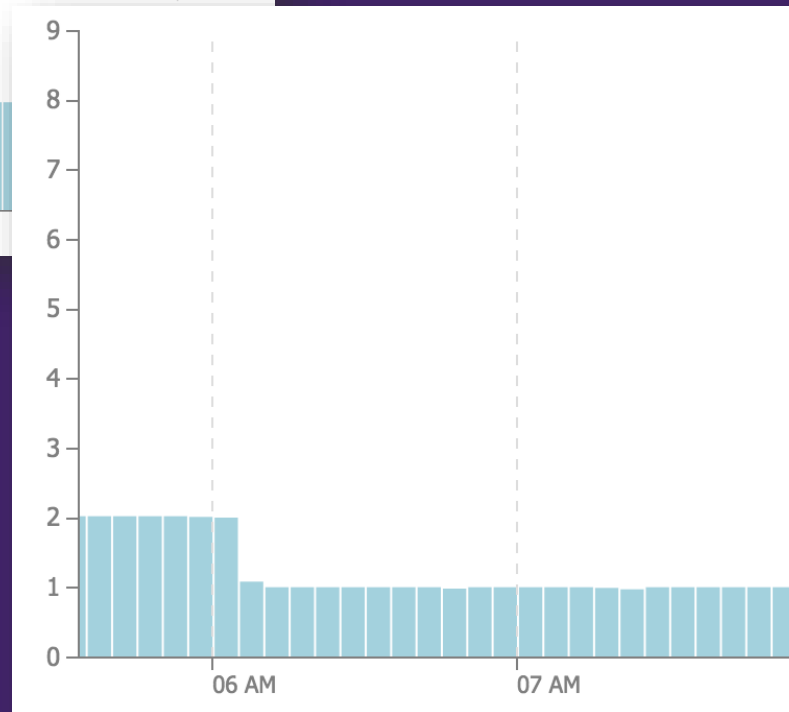
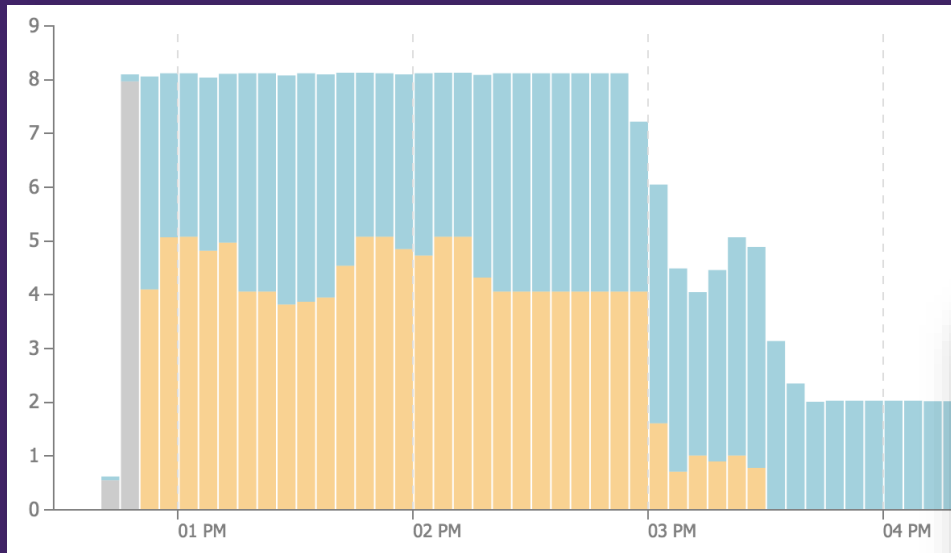


Resource Monitoring

- Monitoring credit usage by user managed WH and cloud services (Not Serverless)
- Can set at Account level or WH level
- Can set Credit Quota (Eg, 100 Credit)
- Monitor consumption by User managed WH and Cloud services
- You can define frequency at which monitoring resets
(Daily, Weekly, Monthly, Yearly, Never)
- Action on reaching threshold
 - *Suspend Immediately*
 - *Suspend*
 - *Notify*

```
create or replace resource monitor limiter with credit_quota=5000
notify_users = (jdoe, "jane smith", "john doe")
triggers on 75 percent do notify
on 100 percent do suspend
on 110 percent do suspend_immediate
```

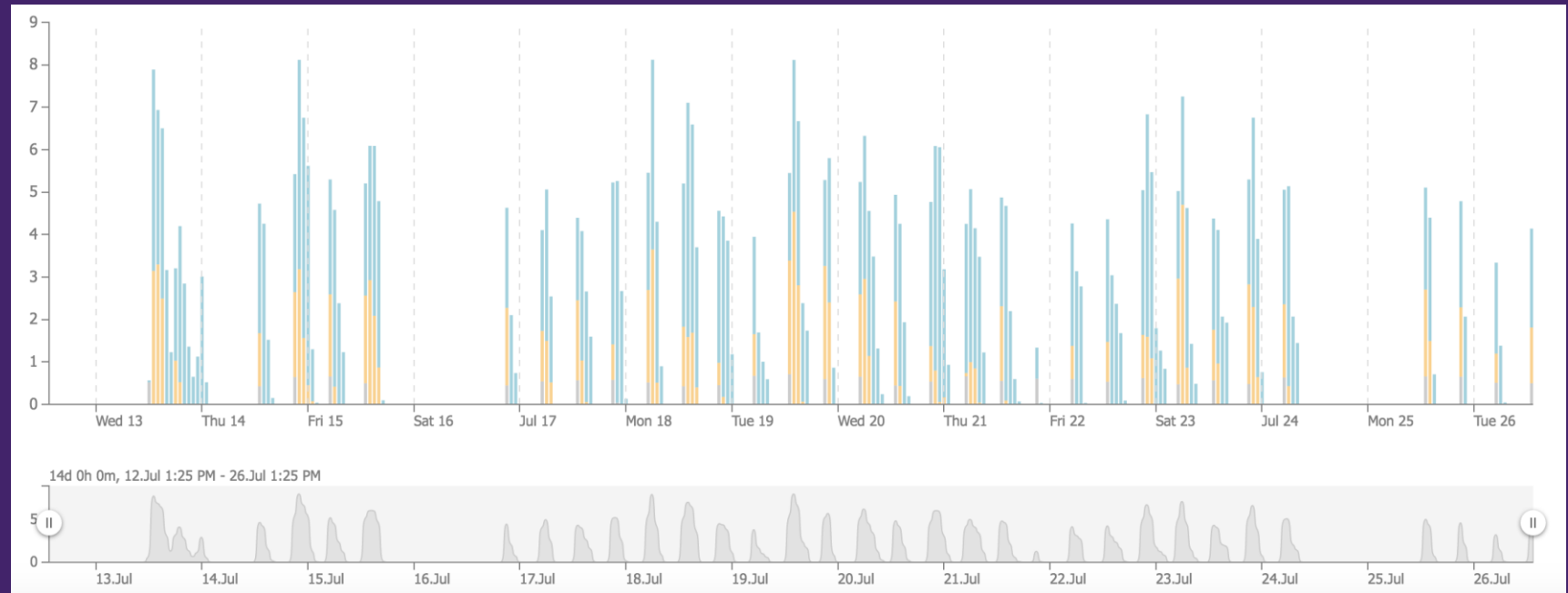




Source : Snowflake



Monitoring Warehouse Load



Source : Snowflake



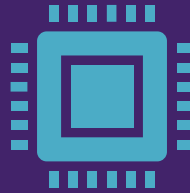
Performance Optimization



Performance Optimization



Storage



Warehouse



Query



Performance Optimization

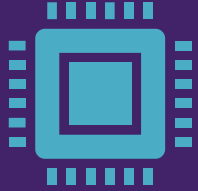


Storage

- Use Clustering
- Use Search Optimization Service (SOS)
- Use Materialized Views
- Purge stage data
- Use temporary table, where applicable
- Save on Time-Travel (`DATA_RETENTION_TIME_IN_DAYS`)
- Monitor Memory consumption



Performance Optimization



Warehouse

- Reduce Queues
- Resolve memory Spillage
- Right Size WH
- Query Acceleration Service (QAS)
- WH Cache Optimization
- Limit concurrently running queries (MAX_CONCURRENCY_LEVEL)
- Use different WH for different task
- Monitor Compute
- Resource monitoring



Performance Optimization



Query

- Use Query Profiles stats
- Use Query History
- Avoid using SELECT *, Use columns
- Use filtering, where it is applicable





Access Control Framework





Access Control Framework

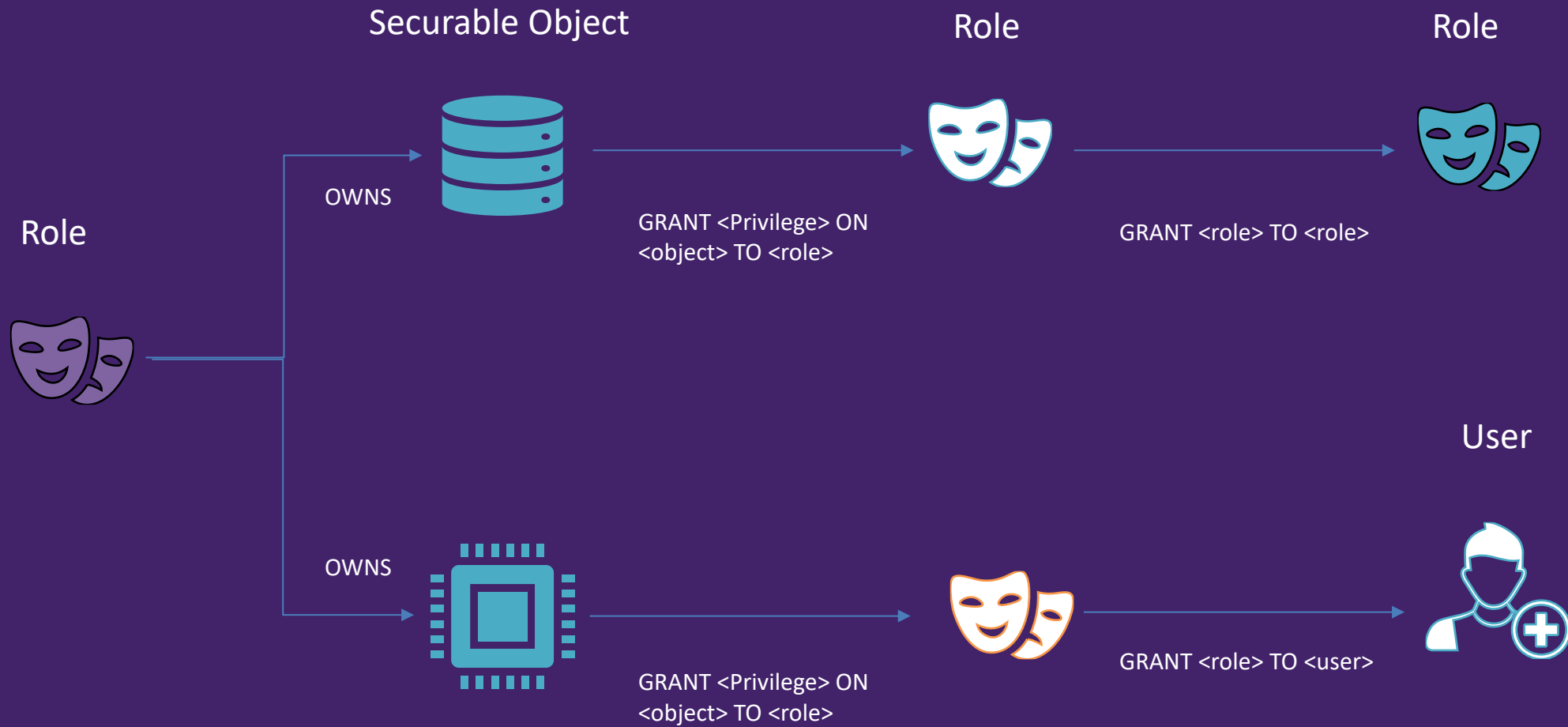
Discretionary Access Control (DAC): Each object has an owner, who can in turn grant access to that object.

Role-based Access Control (RBAC): Access privileges are assigned to roles, which are in turn assigned to users.





Access Control Framework





Access Control Framework

- **Securable object:** An entity to which access can be granted. Unless allowed by a grant, access is denied.
- **Role:** An entity to which privileges can be granted. Roles are in turn assigned to users. Note that roles can also be assigned to other roles, creating a role hierarchy.
- **Privilege:** A defined level of access to an object. Multiple distinct privileges may be used to control the granularity of access granted.
- **User:** A user identity recognized by Snowflake, whether associated with a person or program.





Types of Role

Account roles

To permit SQL actions on any object in your account, grant privileges on the object to an account role.

Database Roles (Preview)

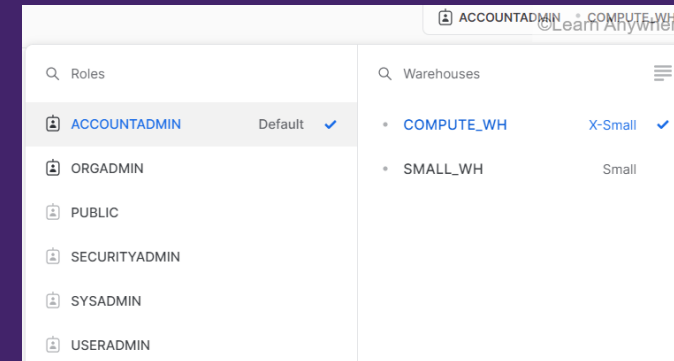
To limit SQL actions to a single database, as well as any object in the database, grant privileges on the object to a database role in the same database.



Types of Active Role

Primary Role

The default assumed Role
USE ROLE <ROLE NAME>



Secondary Role

Can assign any number of secondary role.

```
15  USE SECONDARY ROLES SYSADMIN;
16  SELECT CURRENT_SECONDARY_ROLES();
17
```

©Learn Anywhere

Results Chart

	CURRENT_SECONDARY_ROLES()
1	{"roles":"SYSADMIN","value":"SYSADMIN"}

System Defined Roles





System Defined Roles

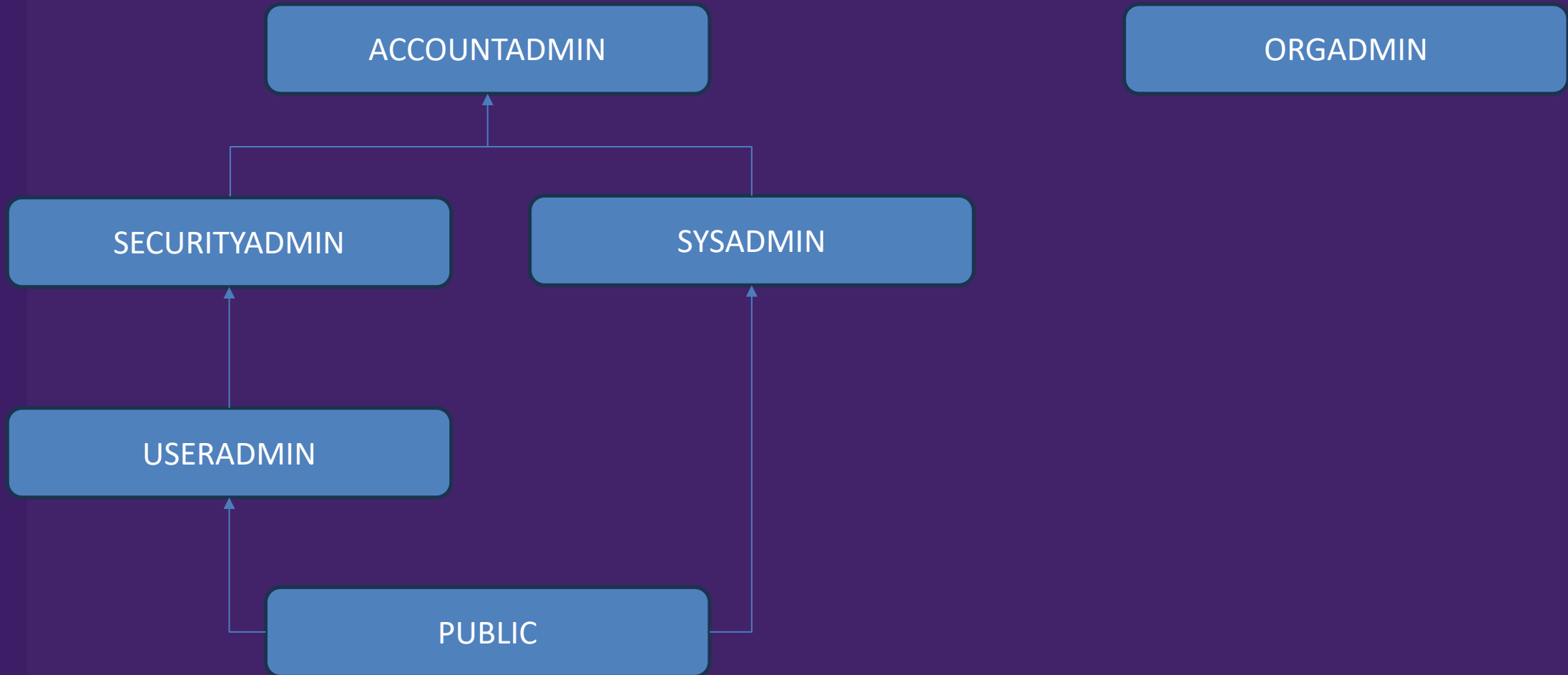
ORGADMIN

- Role that manages ops at organization level
- Can create account in the organization
- Can view all account in the organization and all enabled region
SHOW ORGANIZATION ACCOUNTS, SHOW REGIONS
- Can check usage information across the organization



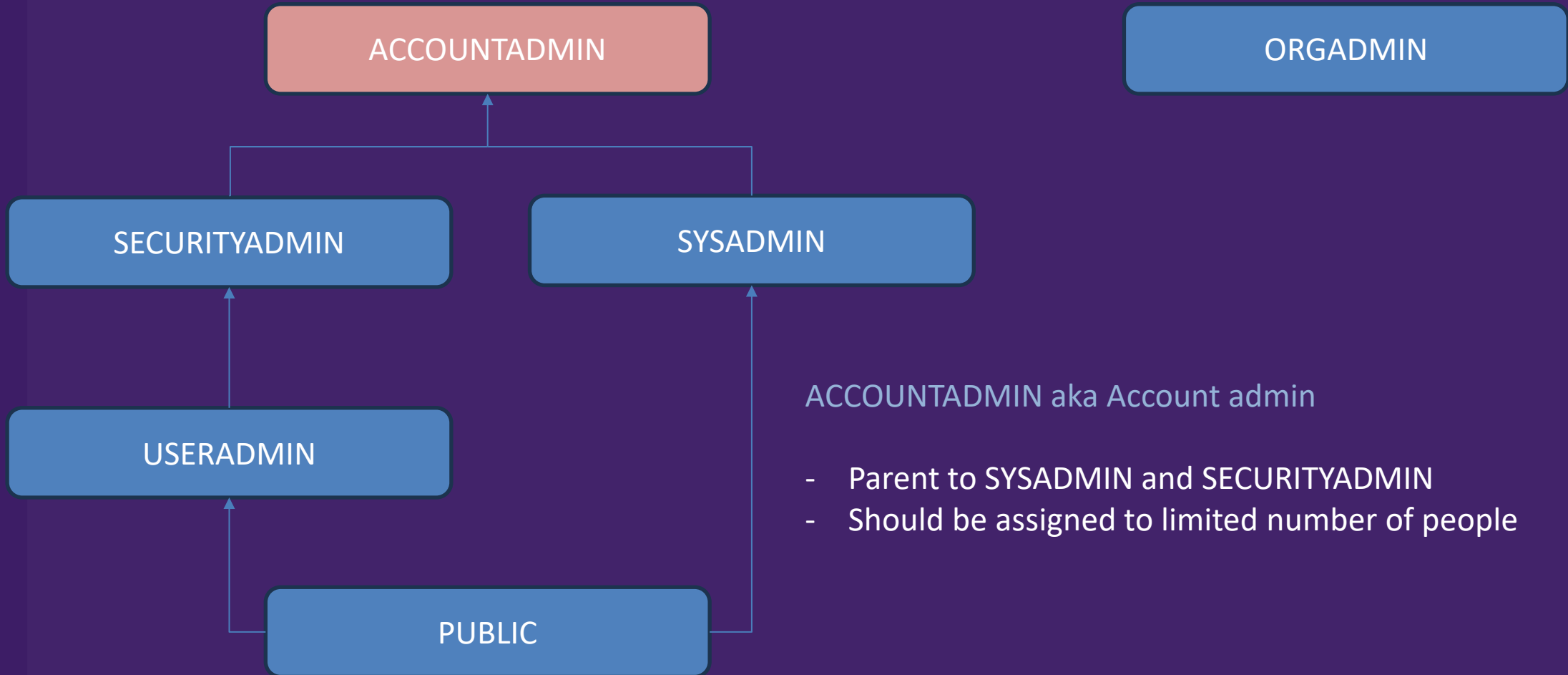


System Defined Roles





System Defined Roles



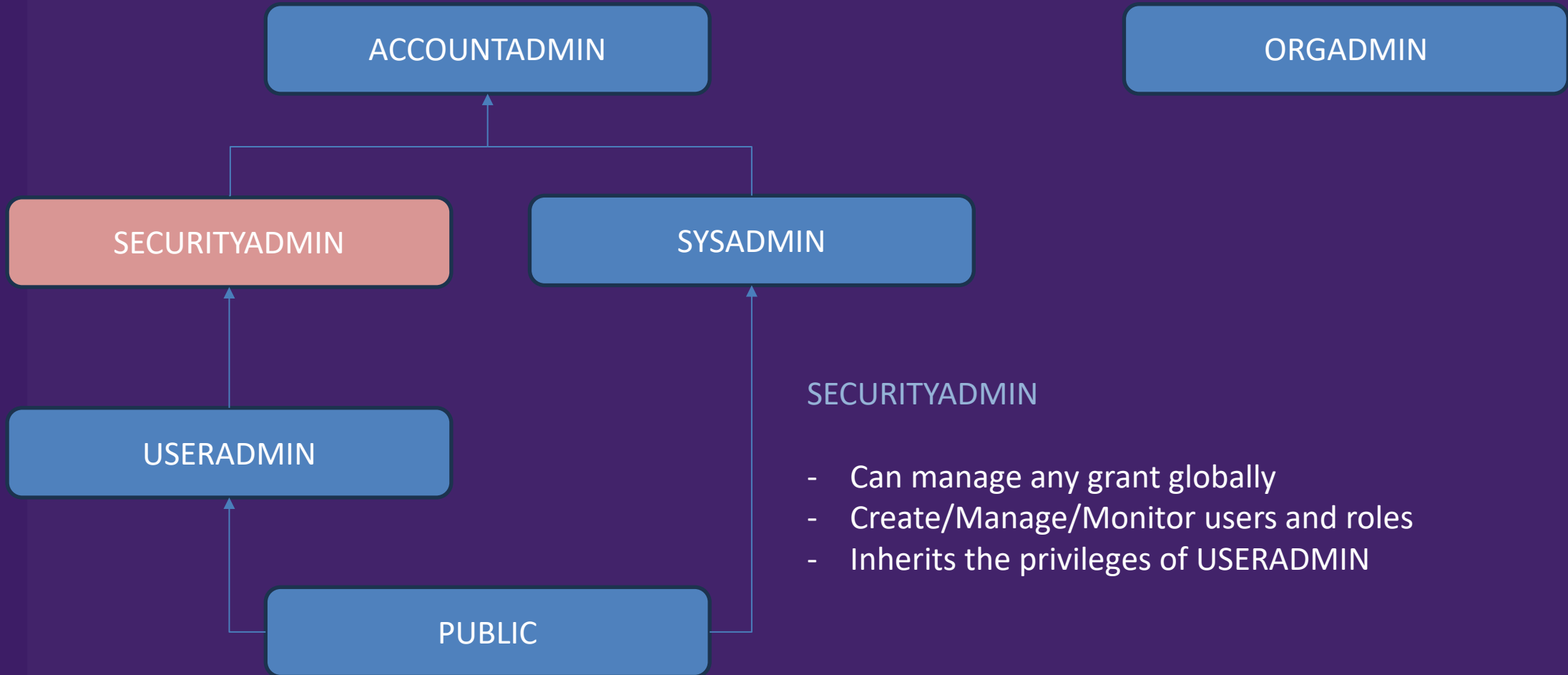
ACCOUNTADMIN aka Account admin

- Parent to SYSADMIN and SECURITYADMIN
- Should be assigned to limited number of people



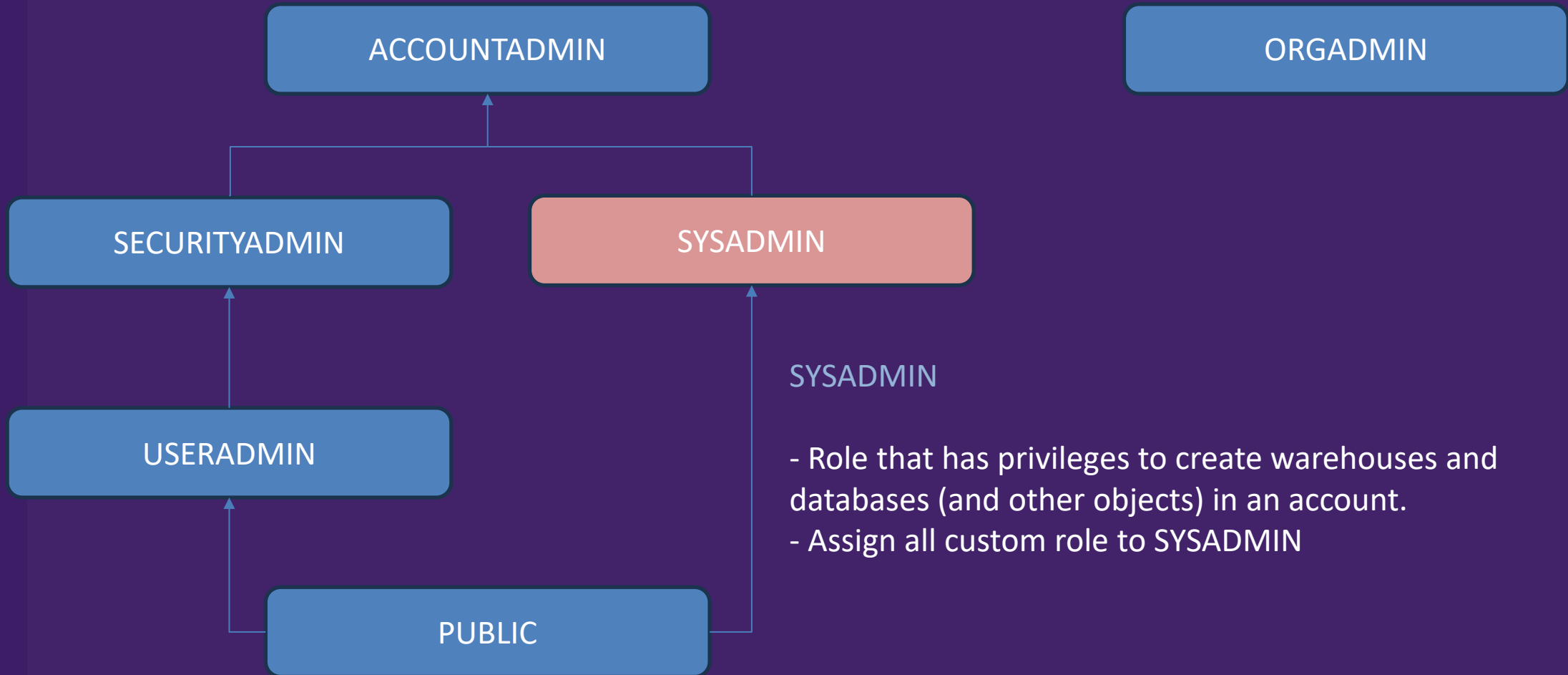


System Defined Roles





System Defined Roles



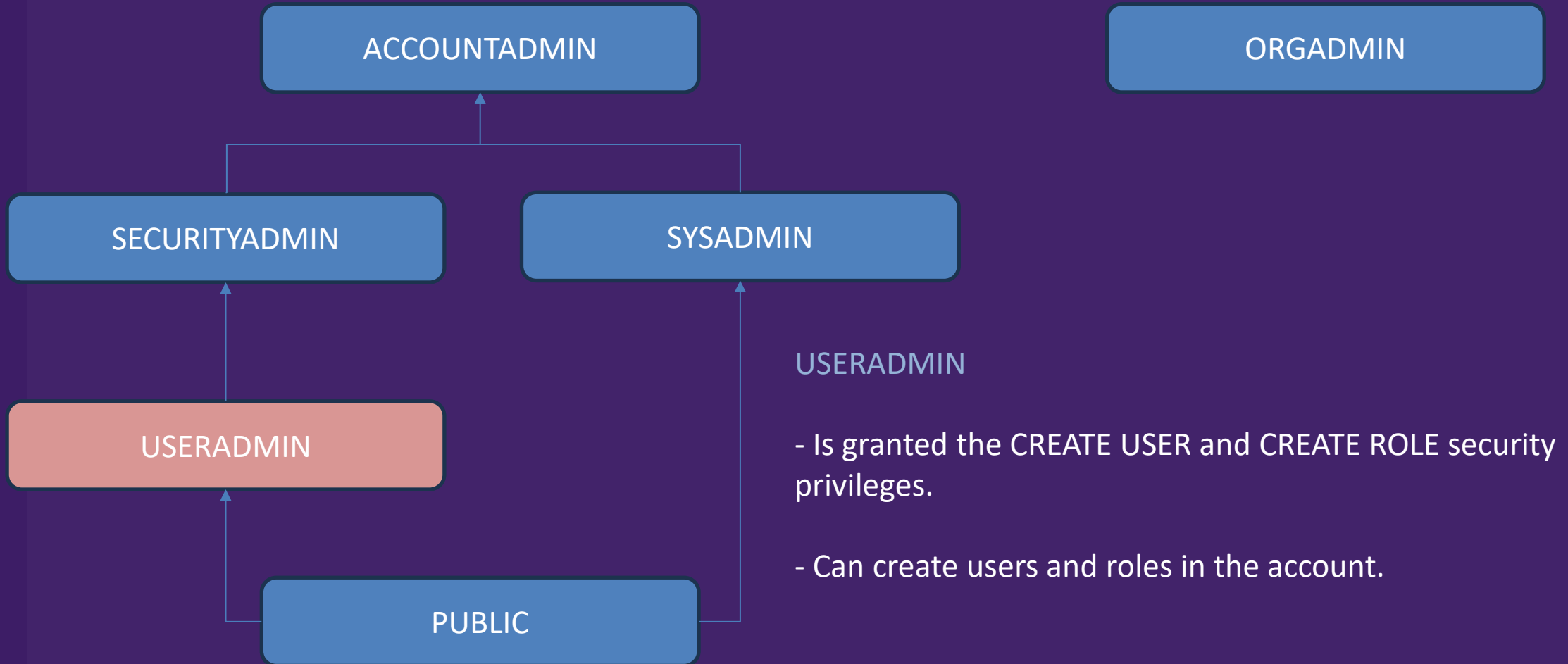
SYSADMIN

- Role that has privileges to create warehouses and databases (and other objects) in an account.
- Assign all custom role to SYSADMIN



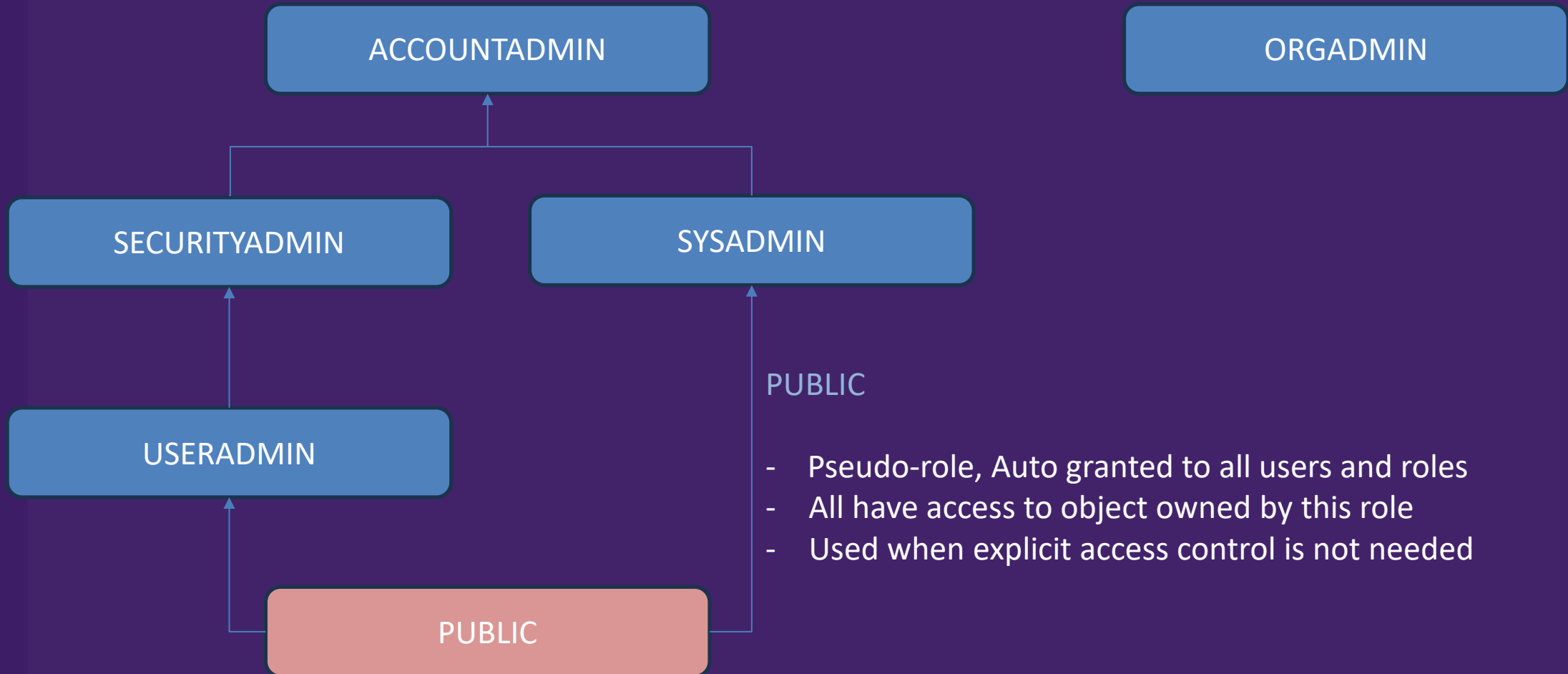


System Defined Roles



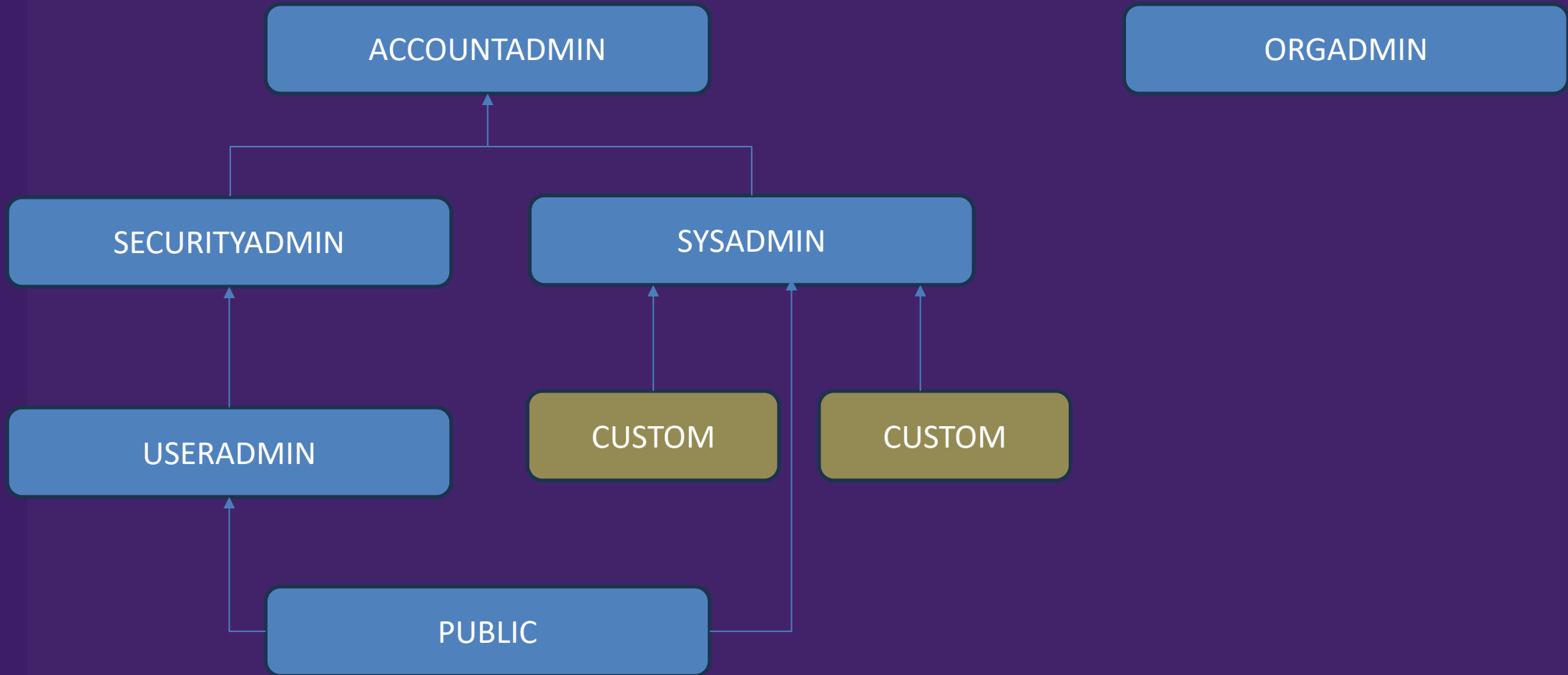


System Defined Roles





System Defined Roles



A blue icon depicting a person in a suit standing above a three-level organizational chart.

Access Control Privilege





Access Control Privilege

Privileges are granted to roles, and roles are granted to users, to specify the operations that the users can perform on objects in the system.



GRANT <Privilege> ON
<object> TO <role>

Role





Access Control Privilege

Table Privilege:

SELECT
TRUNCATE
DELETE
OWNERSHIP
...

GRANT SELECT ON TABLE <Table> TO ROLE <Role>;

More details : <https://docs.snowflake.com/en/user-guide/security-access-control-privileges>



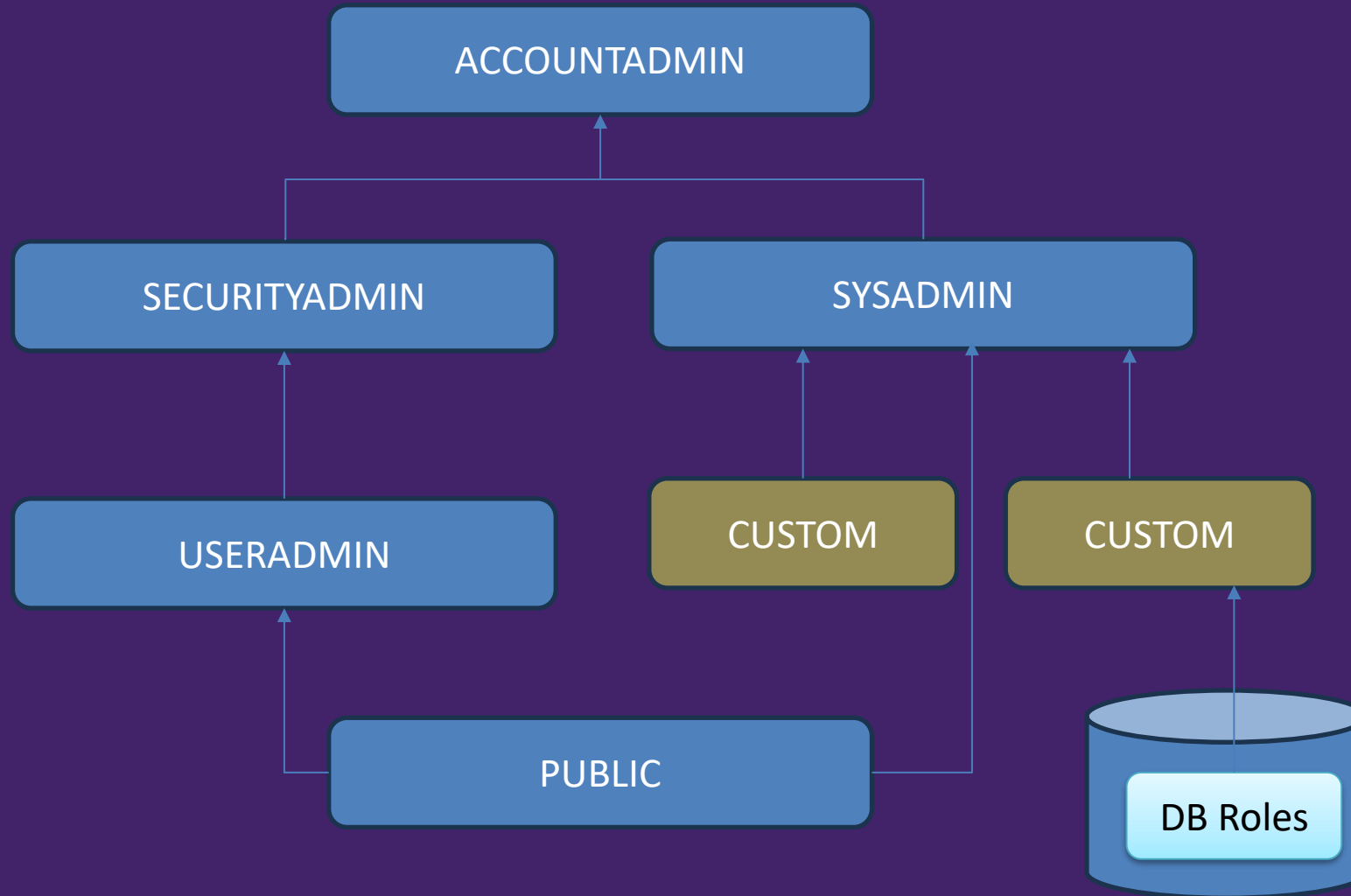


Role Hierarchy & Privilege Inheritance





Role Hierarchy & Privilege Inheritance





Role Hierarchy & Privilege Inheritance

Securable Object

Role



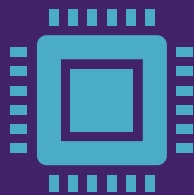
GRANT <Privilege> ON
<object> TO <role>



GRANT <role> TO <role>



GRANT <role> TO <user>



GRANT <Privilege> ON
<object> TO <role>

GRANT <Privilege> ON
<object> TO <role>

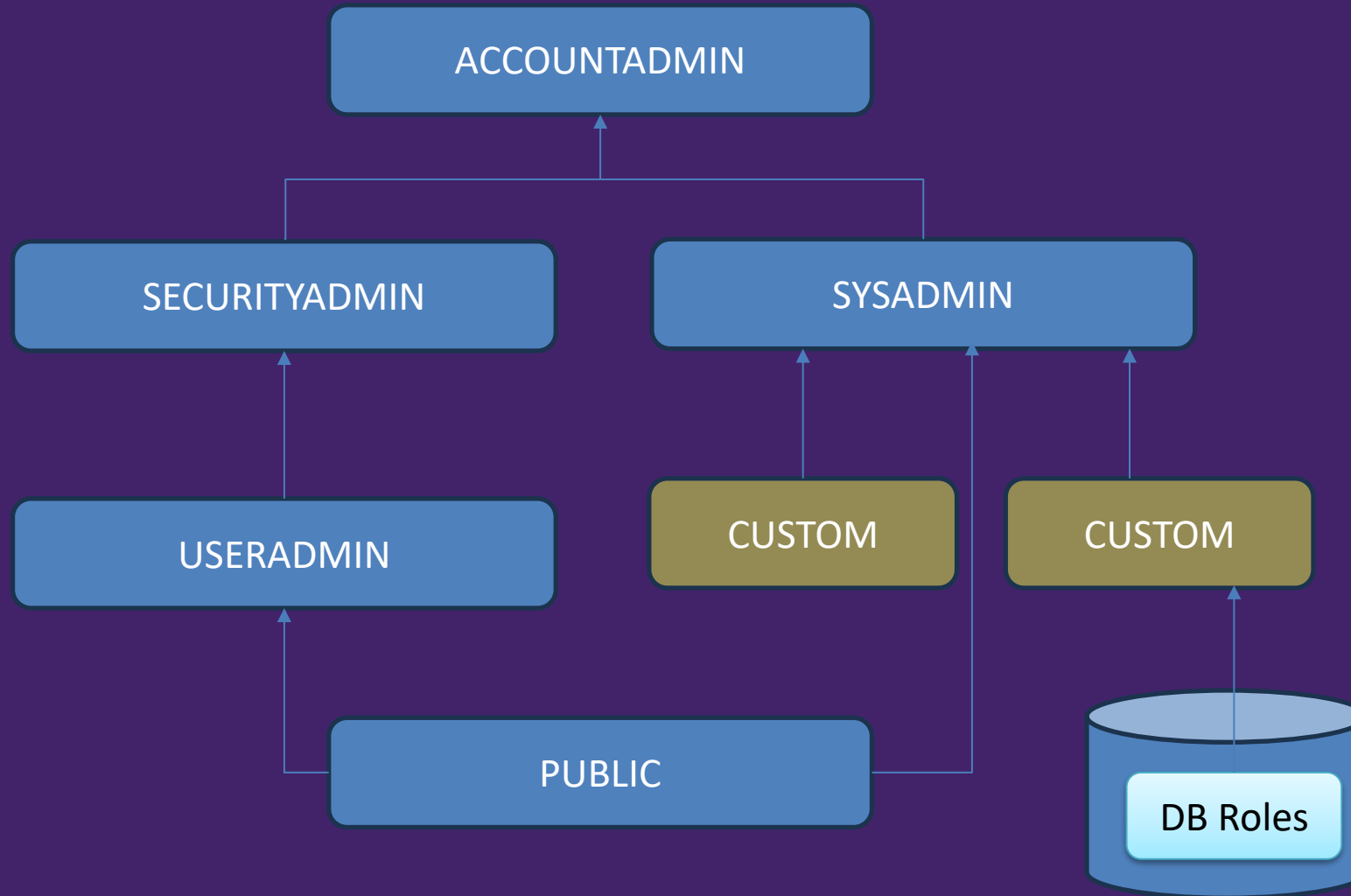


GRANT <role> TO <role>





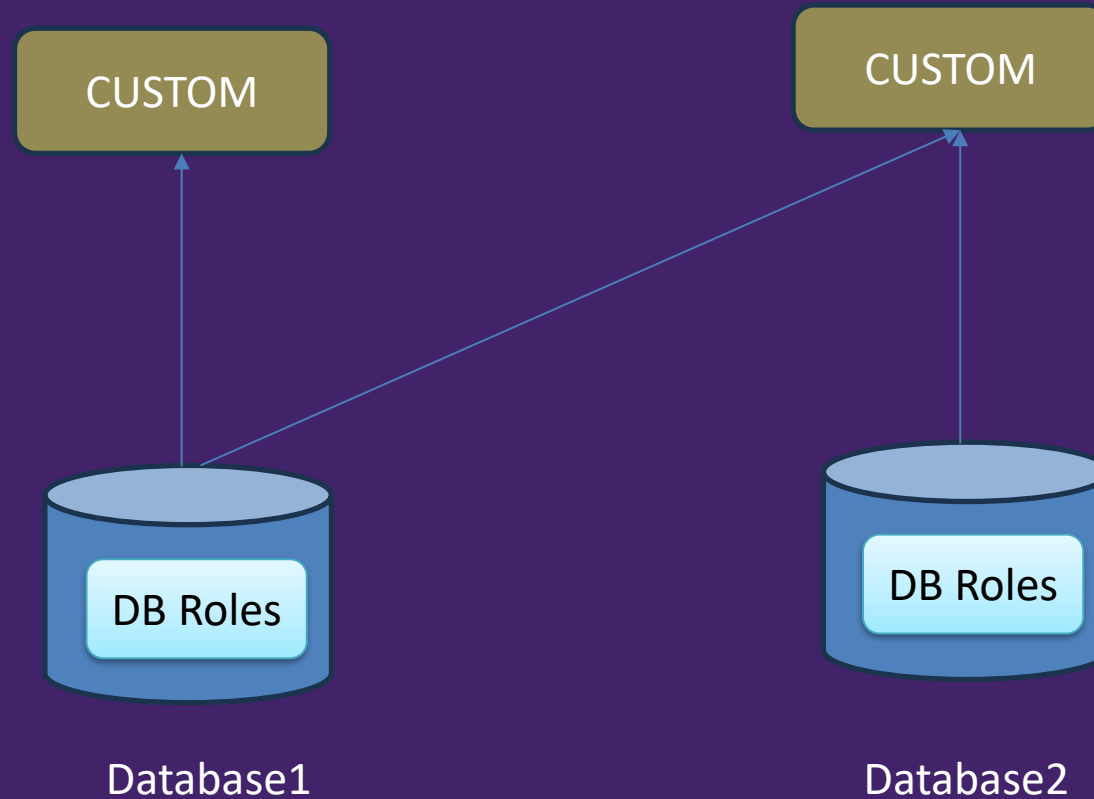
Database Role





Database Role

To limit SQL actions to a single database, as well as any object in the database, grant privileges on the object to a database role in the same database.





Column Level Security





Column Level Security

Dynamic Data Masking (Needs Enterprise Edition or Higher)

- Use masking policies to selectively mask column data

External Tokenization

- Tokenization is the process of removing sensitive data by replacing it with an undecipherable token





Column Level Security

Dynamic Data Masking

- Use masking policies to selectively mask column data

	ID	NAME	EMAIL	CREDIT_CARD_NUMBER	©Learn Anywhere
1	1	John Doe	john.doe@example.com	1234-5678-9012-3456	
2	2	Alex Doe	Alex.doe@example.com	4567-8901-2345-6789	

	...	ID	NAME	EMAIL	CREDIT_CARD_NUMBER	©Learn Anywhere
1		1	John Doe	john.doe@example.com	*****	
2		2	Alex Doe	Alex.doe@example.com	*****	



*(Needs Enterprise
Edition or Higher)*



Row Level Security





Row Level Security



Sales



Marketing



IT

ID	NAME	EMAIL	...	DEPARTMENT	©Learn Anywhere
1	John Doe	john.doe@example.com		Sales	
2	Jane Doe	jane.doe@example.com		Marketing	
3	Peter Smith	peter.smith@example.com		IT	





Row Level Security



ID	NAME	EMAIL	DEPARTMENT	©Learn Anywhere
1	John Doe	john.doe@example.com	Sales	

ID	NAME	EMAIL	...	DEPARTMENT	©Learn Anywhere
1	John Doe	john.doe@example.com		Sales	
2	Jane Doe	jane.doe@example.com		Marketing	
3	Peter Smith	peter.smith@example.com		IT	





Row Level Security



ID	NAME	EMAIL	...	DEPARTMENT	©Learn Anywhere
2	Jane Doe	jane.doe@example.com		Marketing	

ID	NAME	EMAIL	...	DEPARTMENT	©Learn Anywhere
1	John Doe	john.doe@example.com		Sales	
2	Jane Doe	jane.doe@example.com		Marketing	
3	Peter Smith	peter.smith@example.com		IT	





Row Level Security



ID	NAME	EMAIL	...	DEPARTMENT	©Learn Anywhere
2	Jane Doe	jane.doe@example.com		Marketing	

ID	NAME	EMAIL	...	DEPARTMENT	©Learn Anywhere
1	John Doe	john.doe@example.com		Sales	
2	Jane Doe	jane.doe@example.com		Marketing	
3	Peter Smith	peter.smith@example.com		IT	





Multi Factor **Authentication** (MFA)





Multi Factor **Authentication** (MFA)



Password



Device



Finger Print



Location





Multi Factor Authentication (MFA)

- Powered By Duo Security Service
- Managed by Snowflake
- Not enabled by default
- Recommended to have MFA for all ACCOUNTADMIN role
- Ideally for Snowsight but fully supported for Snowsql, JDBC/ODBC drivers

Notifications

☒ Email notifications from [Snowflake](#) [Learn more](#)
☐ When queries finish in the background

Multi-factor authentication

Each time you sign in to Snowflake, you'll use your password and a verification code. [Learn more](#)

Enroll





Multi Factor Authentication (MFA)

©Learn Anywhere



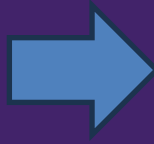
Sign in to Snowflake

Username

Password

[Forgot password](#)

Sign in



©Learn Anywhere

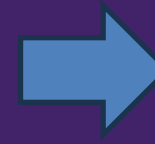


Settings

Send Me a Push

Call Me

Enter a Passcode



Federated Authentication (SSO)





Federated Authentication (SSO)

In a federated environment, user authentication is separated from user access through the use of one or more external entities that provide independent authentication of user credentials.





Federated Authentication (SSO)

Service Provider :

In a Snowflake federated environment, Snowflake serves as the SP.

Identity provider (IdP):

Creating and maintaining user credentials and other profile information.

Authenticating users for SSO access to the SP.

Snowflake supports most SAML 2.0-compliant vendors as an IdP



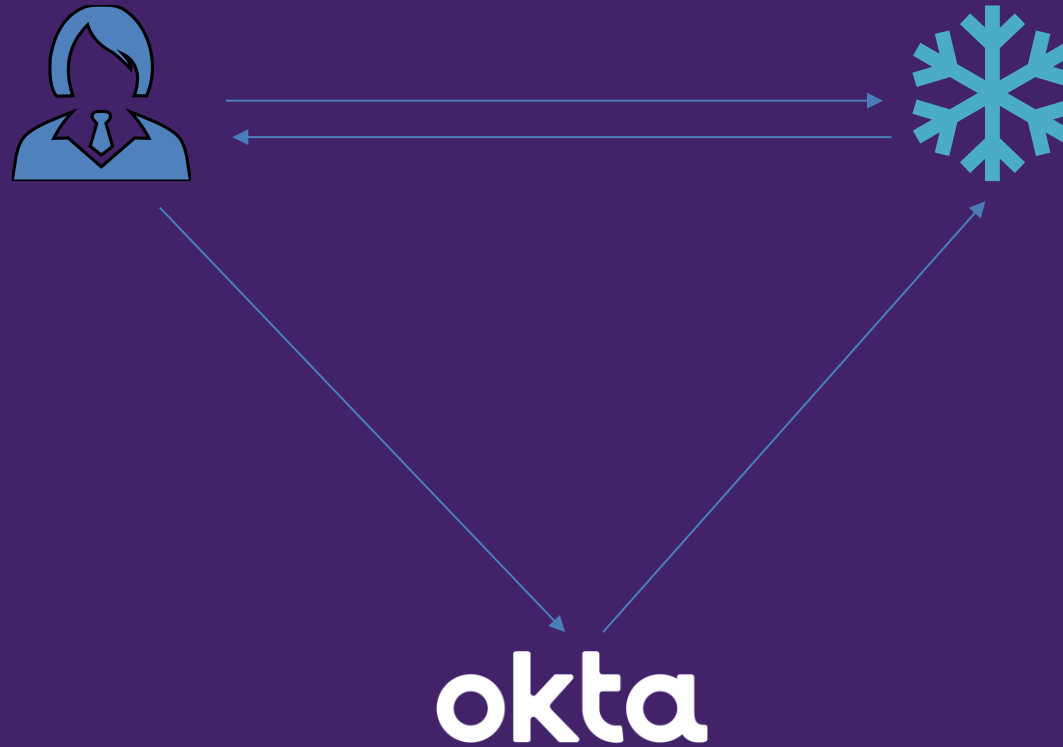
okta





Federated Authentication (SSO)

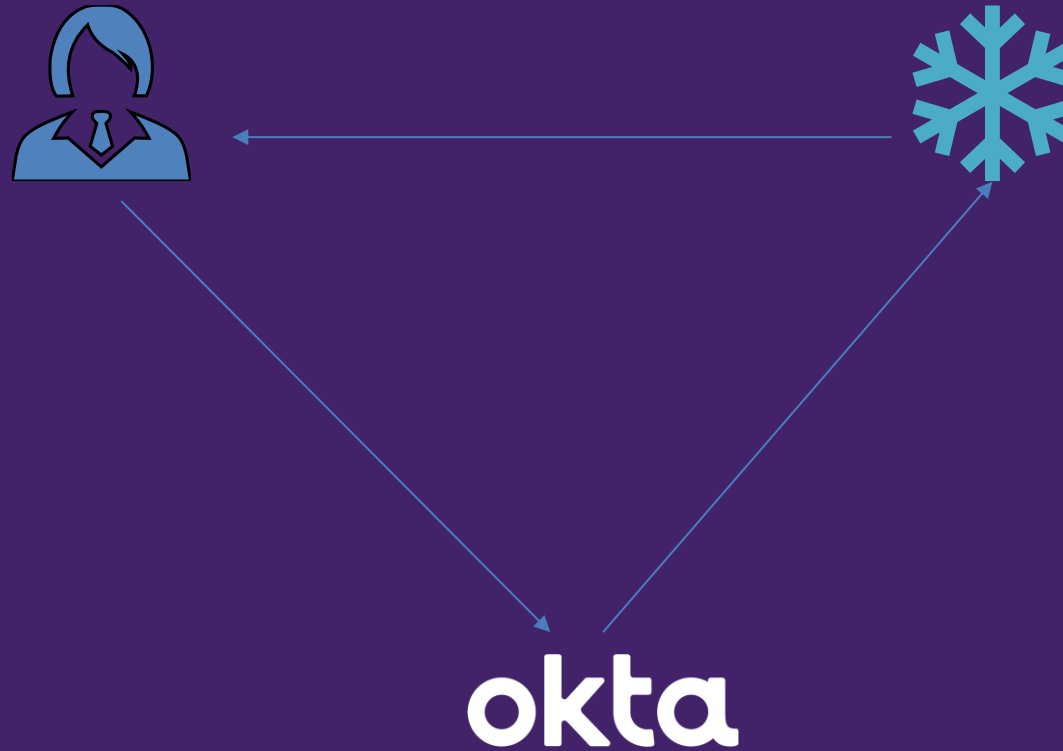
Login Workflow : SF Initiated





Federated Authentication (SSO)

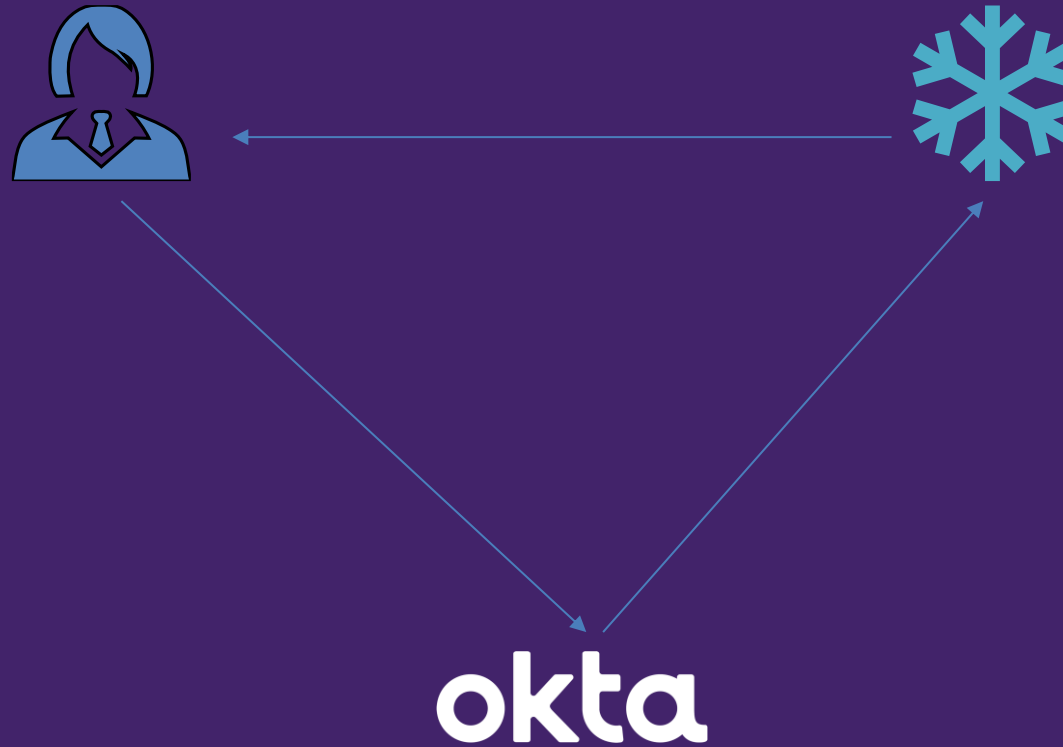
Login Workflow : IdP Initiated





Federated Authentication (SSO)

Logout Workflow : SF Initiated





Key-Pair Authentication





Key-Pair Authentication

Snowflake supports using key pair authentication for enhanced authentication security as an alternative to basic authentication.

This authentication method requires, as a minimum, a 2048-bit RSA key pair.





Key-Pair Authentication

Step 1: Generate the Private Key (OpenSSL)

Step 2: Generate a Public Key (OpenSSL)

Step 3: Store the Private and Public Keys Securely

Step 4: Assign the Public Key to a Snowflake User

Step 5: Verify the User's Public Key Fingerprint

Step 6: Configure the Snowflake Client to Use Key Pair Authentication





Data Encryption





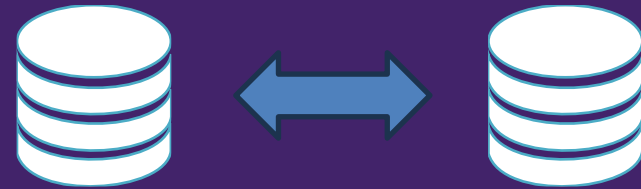
Data Encryption

AT REST



Encrypted with AES 256 Bit

AT TRANSIT



TLS 1.2

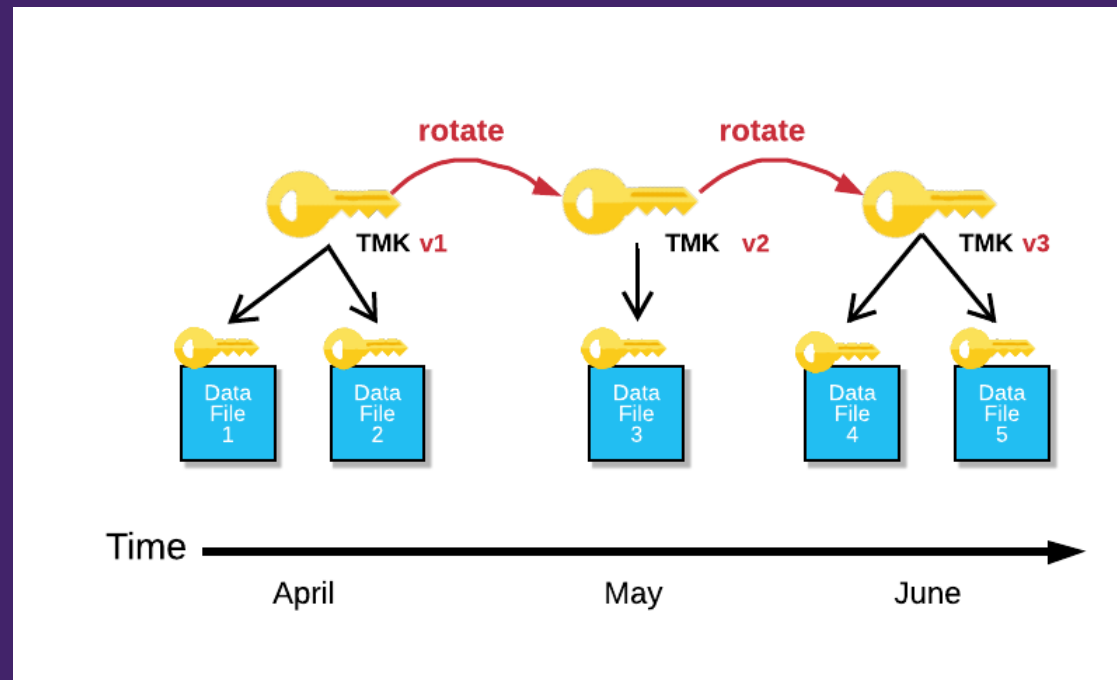




Data Encryption

Periodic Key Rotation

- Snowflake auto-rotates the key every 30 days
- Post 30 days, Active Keys get retired



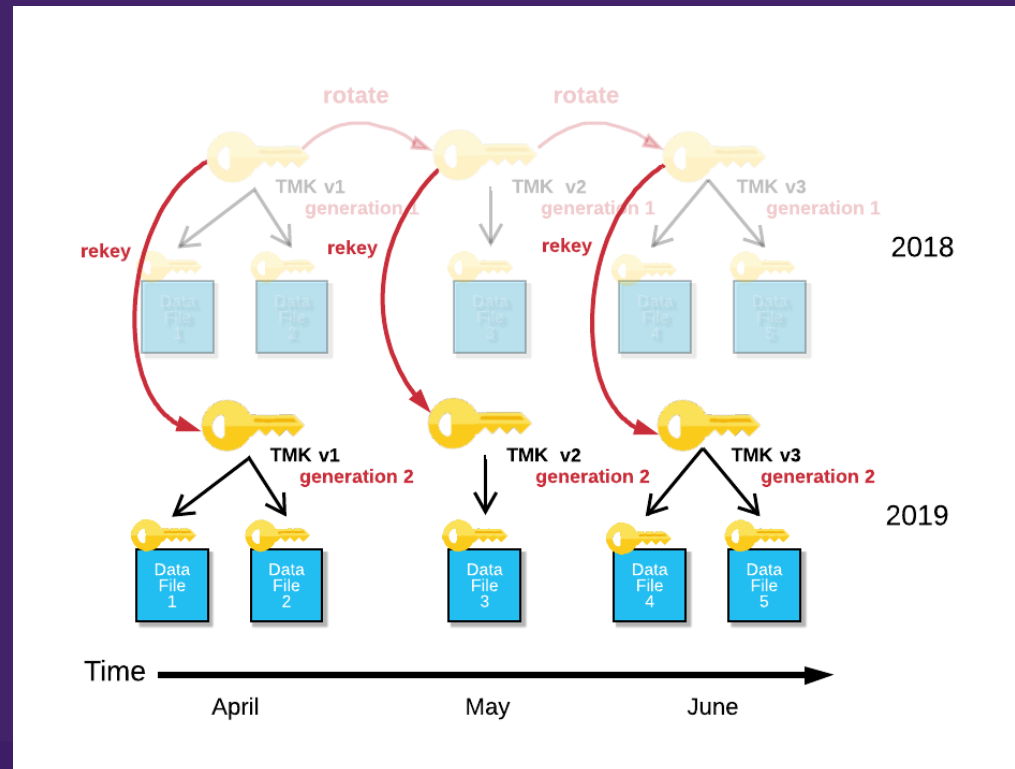
Source: Snowflake



Data Encryption

Periodic Rekeying (Enterprise Edition on Higher)

- Manual Enablement
- Destroys retired key post one year
- Ensures that data is encrypted with latest tech



Source: Snowflake





Data Encryption

Tri-Secret Secure (Business Critical Edition on Higher)

- Combination of CMK and SMK
- Composite Master Key is used
- If CMK is revoked, You cant decrypt data in Snowflake
- Provides 3 levels of data protection: encryption, authentication, and control.



A light blue icon of a globe with a network of lines and nodes.

Network Policies





Network Policies

- Allows to restrict access to SF based on IP address
 - Allowed IP List
 - Blocked IP List
- SECURITYADMIN or Higher (eg ACCOUNTADMIN) can create network policies
- Can be set at ACCOUNT, Security Integration, USER level
- Only one Policy per ACCOUNT/ Security Integration /USER
- Supports CIDR Notation
192.168.1.0/24 >> 192.168.1.0 - 192.168.1.255
- Blocked IP gets preference
- Support only Internet Protocol version 4 (i.e. IPv4) addresses, not IPv6
- Define Network Rule and add to Network policies





Organization & Accounts





Organization & Accounts

An *organization* is :

- First-class Snowflake object that links the accounts owned by the org
- Organizations simplify
 - Account management and billing
 - Replication and Failover/Failback
 - Snowflake Secure Data Sharing and administration tasks.
- Use ORGADMIN Role

ORGADMIN has access to Account properties but not to account data





Organization & Accounts

- Account identifier uniquely identify the SF account

`<account_identifier>.snowflakecomputing.com`



©Learn Anywhere

Welcome to Snowflake!

Your account has been activated.

- Username: **TRAINING**
- Dedicated Login URL: **<https://pmvxjtn-dx34919.snowflakecomputing.com>**

To login, simply visit your dedicated URL above or visit
app.snowflake.com

LOG IN TO SNOWFLAKE





Object Tags





Object Tags

Object tags in Snowflake are schema-level objects that allow you to attach metadata to other Snowflake objects, such as tables, views, and columns. Needs Enterprise Edition or Higher.

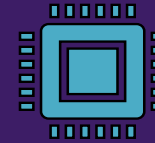




Object Tags



TAG : MKT



TAG : MKT





Object Tags

Purpose of Tags: Tags help in monitoring sensitive data for compliance, discovery, protection, and resource usage. They can be used in both centralized and decentralized data governance approaches.

Tag Creation and Assignment: Tags are schema-level objects assigned to other Snowflake objects with a key-value pair.

Tag Usage and Monitoring: Tags can be queried to monitor usage, facilitate data governance operations, and ensure compliance.

Tag Limits and Management: There are limits on the number of unique tags per object (50). Tags can be inherited based on the Snowflake securable object hierarchy, and tag assignments can be replicated within databases.



Secure Data Sharing





Secure Data Sharing



Data Provider



Data Consumer

- Tables
- External Tables
- Secure Views
- Secure Materialized View
- Secure UDFs





Secure Data Sharing



Data Provider



Data Consumer

- Create a SHARE
- GRANT Access to the SHARE
- Can share data from multiple DBs
- Share with Consumer Account
- Can add new object to share
- Can be revoked anytime
- Pay of the Storage

- Pays for the compute
- A read only DB is created
- Treated just like any object

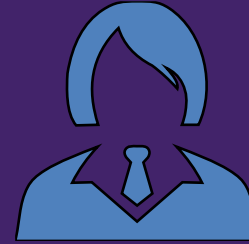




Secure Data Sharing



Data Provider



Readers Account

- Create a Share
- Add Object to share
- GRANT Access to the DB
- Can share data from multiple DBs
- Share with Consumer Account
- Can add new object to share
- Can be revoked anytime
- Pays all cost

- Can Run Queries
- No other access
- Doesn't pay





Secure Data Sharing

Considerations

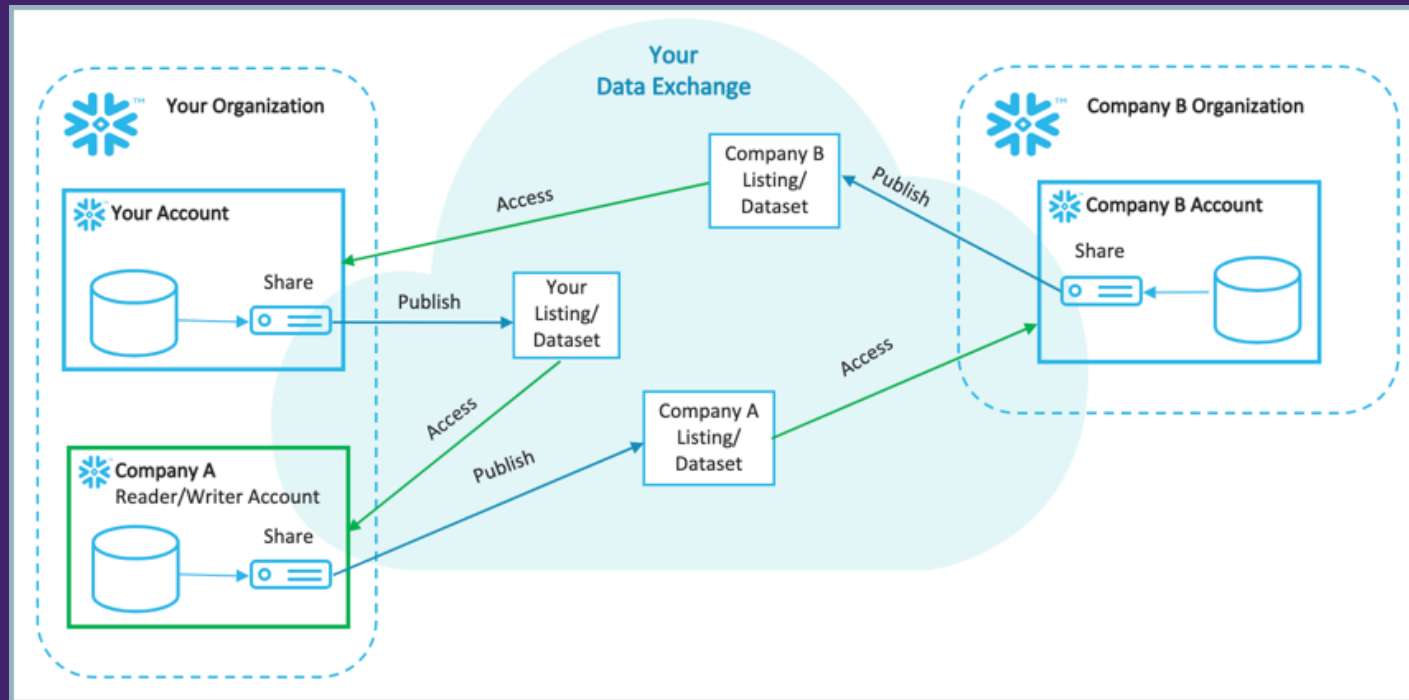
- Shared databases are read-only. Users in a consumer account can view/query data, but cannot insert or update data, or create any objects in the database
- No limit on Number of Shares or Number of account it is shared with
- Can not create a clone of a shared database or any schemas/tables in the database
- Not supported, Time Travel for a shared database or any schemas/tables in the database
- Not supported, Editing the comments for a shared database.
- Shared databases and all the objects in the database cannot be re-shared with other accounts
- Shared databases cannot be replicated
- Can consume as many as shares you want but can create only db per share
- Cant share from Business critical to non-Business critical account, can override settings
- Sharing happens between same cloud provider and same region
- Sharing TO and FROM VPS is not supported



Data Exchange



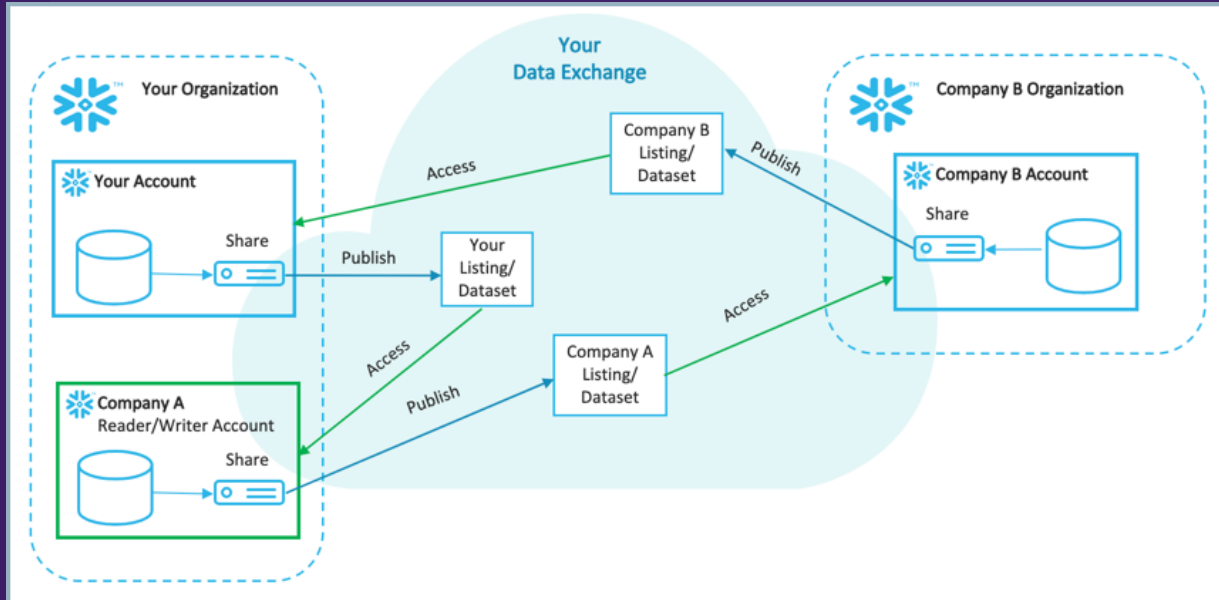
Data Exchange



Source : Snowflake



↔ Data Exchange



- Data shared between invited group of entities
- Contact Support to enable it
- Can share data with partners
- Can share data within the org
- Manage the control

Source : Snowflake







Marketplace

- Provides wide variety of data/apps/services
- Discover data/apps as per you need
- Quick and hassle free access to data/apps
- Native to Snowflake
- Lots of free and freemium offering





Marketplace

- Provides wide variety of data/apps
- Discover data/apps as per you need
- Quick and hassle free access to data/apps
- Native to Snowflake
- Lots of free and freemium offering

The screenshot shows the Snowflake Marketplace user interface. At the top, there is a search bar with the placeholder text "Search providers and data products". To the right of the search bar are links for "Browse Data Products", "Providers", and "My Requests". Below the search bar is a horizontal navigation bar with several filter buttons: "Installed", "Instantly Accessible", "Free Data", "Weather Data", "Financial Data", "360-Degree Customer View", "Demand Forecasting", and "Japan Data". Below the filters is a promotional banner for "Join us at Snowflake Industry Day" with a megaphone icon and the text "Learn how the below data and apps from Snowflake Marketplace solve key industry challenges." Below the banner are four product cards arranged in a 2x2 grid. Each card features a provider logo, the product name, and the provider's name.

Search providers and data products

Browse Data Products Providers My Requests

Installed Instantly Accessible Free Data Weather Data Financial Data 360-Degree Customer View Demand Forecasting Japan Data

Join us at Snowflake Industry Day
Learn how the below data and apps from Snowflake Marketplace solve key industry challenges.

- Acxiom InfoBase® Sample Audience (Synthetic Data)**
Acxiom
- Braze Engagement Benchmarks**
Braze
- CARTO Analytics Toolbox (Core)**
CARTO
- Cybersyn eCommerce Benchmarks**
Cybersyn, Inc



Data Provider





Data Provider

- Data>> Provider Studio
- Need ACCOUNTADMIN or relevant Auth
- Can create and publish listing
- Monitor the performance of listing





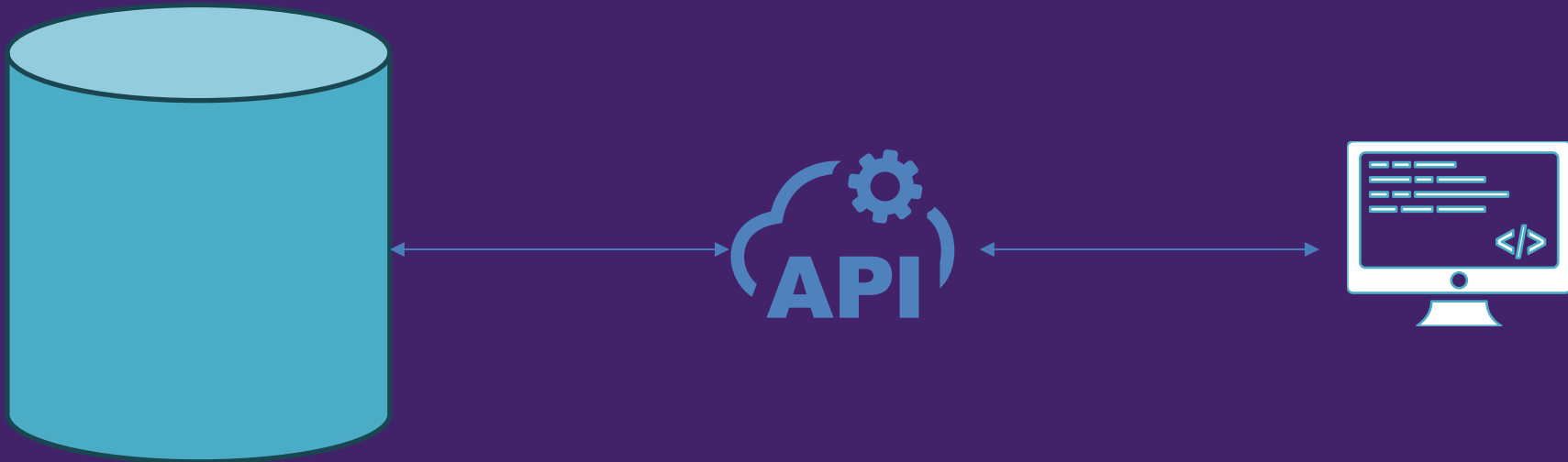


Snowflake SQL API



Snowflake SQL API

The Snowflake SQL API is a REST API that you can use to access and update data in a Snowflake database.



Use OAuth or Key Pair Authentication



Snowflake SQL API

Features

- Submit SQL statements for execution (Queries, DDL,DML)
- Check the status of the execution of a statement
- Cancel the execution of a statement
- Manage Deployments (provision user, role etc.)

Limitation

- PUT and GET are not supported

Billing

- Leverages cloud service layer



Snowflake Drivers & Connectors



Snowflake Drivers & Connectors





Snowflake Scripting





Snowflake Scripting

Snowflake Scripting is an extension to Snowflake SQL that adds support for procedural logic.

You can use Snowflake Scripting to write stored procedures and procedural code outside of a stored procedure.





Snowflake Scripting

DECLARE

... (variable declarations, cursor declarations, etc.) ...

BEGIN

... (Snowflake Scripting and SQL statements) ...

EXCEPTION

... (statements for handling exceptions) ...

END;

©Learn Anywhere

Source: Snowflake





Snowflake Scripting

- Declare Variables
- Return a value
- Can use branching construct, IF and CASE
- Can use Loops, FOR, WHILE, REPEAT , LOOP
- Can use CURSOR
- Can use RESULTSET
- Can determine number of rows impacted by DML, (SQLROWCOUNT, SQLFOUND, SQLNOTFOUND)
- Handles Exception





Snowflake Scripting

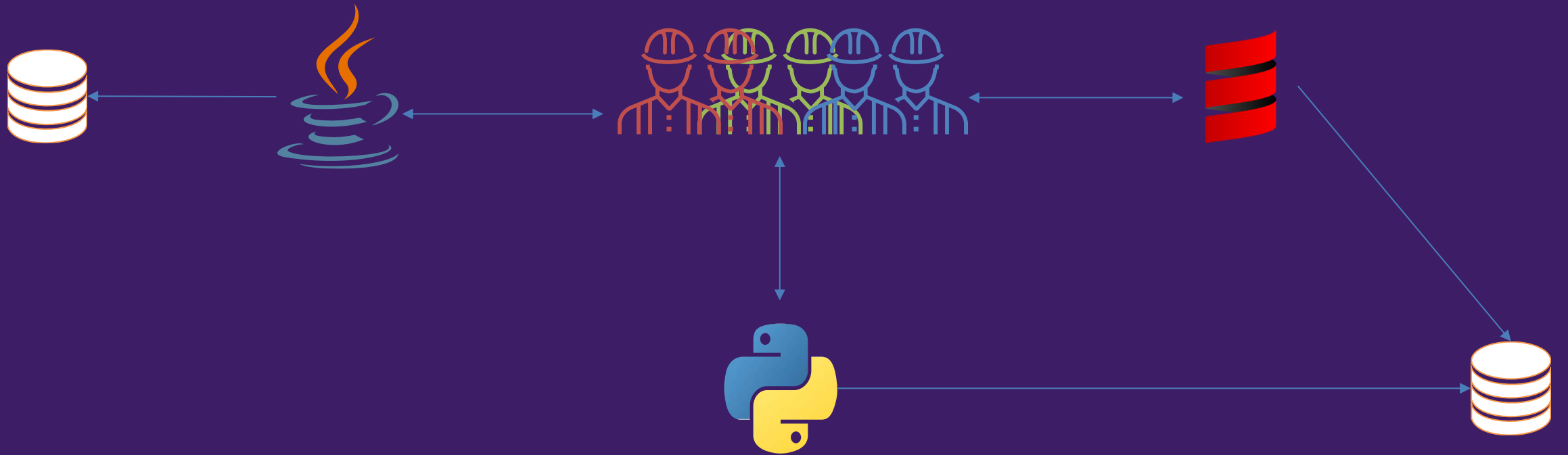
```
CREATE OR REPLACE PROCEDURE area()  
  RETURNS FLOAT  
  LANGUAGE SQL  
  AS  
    DECLARE  
      radius FLOAT;  
      area_of_circle FLOAT;  
  BEGIN  
    radius := 3;  
    area_of_circle := PI() * radius * radius;  
    RETURN area_of_circle;  
  END;
```

©Learn Anywhere

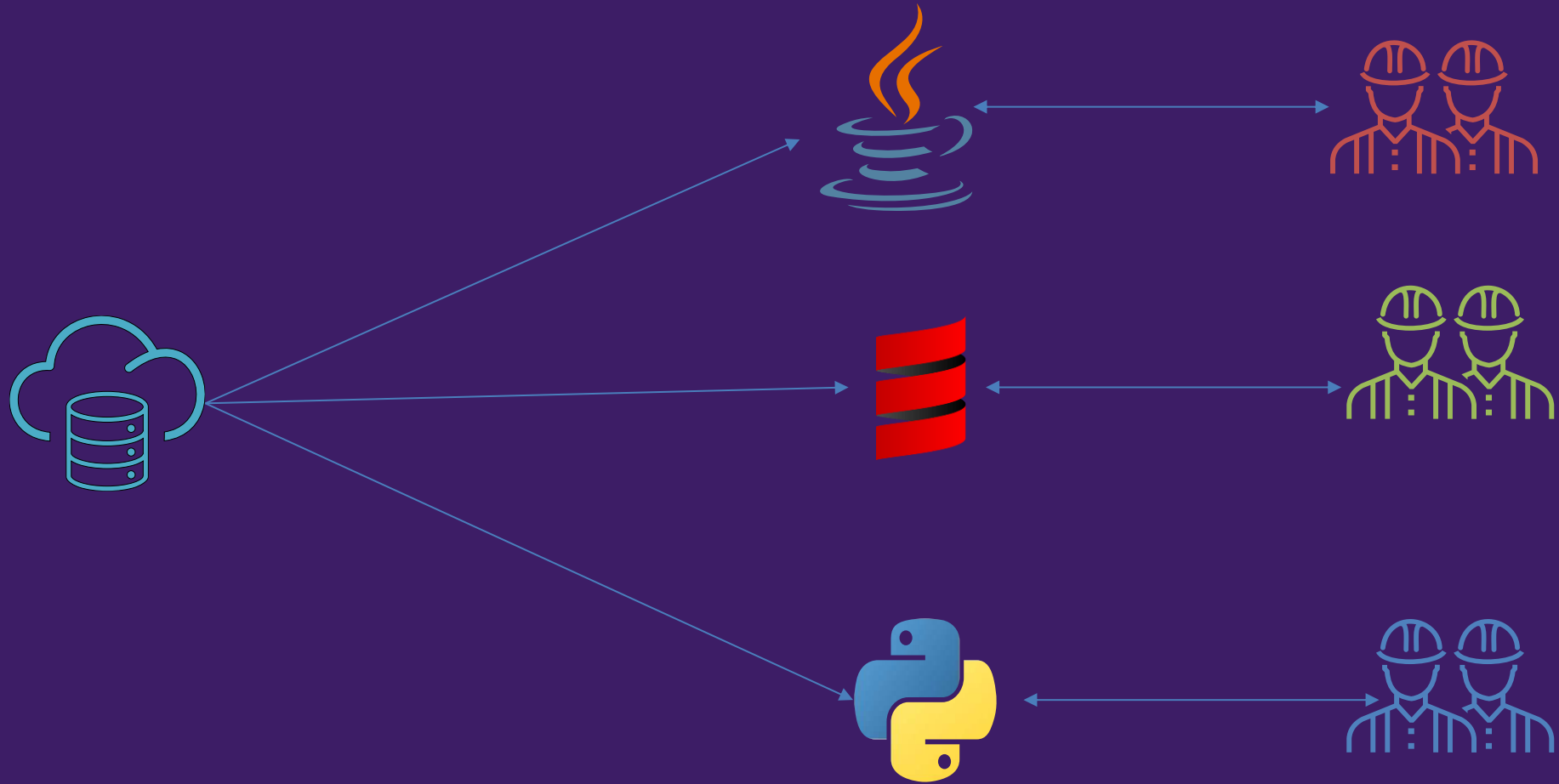




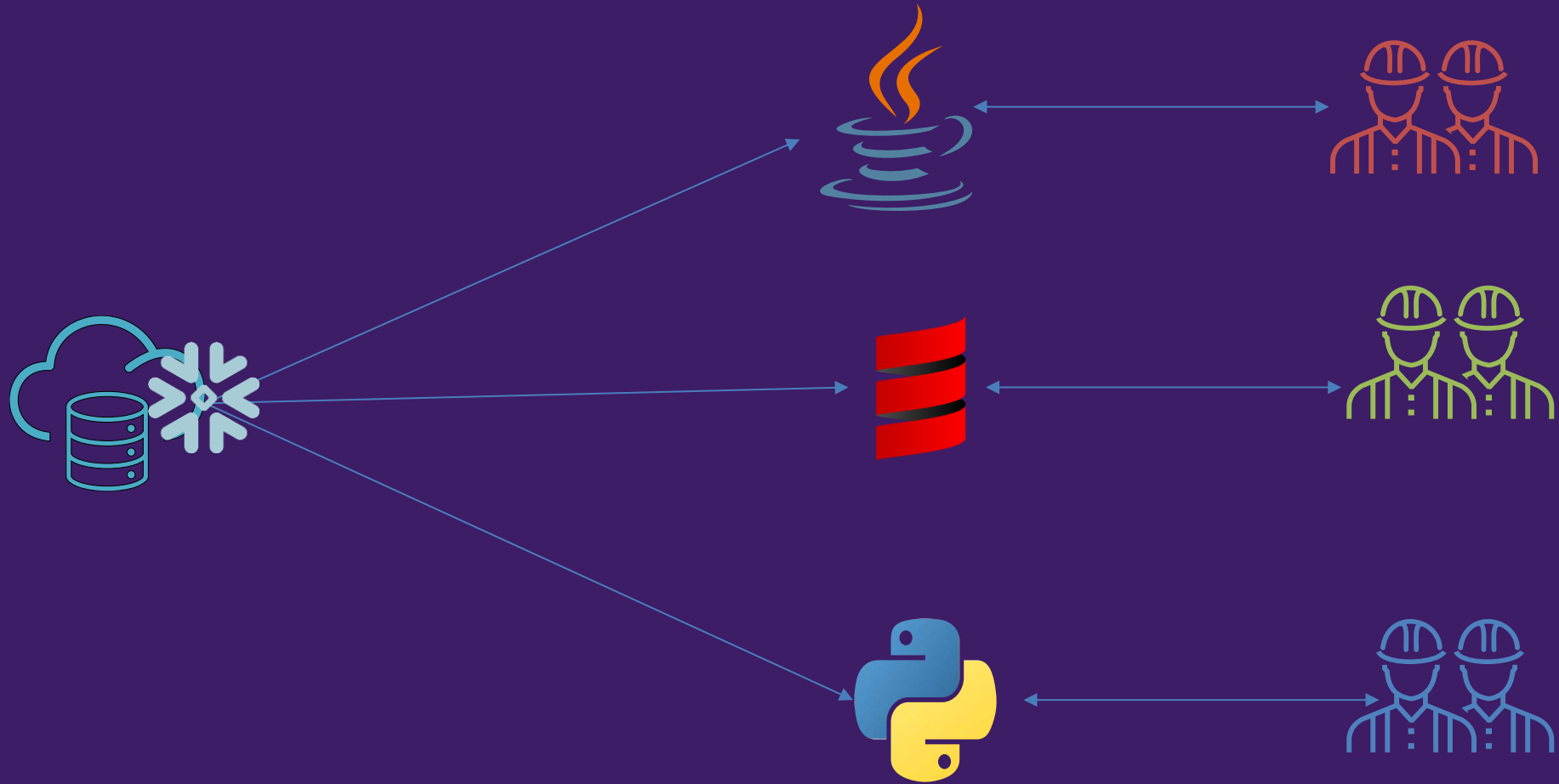
SnowPark



SnowPark



SnowPark



Replication



Replication



AWS : US West (Oregon)

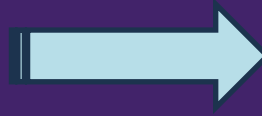


Replication

PRIMARY DB



AWS : US West (Oregon)



SECONDARY DB



AZURE : UAE North (Dubai)

External tables
Event tables
Stages
Pipes
DBs created from SHARE



Replication

PRIMARY DB



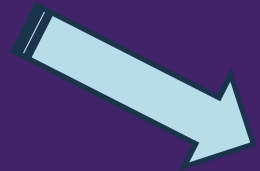
AWS : US West (Oregon)



SECONDARY DB



AZURE : UAE North (Dubai)



SECONDARY DB



GCP : EU WEST2 (London)



Replication

- Only for DB objects
- Privileges granted on DB are not replicated
- Error when replicating to lower edition (Can overcome)
- Can replicate to multiple accounts
- Data transfer and Data Compute cost



Replication

Feature	Standard	Enterprise	Business Critical	VPS
Database replication	✓	✓	✓	✓
Share replication	✓	✓	✓	✓
Replication Group	✓	✓	✓	✓
Account object (other than database and share) replication			✓	✓
Failover Group			✓	✓
Data protected with Tri-Secret Secure			✓	✓

Source: Snowflake



Replication

PRIMARY DB



AWS : US West (Oregon)



SECONDARY DB



AZURE : UAE North (Dubai)

External tables

Event tables

Stages

Pipes

DBs created from SHARE

Hybrid Table

Apave Iceberg Table



Snowflake AI and ML



Snowflake AI and ML

Snowflake ML provides functionality for you to build your own models.

Snowflake Cortex is a suite of AI features that use large language models (LLMs) to understand unstructured data, answer freeform questions, and provide intelligent assistance. This suite of Snowflake AI Features comprises:

- **Snowflake Cortex LLM Functions**
- Snowflake Copilot
- Document AI
- Cortex Fine-tuning
- Cortex Search
- Cortex Analyst



Snowflake Cortex LLM Function

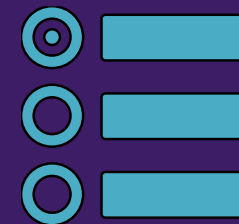
Snowflake Cortex LLM Function Provides access to industry-leading LLMs hosted and managed by Snowflake. Like Meta, Mistral, Reka etc.



COMPLETE Function



Task-Specific Function



Helper Function



Snowflake Cortex LLM Function



COMPLETE Function

General-purpose function for tasks like sentiment classification, synthetic data generation, and customized summaries.

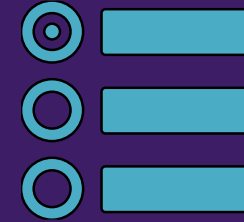
`COMPLETE()`



Task-Specific Function

Task-specific functions are purpose-built and managed functions that automate routine tasks.

`CLASSIFY_TEXT()`
`PARSE_DOCUMENT()`
`TRANSLATE()`



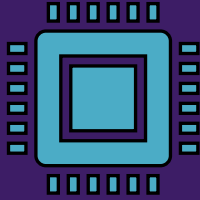
Helper Function

Helper functions are purpose-built and managed functions that reduce cases of failures when running other LLM functions.

`COUNT_TOKENS()`
`TRY_COMPLETE()`



Snowflake Billing



Compute

- User Managed VW
- Snowflake Managed VW
- Cloud Services



Storage

- Data Staging
- Database Tables
- Materialized View
- Time Travel and Fail Safe



Data Transfer Cost

- Data Transfer

Billing depends on Snowflake Edition, Cloud provider and Region
Check <https://www.snowflake.com/pricing/>



Cost Optimization



+ Create

Home

Search

Projects

Data

Data Products

AI & ML

Monitoring

Admin

Cost Management

Warehouses

Compute Pools

Users & Roles

Accounts

Security

Contacts

Billing & Terms

Cost Management

Organization Overview Account Overview Consumption Budgets Resource Monitors

Account spend for **HAB96338** ⓘ from **Apr 6 - Apr 13** ▾

\$0.67
Spend in currency

0.50
Spend in credits

\$4
Compute price/credit ⓘ

\$0.08
Average daily cost

Top warehouses by cost

View All >

🔒 COMPUTE_WH		0.50
🔒 CLOUD_SERVICES_ONLY		0.00

Cost insights ⓘ

We checked for the following insight types but did not find savings opportunities:

- ✓ Large tables that are never queried
- ✓ Tables where data written but not read
- ✓ Rarely used materialized views
- ✓ Short-lived permanent tables
- ✓ Rarely used search optimization paths
- ✓ Rarely used tables with automatic clustering
- ✓ Inefficient usage of multi-cluster warehouse autoscale

Cost Optimization

Insight: Large tables that are never queried.

Recommendation :

- Drop the table
- Save on storage
- Make sure table is not used only for backup



Cost Optimization

Insight: Tables over 100GB from which data is written but not read.

Recommendation :

- Drop the table
- Save on storage and future compute
- Make sure table is not used only for backup



Cost Optimization

Insight: Rarely used materialized views, less than 10 times/week

Recommendation :

- Drop the View
- Save on storage and future compute
- Make sure table is not used only for backup



Cost Optimization

Insight: Short-lived permanent tables, identifies > 100GB tables which are deleted in 24 hours

Recommendation :

- Consider using temp or transient table
- Save on failsafe and Time travel costs



Cost Optimization

Insight: Rarely used search optimization paths, used < 10 times/week.

Recommendation :

- Consider removing search optimization path
- Make sure table is not used only for backup



Cost Optimization

Insight: Rarely used auto clustering, used < 100 times/week.

Recommendation :

- Consider disabling auto clustering
- Make sure table is not used only for backup
- Used by other accounts via share



Cost Optimization

Insight: Identifies min and max cluster count as same

Recommendation :

- Consider lowering the min value



Cost Optimization



Cost Optimization

Type	Description	Use Cases
Permanent Table	Default table type with full data recovery features like Time Travel and Fail-safe.	Long-term data storage, historical data analysis, and critical data that requires recovery options.
Transient Table	Similar to permanent tables but without Fail-safe and with limited Time Travel.	Temporary data storage for ETL processes or intermediate data that doesn't need long-term recovery.
Temporary Table	Exists only within the current session and is automatically dropped afterward.	Session-specific data, staging data for transformations, or testing queries without persistence.
External Table	References data stored outside Snowflake, such as in cloud storage.	Querying semi-structured or unstructured data without loading it into Snowflake.
View	A virtual table that represents the result of a query.	Simplifying complex queries, combining data from multiple tables, or restricting access to sensitive data.
Materialized View	A pre-computed dataset derived from a query and stored for faster access.	Improving query performance for repeated patterns, speeding up aggregation, projection, and selection operations.



Cost Optimization



Cost Optimization

Cost Consideration: When moving data among regions with replication and Fail-over.

Data Transfer:

- Initial Replication
- Subsequent sync

Compute Resources :

- To determine the delta
- To copy the data

Storage Cost :

- Target account storage

Control Cost :

- Selecting which object to be replicated
- Adjust the refresh frequency
- Monitoring the cost



Cost Optimization



Cost Optimization

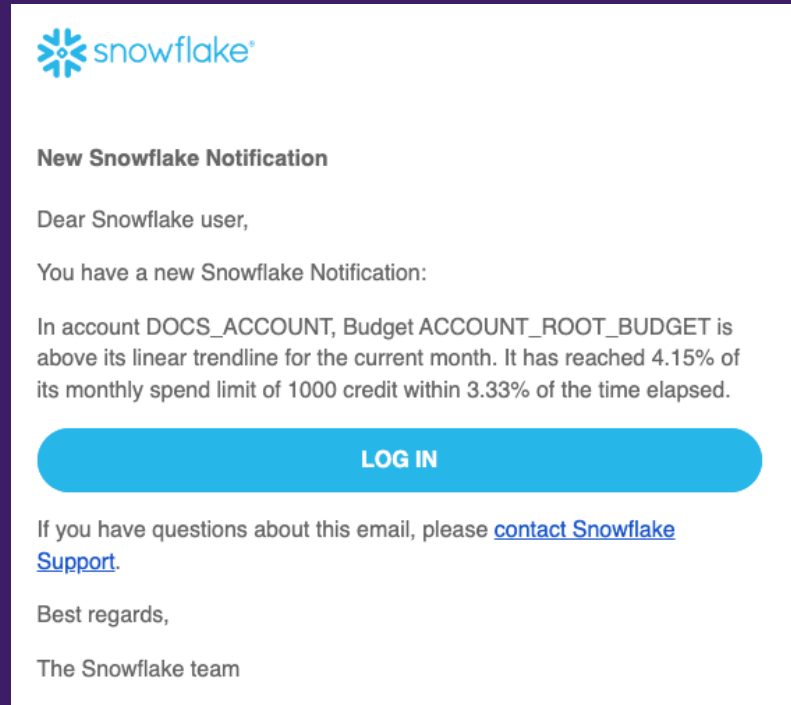
Monitor and Control Cost

Resource Monitor:

- Setup on Warehouse
- Can notify and suspend VW

Budget :

- At Account level
- Sends notification (also has forecasting feature)



Cost Optimization



Cost Optimization

Attribute Costs

Tagging

For logical bifurcation and chargeback for accounting purpose.
You can use **query tag** or **object tag**.

To find cost use ACCOUNT_USAGE Views

- TAG_REFERENCES
- WAREHOUSE_METERING_HISTORY
- QUERY_ATTRIBUTION_HISTORY



Thank You!

Thank you for taking this course, please do leave a rating for us!

